WEEK 37 · FRONTIER · v1.0 · BUILT, TESTED, CPU-PROVEN

Universal AWQ Quantization Playbook

Any open model, from 1B to 1T parameters → right-sized rented GPU → 4-bit AWQ → evaluated, tuned, exported, shipped. Every command, every decision, no sugarcoating.

What this is	The end-to-end operating manual and code tour for `github.com/kashyapphassan2000-netizen/WEEK--37`. It covers the whole flow: pick a model, size the hardware (AI-assisted or by formula), get tokens, rent the GPU, connect (SSH / VS Code / WinSCP), install the stack, run AWQ, evaluate, tune, export and hand back the model.
Built on	Your *AWQ From Scratch — Week 37 Build Guide* (40 pp). I kept its algorithm and day-by-day spirit, fixed the bugs I found in it (Section 2) and rebuilt the pipeline so it works on any size of model, not just models that fit on one GPU.
Status	**Built.** The code is in the repo, 30 unit and integration tests pass, and the full pipeline (gates 00–05 plus tuning) ran end to end on a real model (Qwen2.5-0.5B, CPU). The measured numbers are in Section 19. Every file is explained in Section 18 and printed in full in the Appendix.
Date	18 October 2026. GPU prices were checked this month (RunPod page dated 8 Oct 2026) and they change every month, so re-check before you rent.

BRUTAL TRUTH The build machine had **no GPU**. Everything in Section 19 was measured on a 4-core CPU with Qwen2.5-0.5B. That proves correctness and the full workflow, not GPU speed. 7B, 70B and 1T numbers come from **your** rented-GPU run (template: `proof/GPU_RUN_TEMPLATE.md`). Nothing in the repo is a number that wasn't measured. Estimates are labelled as estimates.

REMOVED The AISIL / ISO 26262 / automotive appendix in the original PDF is dropped. This playbook is purely AI engineering.

Contents

0	The flow at a glance, and what "universal" honestly means
1	Brutal truths before you spend a rupee
2	Audit of the original Week-37 PDF: bugs found and what changes
3	Step 1: Choose your model (HF Hub, gated Llama/Gemma, ModelScope, your own model)
4	Step 2: Size the hardware. Option A: AI-assisted prompt. Option B: manual formula
5	Step 3: Accounts, tokens and model access
6	Step 4: Rent the GPU (providers, prices, decision map, budget)
7	Step 5: Connect (SSH keys, VS Code Remote-SSH, WinSCP, tmux)
8	Step 6: Install the stack (driver, CUDA, PyTorch: what to install and what NOT to)
9	Step 7: The codebase (every file and what it does, in plain words)
10	Step 8: Transfer and run (the exact command sequence)
11	Step 9: Evaluation (what to measure and pass/fail gates)
12	Step 10: Hyperparameter tuning (the knobs that actually raise accuracy)
13	Step 11: Export, give back, shut down
14	Proof of work: what lands in GitHub
15	Troubleshooting
16	Command master sheet (one page)
17	Status: what is done, what is not (honest)
18	Code walkthrough: every file in plain words
19	Proof: measured results (CPU run, Qwen2.5-0.5B) and tuning
20	Hardware reports for reference models (1B → 1T)
A	Appendix: full source code

0. The flow at a glance

1 · Pick model
HF id, gated repo, ModelScope, or your own folder. Needs original 16-bit or FP8 weights, never GGUF.

2 · Size hardware
AI prompt (Option A) or formula (Option B), cross-checked by `01_inspect_model.py`.

3 · Access
HF token (read/write), accept license, `hf auth login`.

4 · Rent GPU
Pick provider + GPU + count from the decision map. Attach a volume.

5 · Connect
SSH key → VS Code Remote-SSH and/or WinSCP. Start `tmux`.

6 · Install
Check driver, create venv, install the right PyTorch wheel, run `00_env_check.py`.

7 · Transfer code
`git clone` (best) or drag the folder with WinSCP.

8 · Download weights
`hf download` to the volume (not the container disk).

9 · Baseline
FP16 + naive RTN perplexity. This is the number to beat.

10 · Quantize
Block-by-block: calibrate → scale search → clip search → pack INT4.

11 · Verify
Logit check vs FP16, real VRAM, real size on disk.

12 · Evaluate
PPL (WikiText-2, C4), lm-eval tasks, generation side-by-side, speed.

13 · Tune
Group size, calibration data, samples, skip-layers. Tuned on a held-out split, never the test set.

14 · Export
Native format + standard AWQ (vLLM-loadable) where the architecture allows.

15 · Give back
Push to HF (private first), download via WinSCP, write the model card.

16 · Shut down
Stop, then delete the pod *and* the volume. Check the billing page.

What "universal" honestly means

Works	Works with caveats	Does not work (and why)
Any decoder-only text LLM that loads with `AutoModelForCausalLM`: Llama 2/3/3.1/3.2/3.3, Mistral, Mixtral, Qwen 2/2.5/3, Gemma 2/3, Phi-3/4, OLMo, Falcon, GPT-NeoX, StarCoder2, DeepSeek-style... at any size, because the pipeline streams one decoder block at a time.	**MoE** (Mixtral, Qwen-MoE, DeepSeek-V3, Kimi K2): experts are calibrated with the tokens actually routed to them, and router layers stay 16-bit. **FP8 checkpoints** (DeepSeek-V3/R1, Kimi K2): dequantized block by block while loading. **Custom code** (`trust_remote_code`): works if the model is built from a repeated list of blocks containing `nn.Linear`. **Vision-language models**: only the language tower gets quantized. **Your own PyTorch model**: via a small plug-in function (Section 3.4).	**GGUF / Ollama files**: they're already quantized. Re-quantizing stacks the errors, so go back to the original weights. **Already-quantized repos** (GPTQ, AWQ, bnb, MLX 4-bit): same reason. **Encoders, diffusion, audio**: the AWQ math applies, but calibration and eval are different. Not in v1. **"Better than FP16"**: quantization can't improve the base model. Tuning only shrinks the gap.

1. Brutal truths before you spend a rupee

1. **AWQ is weight-only 4-bit.** It shrinks weights about 3.5–4× and memory at inference roughly 3×. It does **not** make a from-scratch PyTorch implementation fast. Speed comes from fused kernels (vLLM / SGLang AWQ-Marlin). That's why export to the standard AWQ format (Section 13) matters.
2. **Small models get hurt more.** A 1B model typically loses 5–10% perplexity at 4-bit. A 70B model typically loses 1–3%. Code, math and long-context tasks degrade more than perplexity suggests, so always test on *your* task.
3. **The FP16 baseline is often the most expensive part.** Quantizing 70B needs one 80 GB GPU (block-by-block). *Evaluating* 70B in FP16 needs about 170 GB of VRAM (2–3 GPUs). For a 1T model, an FP16 baseline isn't practical. You compare against the published numbers instead, and the guide says so out loud.
4. **"Trillion-parameter" is a logistics problem before it's a GPU problem.** Kimi-K2-class means about 1 TB to download, 2–4 TB of disk, many hours of quantization and an 8-GPU node just to *serve* the result. Budget hundreds of dollars, not $3.
5. **An AI assistant's hardware estimate is a draft, not a decision.** LLMs mis-read configs and hallucinate parameter counts. Option A is fast. Option B and `01_inspect_model.py` are the truth.
6. **Tuning on the test set is cheating.** All tuning in this playbook uses a held-out calibration-style split. WikiText-2 test and lm-eval are run once, at the end.
7. **Spot / community GPUs get reclaimed.** The pipeline writes a checkpoint after every block and resumes automatically. Use `tmux`. Keep your work on a persistent volume.
8. **Idle GPUs burn money.** The most common beginner cost isn't compute. It's forgetting to delete a pod or a volume.

2. Audit of the original Week-37 PDF

I read all 40 pages. The math (Sections 1–2 of the original) is right and the 7-day structure is good. These are the issues that would bite you on a real run, and what the new code does about each:

#	Problem in the original	Consequence → fix in this project
1	Loads the **whole FP16 model onto one GPU** for collection, search and eval.	Caps out around 30B on an 80 GB card, so it can't do 70B, 405B or 1T. **Fix:** block-by-block streaming. Only one decoder block plus its calibration activations sit on the GPU. Weights are read shard by shard from `safetensors`.
2	Calibration loader uses `padding="max_length"`.	**Crashes on Llama tokenizers** (they have no pad token), and where it doesn't crash it feeds pad tokens into the statistics. **Fix:** concatenate texts and cut exact `seq_len` windows, with no padding.
3	The AWQ VRAM figure is measured after loading FP16 to the GPU and swapping layers, without resetting peak stats.	The reported "AWQ VRAM" is really the FP16 peak, so the "3× drop" claim can't be measured this way. **Fix:** build the quantized model on the `meta` device, load packed weights only, reset peak stats, then measure.
4	Scales are searched per single Linear, and `1/s` is applied at runtime in every layer.	Correct, but slow, and it can't be exported to the standard AWQ format (vLLM expects `1/s` folded into the previous norm/linear). **Fix:** Linears that share the same input tensor (q/k/v, gate/up) are detected automatically and get one scale per group. That scale is folded into the producer for known families (Llama, Mistral, Qwen2/3, Gemma*, Mixtral attention). Unknown families keep a runtime `inv_s` (always correct, native format only).
5	No weight-clipping search. Symmetric only. Normalises `s` by its mean.	Leaves accuracy on the table compared with the official AWQ. **Fix:** add the AWQ clip search, an asymmetric (zero-point) mode (the default, and the standard format) and the official `s/√(max·min)` normalisation.
6	Excludes the output head by the name `"lm_head"`.	GPT-NeoX calls it `embed_out` and others call it `output`, so they would get quantized by accident. **Fix:** only quantize Linears *inside* the repeated decoder blocks. Routers and tiny layers are skipped automatically.
7	The "expected" numbers (e.g. AWQ PPL 9.97, 0.74 GB) are illustrative.	Putting them in a blog or README as results would be false. **Fix:** the repo reports only measured JSON, with the hardware, commit hash and timestamp.
8	`huggingface-cli login`	Deprecated in current `huggingface_hub`. **Fix:** `hf auth login`, `hf download`, `hf upload`.
9	No resume and no checkpointing.	A reclaimed spot pod at hour 5 of a 70B run means starting over. **Fix:** per-block checkpoints and `--resume`.
10	No MoE, FP8, custom-code or own-model support.	**Fix:** Section 0 table, plus the plug-in hook in 3.4.

KEPT The derivation (`y = (x/s)·(W·diag(s))ᵀ`, `s = mean|x|^α`, α grid search), the pack/unpack roundtrip test, the forward-verify gate and the day-by-day pass/fail discipline.

3. Step 1: Choose your model

3.1 Where models come from

Source	How you reference it	Notes
Hugging Face Hub, public	`Qwen/Qwen2.5-7B-Instruct`	No license click needed. Easiest place to start.
Hugging Face Hub, gated	`meta-llama/Llama-3.1-8B-Instruct`, `google/gemma-2-9b-it`	Open the model page, accept the license and wait for approval (minutes to a day). Your token must belong to the account that was approved.
Meta direct (llama.com)	Signed download URL → local folder	Use the HF version instead unless you have a reason not to. The direct download comes in Meta's own format and has to be converted to HF format first.
ModelScope (China mirror)	`modelscope download --model qwen/Qwen2.5-7B-Instruct --local_dir ./m`	Same weights. Useful when HF is slow or blocked. Pass the local folder to the pipeline.
Your own fine-tune / private model	`/workspace/models/my-model` (folder)	The folder must contain `config.json`, `*.safetensors` (or `pytorch_model*.bin`, which gets auto-converted) and the tokenizer files. Upload it with WinSCP or push it to a **private** HF repo.
Ollama / LM Studio / GGUF	Not a valid source	Find the original HF repo the GGUF was made from and use that.

3.2 Read the model card in 60 seconds

Open `config.json` on the model page (Files tab). These are the fields that decide your hardware:

Field	Meaning in plain words	Llama-3.1-8B
`num_hidden_layers` (L)	How many repeated blocks. The pipeline processes them one at a time.	32
`hidden_size` (h)	Width of the token vector.	4096
`intermediate_size` (I)	Width inside the MLP. Usually the biggest activation.	14336
`num_key_value_heads`, `head_dim`	Decide the KV-cache size at inference.	8, 128
`vocab_size`, `tie_word_embeddings`	Embedding / output head size. These stay 16-bit.	128256, false
`num_experts` / `n_routed_experts`, `num_experts_per_tok`, `moe_intermediate_size`	Present only for MoE models.	(dense)
`torch_dtype`, `quantization_config`	bf16/fp16 is good. fp8 is OK (dequantized on load). If it already says gptq/awq/bnb, stop: wrong source.	bfloat16

3.3 Compatibility gate (run before renting anything)

```
# runs on your laptop, CPU only, downloads ~10 KB of metadata (config + safetensors headers), no weights
python scripts/01_inspect_model.py --model meta-llama/Llama-3.1-8B-Instruct
```

It prints the exact parameter count (read from the safetensors headers), architecture, dense/MoE, checkpoint dtype, the number of quantizable Linears, a PASS/FAIL compatibility verdict and the full hardware table from Section 4. **If it says FAIL, don't rent a GPU.**

3.4 Your own (non-HF) PyTorch model

AWQ needs three things: the model, a list of the repeated blocks, and a way to feed it calibration inputs. You give it those in one small Python file:

```
# my_plugin.py: the pipeline imports this when you pass --plugin my_plugin.py
def load():                      # return (model, tokenizer_or_None)
    ...
def blocks(model):               # return the nn.ModuleList of repeated blocks
    return model.transformer.layers
def calib_batches(tokenizer, n, seq_len):   # yield model inputs (dicts or tensors)
    ...
```

BRUTAL TRUTH If most of your model's compute isn't `nn.Linear` (for example a convolution-heavy model), AWQ is the wrong tool.

4. Step 2: Size the hardware

You need three answers: **which GPU**, **how many**, and **for how long**, and you need them separately for the three phases. They have very different needs:

Phase	What sits in VRAM	Driven by
A. Quantize	One decoder block + calibration activations for that block	Largest block size, `n_samples × seq_len`, MLP width
B. FP16 baseline eval	The whole model in 16-bit + eval activations	Total parameters × 2 bytes
C. Evaluate / serve AWQ	The whole model in 4-bit + KV cache	Total parameters × ~0.52 bytes + context length

4.1 Option A: AI-assisted (paste this prompt)

Paste this into ChatGPT, Claude, Gemini or any assistant that can browse. Replace the first line. If it can't open links, paste the model's `config.json` under the prompt.

```
MODEL: <paste Hugging Face link or model id here>

You are a GPU capacity planner. I will quantize this model to 4-bit AWQ
(group size 128, asymmetric) with a block-by-block pipeline: only ONE decoder
block plus its calibration activations is on the GPU at a time.
Calibration = 128 samples x 512 tokens. Usable VRAM = 90% of nominal.

1. Read config.json (and model.safetensors.index.json if present). QUOTE these
   values, do not estimate them: architecture, total parameters, dense or MoE
   (num experts, experts per token, moe_intermediate_size), num_hidden_layers,
   hidden_size, intermediate_size, num_key_value_heads, head_dim, vocab_size,
   tie_word_embeddings, checkpoint dtype (bf16/fp16/fp8), max context length,
   whether trust_remote_code is required.
2. Show the arithmetic for:
   a) FP16 size = params x 2 bytes
   b) AWQ size = quantized-linear params x 0.52 bytes + embeddings/lm_head x 2 bytes
   c) Largest decoder block in FP16 (all experts included for MoE)
   d) Quantize-phase VRAM = 2 x block + 6 x 128 x 512 x max(I, top_k x moe_I) bytes + 2 GB
   e) FP16-baseline-eval VRAM = FP16 size x 1.2
   f) AWQ serving VRAM at 8192 context, batch 1 = AWQ size + KV cache + 15%
      where KV = 2 x layers x kv_heads x head_dim x 8192 x 2 bytes
   g) Disk = download size + AWQ size + 20 GB;  CPU RAM >= 3 x block + 2 x calib act + 8 GB
   NOTE: quantize phase = exactly ONE GPU; serving with vLLM = 1, 2, 4 or 8 GPUs.
3. For each phase pick GPU type AND count from: RTX 4090 24GB, L40S 48GB,
   A100 80GB, H100 80GB, H200 141GB, B200 180GB.  count = ceil(VRAM / usable).
4. Estimate wall time: quantize ~= 3 min per billion params on A100-class
   (x1.5 on RTX 4090), plus download at 200 MB/s, plus 30 min setup/eval.
   Estimate cost with these $/GPU-hour: RTX4090 0.34, L40S 0.79, A100 1.19,
   H100 1.99, H200 3.59, B200 5.98 (RunPod community, Oct 2026).
5. List anything that breaks a standard AutoModelForCausalLM pipeline
   (custom code, FP8 weights, multimodal, MoE, already-quantized weights).
Output: one table per phase + a final "rent this" line. If you could not read
the files, say so instead of guessing.
```

VERIFY Run `python scripts/01_inspect_model.py --model <id>` and compare. If the AI's parameter count or VRAM differs by more than 10%, trust the script. The script reads the real tensor shapes.

4.2 Option B: Manual formula (pen and paper)

```
P      = total parameters (billions)            ← model card / 01_inspect_model.py
P_emb  = vocab x hidden (x2 if lm_head is NOT tied) / 1e9
P_lin  = P - P_emb                               ← the part AWQ quantizes
L, h, I, top_k, I_moe                            ← config.json

FP16 size (GB)          = 2 x P
AWQ size  (GB)          = 0.52 x P_lin + 2 x P_emb      (4 bits + scale/zero per 128 weights ≈
4.16 bits)
Block (GB)              = 2 x P_lin / L                  (for MoE: count ALL experts in one
layer)
Calib act (GB)          = n_samples x seq_len x max(I, top_k x I_moe) x 2 / 1e9

VRAM quantize (GB)      = 2 x Block + 6 x Calib act + 2
VRAM FP16 eval (GB)     = 1.2 x FP16 size
KV cache (GB)           = 2 x L x kv_heads x head_dim x context x batch x 2 / 1e9
VRAM serve AWQ (GB)     = 1.15 x (AWQ size + KV cache)

# GPUs for a phase      = ceil( VRAM_phase / (0.9 x VRAM of one GPU) )
Disk (GB)               = download size + AWQ size + 20
CPU RAM (GB)            ≥ max(16, 3 x Block + 2 x Calib act + 8)     (streaming loader)
# Phase A uses ONE GPU (a block is never split across cards). Phase C (vLLM) uses 1, 2, 4 or 8
GPUs.
Hours (estimate)        ≈ P x 3 min / 60  (A100-class; x1.5 RTX 4090)  + download + 0.5
Cost                    = Σ phases ( #GPUs x $/GPU-hr x hours ) x 1.25   (setup + idle margin)
```

ESTIMATE The "3 min per billion parameters" speed is my estimate for this pure-PyTorch implementation with clip search on. The first real GPU run calibrates it, and v1.0 of this PDF will carry the measured value.

4.3 Worked example: Llama-3.1-8B

```
P = 8.03   P_emb = 128256 x 4096 x 2 / 1e9 = 1.05   P_lin = 6.98   L = 32   I = 14336
FP16 size         = 16.1 GB
AWQ size          = 0.52 x 6.98 + 2 x 1.05 = 3.63 + 2.10 = 5.7 GB      (2.8x smaller on disk)
Block             = 2 x 6.98 / 32 = 0.44 GB
Calib act         = 128 x 512 x 14336 x 2 / 1e9 = 1.88 GB
VRAM quantize     = 0.88 + 11.3 + 2 = ~14 GB     → 1x RTX 4090 24 GB
VRAM FP16 eval    = 1.2 x 16.1 = 19.3 GB         → 1x RTX 4090 (tight) or 1x L40S 48 GB (safe)
KV @ 8k           = 2 x 32 x 8 x 128 x 8192 x 2 / 1e9 = 1.07 GB
VRAM serve AWQ    = 1.15 x (5.7 + 1.07) = 7.8 GB → fits a 12 GB consumer card
Disk              = 16.1 + 5.7 + 20 = ~42 GB  (rent a 60–80 GB volume for headroom)
Time / cost       = ~2–3 h on 1x RTX 4090 @ $0.34–0.89/h  →  ≈ $1–3
```

4.4 The decision map (generated by 01_inspect_model.py from real configs)

Model	Type	Params	FP16	AWQ	A. Quantize on (VRAM)	B. FP16 eval on	C. Serve AWQ on (8k ctx)	A hours*	A cost*
Qwen/Qwen2.5-0.5B	dense	0.5 B	1 GB	0.5 GB	1× RTX 4090 (5.9 GB)	1× RTX 4090 (1.2 GB)	1× RTX 4090 (0.6 GB)	0.54	$0.23
Qwen/Qwen2.5-7B-Instruct	dense	7.6 B	15 GB	5.6 GB	1× RTX 4090 (17.8 GB)	1× RTX 4090 (18.3 GB)	1× RTX 4090 (7.0 GB)	1.09	$0.46
unsloth/Meta-Llama-3.1-8B-Instruct	dense	8.0 B	16 GB	5.7 GB	1× RTX 4090 (14.1 GB)	1× RTX 4090 (19.3 GB)	1× RTX 4090 (7.8 GB)	1.12	$0.48
Qwen/Qwen3-30B-A3B	MoE 128e top-8	30.5 B	61 GB	16.8 GB	1× RTX 4090 (9.3 GB)	4× RTX 4090 (73.3 GB)	1× RTX 4090 (20.3 GB)	2.86	$1.22
Qwen/Qwen2.5-32B-Instruct	dense	32.8 B	66 GB	19.3 GB	1× L40S (25.7 GB)	4× RTX 4090 (78.6 GB)	2× RTX 4090 (24.7 GB)	2.64	$2.61
unsloth/Llama-3.3-70B-Instruct	dense	70.6 B	141 GB	39.8 GB	1× L40S (28.0 GB)	8× RTX 4090 (169.3 GB)	1× A100 80GB (48.9 GB)	5.11	$5.04
mistralai/Mixtral-8×22B-v0.1	MoE 8e top-2	140.6 B	281 GB	73.7 GB	1× L40S (24.9 GB)	5× A100 80GB (337.5 GB)	2× A100 80GB (86.9 GB)	9.68	$9.56
deepseek-ai/DeepSeek-V3	MoE 256e top-8, FP8 ckpt	671.0 B	1342 GB	351.8 GB	1× A100 80GB (62.5 GB)	multi-node	8× A100 80GB (405.3 GB)	35.01	$52.07
moonshotai/Kimi-K2-Instruct	MoE 384e top-8, FP8 ckpt	1046.5 B	2093 GB	547.6 GB	1× H200 (85.0 GB)	multi-node	4× B200 (630.5 GB)	31.0	$139.11

Generated from `proof/hardware_reports/*.json` (exact parameter counts from each model's real config). Cost and hours are ESTIMATES at RunPod community prices; the quantize-speed constant (3 min per billion params on an A100) is uncalibrated until a GPU run. "FP16 eval: multi-node" means the 16-bit baseline does not fit one 8-GPU node: use `--skip-fp16` and compare against published scores.

5. Step 3: Accounts, tokens and model access

5.1 Hugging Face token

1. Create an account at `huggingface.co/join` and verify your email.
2. Go to **Settings → Access Tokens → Create new token**.
   - **Read** token: enough to download, including gated models you've been approved for.
   - **Write** token (or fine-grained with "write to repos under your namespace"): needed only for Step 11 upload.
3. Gated models (Llama, Gemma, some Mistral): open the model page → fill in the form → **Agree and access repository**. Wait for "You have been granted access".

```
# on the GPU server (each new pod has its own home folder, so log in again there)
pip install -U "huggingface_hub[cli]"
hf auth login                       # paste the token; answer 'n' to git credential
hf auth whoami                      # must print your username
# non-interactive alternative (scripts / CI):
export HF_TOKEN=hf_xxxxxxxxxxxxxxxx
```

SECURITY Never paste the token into code, `configs/*.yaml` or a notebook you commit. The repo's `.gitignore` blocks `.env`. Revoke the token after the project if you shared the machine.

5.2 Download the weights (to the volume, not the container disk)

```
export HF_HOME=/workspace/hf_cache              # cache on the persistent volume
hf download meta-llama/Llama-3.1-8B-Instruct \
   --local-dir /workspace/models/llama-3.1-8b-instruct \
   --include "*.json" "*.safetensors" "tokenizer*"
du -sh /workspace/models/llama-3.1-8b-instruct   # ≈ FP16 size from Section 4
```

Error	Cause → fix
`401 / GatedRepoError`	License not accepted, or the token is from a different account. Accept the license, wait, then run `hf auth whoami`.
`No space left on device`	You downloaded to the container disk. Set `HF_HOME` and `--local-dir` under `/workspace`.
Very slow	`pip install -U hf_xet` (fast transfer backend). Large models: start the download first, in tmux, while you set up everything else.

5.3 Your own model from your laptop

Either drag the folder to `/workspace/models/` with WinSCP (Section 7.4), or push it privately and download it on the pod:

```
hf upload <you>/my-model ./my-model . --private     # from laptop
hf download <you>/my-model --local-dir /workspace/models/my-model   # on pod
```

6. Step 4: Rent the GPU

6.1 Price sheet (on-demand, USD per GPU-hour, checked Oct 2026)

Provider	RTX 4090 24G	L40S 48G	A100 80G	H100 80G	H200 141G	B200 180G	Notes
RunPod (community / secure)	0.34 / 0.89	0.79 / 1.09	1.19–1.39 / 1.79	1.99 PCIe, 2.69 SXM / 2.89–3.99	3.59 / 5.29	5.98 / 7.99	Per-second billing. Volume $0.10/GB/mo running, $0.20 idle. Cheapest smooth option. Page updated 8 Oct 2026.
Vast.ai (marketplace)	~0.14–0.40	varies	~0.80–1.50	from ~1.47	varies	varies	Cheapest, but host quality varies. Filter by reliability above 98% and a verified badge. Interruptible = cheapest and can be killed.
Hyperstack	—	L40 1.00	1.35 PCIe / 1.60 SXM	1.90 PCIe / 2.40 SXM	check	check	Per-minute billing. VM with user `ubuntu`. Attach a public IP and open port 22. Use only `hyperstack.cloud` (a fake clone site exists).
Lambda	—	—	2.79 (SXM)	3.29 PCIe / 3.99 SXM	—	6.69	Reliable, no egress fees, drivers preinstalled. Prices went up in 2025–26.
Jarvislabs	—	—	1.49	2.69–2.99	—	—	Per-minute billing, INR billing. India-friendly.
E2E Networks (India)	—	from ₹49/h*	≈ ₹180–250/h	₹362/h (official)	check	—	INR, **+18% GST**. Data centres in Delhi NCR and Chennai. *Entry price is for lower-tier GPUs.
AWS / GCP / Azure	—	—	—	~$3–12 per GPU-hr	—	—	Mostly sold as 8-GPU nodes. Only worth it if you have credits or an employer account.
Colab / Kaggle	Free T4 16 GB (session-limited)						Good for ≤3B models and for learning the pipeline at $0.

Sources: runpod.io/gpu-cloud/pricing (8 Oct 2026); hyperstack.cloud/pricing; Lambda list prices via usagepricing.com (Jun 2026); Vast.ai via costbench.com and hostinger.com (Sep 2026); jarvislabs.ai blog; e2enetworks.com H100 page. Third-party trackers disagree. Treat every figure as indicative and check the live page before you rent.

6.2 Which provider, honestly

- **Default:** RunPod community cloud. Cheap, per-second billing, a PyTorch template with drivers ready, persistent volumes, and an easy SSH + SFTP setup. Use **secure** cloud for long 70B+ runs where getting interrupted costs more than the price difference.
- **Lowest price, and you'll babysit it:** Vast.ai. Check the host's disk speed and network before you start a large download.
- **A real VM with root and stable IPs:** Hyperstack or Lambda.
- **Paying in INR / GST invoice:** E2E Networks or Jarvislabs.
- **400B–1T:** you need a provider with H200/B200 nodes and multi-TB volumes: RunPod (secure), Lambda or Hyperstack. Ask about volume size limits *before* renting.

6.3 Renting on RunPod (reference walkthrough)

1. runpod.io → Billing → add $10–25 credit (set a **spending limit**).
2. Settings → SSH Public Keys → paste your public key (Section 7.1).
3. Pods → Deploy → choose the GPU and count from the decision map → template **RunPod PyTorch** (CUDA 12.8+).
4. Edit template: Container disk 30 GB. **Volume disk** = Disk from Section 4 (e.g. 80 GB for 8B, 250 GB for 70B), mounted at `/workspace`. Expose TCP port 22.
5. Deploy → wait for RUNNING → **Connect** → copy the **"SSH over exposed TCP"** command (it has a public IP and a port).

IMPORTANT RunPod's basic proxied SSH (`ssh xxx@ssh.runpod.io`) does **not** support SFTP/SCP, so **WinSCP will fail** with it. Use "SSH over exposed TCP" (public IP + port).

7. Step 5: Connect

7.1 Create an SSH key (one time, on your laptop)

```
# Windows PowerShell (also works on macOS/Linux terminal)
ssh-keygen -t ed25519 -C "kashyap-gpu"          # press Enter for default path; set a passphrase
type $env:USERPROFILE\.ssh\id_ed25519.pub       # Windows: print PUBLIC key → paste into
provider
cat ~/.ssh/id_ed25519.pub                       # macOS/Linux equivalent
```

The **.pub** file is public, so paste it into the provider. The file **without** .pub is your private key. Never share it or upload it anywhere.

7.2 First connection (terminal)

```
ssh -i ~/.ssh/id_ed25519 root@<PUBLIC_IP> -p <PORT>     # RunPod / Vast: user root
ssh -i ~/.ssh/id_ed25519 ubuntu@<PUBLIC_IP>              # Lambda / Hyperstack: user ubuntu,
port 22
nvidia-smi                                               # you should see your GPU(s)
```

7.3 VS Code Remote-SSH (edit and run on the GPU like it's local)

```
# add to  C:\Users\<you>\.ssh\config   (or ~/.ssh/config)
Host awq-gpu
    HostName <PUBLIC_IP>
    Port <PORT>
    User root
    IdentityFile ~/.ssh/id_ed25519
```

In VS Code: install the **Remote - SSH** extension → F1 → *Remote-SSH: Connect to Host* → `awq-gpu` → Open Folder `/workspace`. The integrated terminal now runs on the GPU box.

7.4 WinSCP (drag-and-drop files, Windows)

1. Install WinSCP from **winscp.net** (the official site only).
2. **New Session**: File protocol **SFTP** · Host name `<PUBLIC_IP>` · Port `<PORT>` · User name `root` (or `ubuntu`).
3. **Advanced... → SSH → Authentication → Private key file** → pick `id_ed25519`. WinSCP offers to convert it to `.ppk`. Click OK and save it.
4. **Save** → **Login** → accept the host key (first time only).
5. Left pane is your PC, right pane is the server. Go to `/workspace` on the right and drag the project folder (or your model folder) across.
6. **Transfer settings → Binary** for model files. Exclude `.venv` and `__pycache__`.

WINDOWS GOTCHA Shell scripts edited on Windows get CRLF line endings, and bash then fails with `/bin/bash^M: bad interpreter`. Fix it on the server with `sed -i 's/\r$//' *.sh`, or set VS Code's line ending to LF (bottom-right "CRLF" → "LF"). The repo ships a `.gitattributes` that forces LF.

Mac/Linux: no WinSCP. Use `scp -P <PORT> -r ./WEEK--37 root@IP:/workspace/` or `rsync -avP -e "ssh -p <PORT>" ./WEEK--37 root@IP:/workspace/`.

RECOMMENDED For *code*, use `git clone` on the server instead of WinSCP, so the server always has the exact committed version. Use WinSCP for things that aren't in git: your own model folders, results you want back on your laptop, and quick edits.

7.5 tmux: jobs survive a disconnect

```
tmux new -s awq          # start a session and run long jobs in here
# detach: Ctrl+B then D        list: tmux ls        re-attach: tmux attach -t awq
```

BRUTAL TRUTH If you run a 6-hour quantization in a plain SSH window and your Wi-Fi drops, the job dies. Always run it in tmux.

8. Step 6: Install the stack

8.1 Driver vs CUDA toolkit vs PyTorch (what you actually need)

Piece	What it is	Do you install it?
NVIDIA driver	Lets the OS talk to the GPU. `nvidia-smi` shows it, and "CUDA Version: 12.x" in its corner = the **maximum** CUDA runtime the driver supports.	**No** on RunPod, Vast, Lambda and Hyperstack GPU images. It's preinstalled. Only on a bare VM (8.4).
CUDA runtime	The libraries PyTorch calls.	**No.** They come inside the PyTorch wheel (`cu126` , `cu128` ...).
CUDA toolkit (`nvcc`)	A compiler for custom CUDA kernels.	**No** for this project (pure PyTorch). Only if you build kernels from source.
PyTorch	The framework.	**Yes.** Pick a wheel whose CUDA version is ≤ what `nvidia-smi` shows. Blackwell GPUs (B200, RTX 5090, RTX Pro 6000) need `cu128` or newer.

8.2 Install (copy-paste, Ubuntu GPU image)

```
nvidia-smi                                          # 1. note GPU name + "CUDA Version"
apt-get update && apt-get install -y git tmux htop nvtop python3-venv   # 2. tools (sudo on VMs)
cd /workspace
git clone https://github.com/kashyapphassan2000-netizen/WEEK--37.git    # 3. code
cd WEEK--37
python3 -m venv .venv && source .venv/bin/activate                       # 4. isolated env
pip install -U pip wheel
pip install torch --index-url https://download.pytorch.org/whl/cu128    # 5. match driver
pip install -e ".[eval]"                                                 # 6. project + lm-eval
python scripts/00_env_check.py                                           # 7. must end with ENV
OK
```

`00_env_check.py` prints the Python, torch and CUDA versions, GPU names and VRAM, bf16 support, free disk on `/workspace` , RAM, HF login status and a 4096×4096 matmul timing. If anything is wrong it tells you the exact fix.

8.3 Required software list

Package	Why
`torch` ≥ 2.4	Tensors, GPU, the model.
`transformers` , `accelerate`	Model definitions, building the skeleton on the meta device.
`safetensors` , `huggingface_hub[cli]` , `hf_xet`	Reading and writing weights shard by shard, download/upload.
`datasets`	Calibration (pile-val, C4) and eval (WikiText-2, C4) data.
`pyyaml` , `tqdm` , `numpy`	Config, progress bars, stats.
`lm-eval` (extra)	Downstream accuracy tasks.
`vllm` (optional)	Fast serving and checking the exported standard-AWQ model. Install it in a **separate** venv because it pins its own torch.
`pytest` , `ruff` (dev)	Tests and lint.

8.4 Bare VM with no driver (only if `nvidia-smi` is "not found")

```
sudo apt-get update && sudo apt-get install -y ubuntu-drivers-common
sudo ubuntu-drivers list --gpgpu          # shows recommended server driver
sudo ubuntu-drivers install --gpgpu       # Hopper/Blackwell: use the "-open" server driver it
suggests
sudo reboot
nvidia-smi
```

BRUTAL TRUTH Hand-installing drivers on a rented VM wastes an hour of paid GPU time. Pick the provider's image that already has CUDA or PyTorch.

9. Step 7: The codebase

Everything lives at the root of the WEEK--37 repo. Each file has one job. In v1.0 of this PDF every file is printed in full with a line-by-line, plain-English explanation. This page is the map.

```
WEEK--37/
├── configs/
│   ├── run.yaml              ← THE FILE YOU EDIT (model, work_dir, knobs)
│   ├── gpus.yaml             ← GPU catalog + prices used for sizing/cost (edit for your
provider)
│   ├── tune_space.yaml       ← tuner stages (GPU)        tune_space_cpu.yaml ← small space used
for the CPU proof
│   └── examples/             ← qwen2.5-0.5b-cpu, qwen2.5-7b, llama-3.1-8b, llama-3.3-70b,
qwen3-30b-a3b-moe, deepseek-v3
├── awq/                      ← the library
│   ├── config.py   cli.py   utils.py
│   ├── hardware.py           ← Section-4 formulas as code + GPU picker
│   ├── model_io.py           ← meta skeleton, lazy safetensors reader, FP8, fused experts,
load_quantized
│   ├── blocks.py             ← blocks / quantizable Linears / experts / fold targets for ANY
architecture
│   ├── calib.py              ← calibration + eval windows (no padding)
│   ├── quantizer.py          ← 4-bit math, pack/unpack
│   ├── search.py             ← AWQ scale search + clip search
│   ├── awq_linear.py         ← AwqLinear, AwqExpertsWeight
│   ├── pipeline.py           ← block-by-block loop, checkpoints, --resume
│   ├── evaluate.py           ← PPL, RTN baseline, generations, speed, lm-eval bridge
│   ├── export.py             ← standard AWQ GEMM (vLLM) export + reference decoder
│   └── plugin.py             ← your own non-transformers model
├── scripts/  00_env_check  01_inspect_model  02_baseline  03_quantize  04_verify
│             05_eval  06_tune  07_export  08_publish  run_all.sh
├── tests/    30 tests (tiny Llama / Qwen2-GQA / Mixtral-MoE / custom plugin model)
├── proof/    cpu-qwen2.5-0.5b/ (real logs + JSON), hardware_reports/, GPU_RUN_TEMPLATE.md
└── docs/     Universal_AWQ_Playbook.pdf (this), playbook/ (source + build.py), derivation.md,
blog.md
```

9.1 How the algorithm runs (plain words)

1. **Embed the calibration text.** 128 windows × 512 tokens go through the embedding layer. The result is the input to block 0.
2. **For each block i** (only this block is on the GPU):
   a. Load block i's weights from disk → GPU.
   b. Run the block once with hooks to record the input of every Linear. Linears that receive the *same* tensor (q/k/v; gate/up; each expert's gate/up) form a **group**.
   c. For each group: per-channel importance `m = mean|x|`. Try 20 values of α ∈ [0,1]: `s = m^α` (normalised), fake-quantize `W·diag(s)`, compute the group output with `x/s`, measure the error vs FP16. Keep the best `s`.
   d. **Clip search:** for each Linear, try shrinking the max-abs value per group (100%→50%) and keep what minimises output error. This tames outlier weights.
   e. Fold `1/s` into the previous norm or Linear when it's known. Otherwise store `inv_s` in the layer. Then pack INT4 + scales (+ zeros).
   f. Run the quantized block on the calibration data and log its error vs. the FP block (`block_rel_mse`). The input to block i+1 is the FP block's output (official AWQ behaviour, `search.propagate: fp`), or

the quantized output (`propagate: quantized`, GPTQ-style, a tuning knob). Save a checkpoint, free the GPU, move on.

3. Write `model.safetensors` shards + `quant_config.json` + a copy of the tokenizer and config.

9.2 The single config file

```
# configs/run.yaml
model: Qwen/Qwen2.5-7B-Instruct      # HF id OR local folder (/workspace/models/my-model)
revision: main
trust_remote_code: false             # true only for models that ship their own code (DeepSeek,
Kimi…)
dtype: bfloat16                      # compute dtype during calibration (float32 on CPU)
device: cuda                         # cuda | cpu
quant:
  w_bit: 4
  group_size: 128                    # 128 standard; 64 = more accurate, ~3% bigger
  zero_point: true                   # asymmetric (standard AWQ); false = symmetric
  skip_modules: []                   # names kept 16-bit; routers are skipped automatically
search:
  n_grid: 20                         # alpha points in [0,1]
  clip: true
  clip_grid: 20
  max_shrink: 0.5
calib:
  dataset: pileval                   # pileval | c4 | wikitext | /path/to/your.jsonl  ({"text":
...} per line)
  n_samples: 128
  seq_len: 512
eval:
  ppl_datasets: [wikitext2, c4]
  ppl_seq_len: 2048
  lm_eval_tasks: [arc_easy, arc_challenge, hellaswag, piqa, winogrande]
io:
  work_dir: /workspace/runs/qwen2.5-7b-awq
seed: 42
```

10. Step 8: Transfer and run

```
# ── on the GPU server, inside tmux, venv active ──────────────────────────
cd /workspace/WEEK--37 && git pull                                  # or WinSCP the folder
cp configs/examples/llama-3.1-8b.yaml configs/run.yaml               # edit model / work_dir

python scripts/00_env_check.py                                       # gate 0: ENV OK
python scripts/01_inspect_model.py --config configs/run.yaml    # gate 1: COMPATIBLE + fits
python scripts/02_baseline.py      --config configs/run.yaml    # FP16 + naive RTN PPL →
baseline.json
python scripts/03_quantize.py      --config configs/run.yaml    # AWQ block by block (resumable)
python scripts/04_verify.py        --config configs/run.yaml    # gate 4: logits close, real
VRAM, size
python scripts/05_eval.py          --config configs/run.yaml    # PPL + lm-eval + generation +
speed
python scripts/06_tune.py          --config configs/run.yaml    # optional: improve accuracy
python scripts/07_export.py        --config configs/run.yaml --format native,autoawq
python scripts/08_publish.py       --config configs/run.yaml --repo <you>/Llama-3.1-8B-AWQ --
private

# or everything 00→05 in one go, stopping at the first failed gate:
bash scripts/run_all.sh configs/run.yaml 2>&1 | tee run.log
```

Script	Produces (in `work_dir`)	Pass gate
00_env_check	`env.json`	GPU visible, torch.cuda OK, disk ≥ Section-4 estimate, HF login OK
01_inspect_model	`model_report.json`	Compatible = true. Phase-A VRAM fits the GPU you rented.
02_baseline	`baseline.json`	Finite FP16 PPL in the published ballpark. RTN PPL > FP16 PPL.
03_quantize	`quantized/`, `search_log.jsonl`	Every block written. No NaN/Inf. Per-block error logged.
04_verify	`verify.json`	Every packed layer decodes to finite numbers. Next-token agreement with the original on 10 probe prompts ≥ 75%. On-disk size within 15% of the formula for the dtype you ran in (FP32 on CPU, 16-bit on GPU).
05_eval	`results.json`, `generations.md`	Section 11 gates.

RESUME If the pod dies mid-run: start a new pod, attach the **same volume** and run `03_quantize.py --resume`. It continues from the last finished block.

11. Step 9: Evaluation

11.1 What gets measured

Metric	How	Why
Perplexity WikiText-2 (test)	Non-overlapping 2048-token windows	The standard number every paper reports. Lower is better.
Perplexity C4 (validation subset)	256 × 2048 tokens	A second domain, so you don't over-trust one dataset.
Downstream accuracy	`lm-eval`: ARC-e/c, HellaSwag, PIQA, Winogrande (+ MMLU optional). The in-memory model is handed to lm-eval directly, so the native format works too.	Real task quality, not just next-token loss.
Generation diff	20 fixed prompts, greedy, FP16 vs AWQ side by side → `generations.md`	Catches failures that metrics hide (repetition, language switching).
Your task	Optional `eval.custom_jsonl` with prompt/expected	The only metric that matters for your use case.
Size on disk, peak VRAM, tokens/s	Meta-device load, reset peak stats; timed generate	The reason you quantized in the first place.

11.2 Pass / fail gates

Model size	PPL gap vs FP16 (WikiText-2)	Avg downstream accuracy drop	Also required
≤ 3B	≤ 10%	≤ 3 points	AWQ PPL < naive RTN PPL. No empty generations in `generations.md`. (Memory saving is reported, not gated: small models with big vocabularies keep a large 16-bit embedding, so they only shrink ~2×. 7B+ models shrink ~2.8–3.8×.)
7–34B	≤ 5%	≤ 1.5 points	
≥ 70B	≤ 3%	≤ 1 point	

BRUTAL TRUTH If AWQ doesn't beat naive RTN, there's a bug (usually scales on the wrong axis, or `1/s` not applied). Don't tune, debug. If it beats RTN but misses the gate, go to Section 12.

≥400B / 1T There's no FP16 baseline (too expensive). The gate becomes per-block reconstruction error (logged in `search_log.jsonl`) plus lm-eval scores within 1–2 points of the model's **published** scores, with a note on the method in the model card.

12. Step 10: Hyperparameter tuning

There's no training in AWQ. The "hyperparameters" are the quantization and calibration knobs. Tuning shrinks the gap to FP16 and can never beat FP16. Each trial costs one full quantize + short eval, so tune in stages. Don't brute-force everything at once.

Knob	Values tried	Typical effect	Cost
`calib.dataset`	pileval, c4, **your domain data**	The biggest win when your use case is narrow (code, legal, Hindi...). Calibrate on what you'll serve.	none
`quant.group_size`	128 → 64 → 32	Smaller groups are more accurate. 64 usually recovers a good part of the gap.	+3% / +9% size
`quant.zero_point`	true / false	Asymmetric is usually better. Symmetric is simpler.	tiny
`calib.n_samples` × `seq_len`	128×512 → 256×512 → 128×2048	Small gains. Helps long-context models.	more VRAM/time
`search.n_grid`	20 → 40	Marginal.	2× search time
`search.clip`, `max_shrink`	on/off, 0.5 → 0.3	Clip on is almost always better.	+30% time
`quant.skip_modules`	keep the worst k Linears in 16-bit (from `search_log.jsonl`), often first/last block's `down_proj`	Targeted fix for a few sensitive layers.	+size per layer
`search.propagate`	fp / quantized	quantized = later blocks see (and compensate for) earlier rounding errors.	none
`quant.fold_scales`	auto / never	never = also scale groups that cannot fold (e.g. GQA o_proj), so a bit more accurate. But native format only: no vLLM export.	tiny runtime cost

12.1 Protocol (strict)

1. Tuning metric = PPL on a **held-out** slice (C4 validation windows not used anywhere else). **Never** WikiText-2 test or lm-eval.
2. Stage 1: calibration dataset → Stage 2: group size → Stage 3: samples/length → Stage 4: skip-layers. Carry the winner forward each time.
3. Stop when the gate in 11.2 is met, or after 3 trials in a row that improve less than 0.5%.
4. Run the final eval (11.1) **once** on the winner.

REAL RUN Section 19.7–19.8 shows this protocol on Qwen2.5-0.5B: group 64 was accepted (validation PPL −4.7%), `fold_scales=never` (−0.1%) and quantized propagation were rejected, and the final test-set gap went from +16.0% to +10.7%.

```
python scripts/06_tune.py --config configs/run.yaml --space configs/tune_space.yaml --budget-hours 3
# writes tuning/leaderboard.csv  and  configs/run.best.yaml
```

13. Step 11: Export, give back, shut down

13.1 Formats

Format	Loads in	When
native: our `AwqLinear` safetensors + `quant_config.json`	This repo (`awq.load_quantized()`), any architecture	Always produced. Works for every model the pipeline can quantize.
autoawq (standard AWQ "GEMM": int32 `qweight` / `qzeros`, fp16 `scales`, `quantization_config` in config.json)	vLLM, SGLang (fast AWQ-Marlin kernels). The layout is verified bit-exact against a reference decoder; actually loading it in vLLM is the first thing to do on your GPU box (not done here: no GPU).	Families where `1/s` can be folded (Llama, Mistral, Mixtral, Qwen2/2.5/3 + MoE, Gemma, Phi-3). The script refuses with a clear message otherwise.

```
python scripts/07_export.py --config configs/run.yaml --format native,autoawq
# check the standard export with a real serving engine (separate venv):
python -m venv /workspace/vllm-env && source /workspace/vllm-env/bin/activate && pip install
vllm
vllm serve /workspace/runs/llama-3.1-8b-awq/export/autoawq --quantization awq_marlin --max-
model-len 8192
curl localhost:8000/v1/completions -H "Content-Type: application/json" \
     -d '{"model":"/workspace/runs/llama-3.1-8b-awq/export/autoawq","prompt":"The capital of
France is","max_tokens":16}'
```

13.2 Give it back

```
hf upload <you>/Llama-3.1-8B-AWQ /workspace/runs/llama-3.1-8b-awq/export/autoawq . --private
# 08_publish.py does the same + writes a model card with ONLY measured numbers from results.json
```

Or pull it to your laptop: in WinSCP, right pane `/workspace/runs/…/export` → drag it to the left pane. Also copy `results.json`, `generations.md`, `search_log.jsonl` and `run.log` into the repo's `proof/` folder.

LICENSES A quantized model is a derivative. Llama keeps the Llama license (keep "Built with Llama" and the license file), Gemma keeps Gemma terms, Apache-2.0 stays Apache-2.0. `08_publish.py` copies the base model's license into the repo.

13.3 Shut down (do this every time)

1. Confirm the artifacts are on HF (open the repo page and check the files) or on your laptop (WinSCP).
2. **Stop** the pod. Compute billing stops, but the volume still bills (RunPod idle volume costs $0.20/GB/month).
3. **Terminate/Delete** the pod and the volume when you're done. Check the billing page shows $0/hr.
4. Revoke the HF write token if it was only for this job.

14. Proof of work: what lands in GitHub

Evidence	Where	Honest scope
Full source with plain-English docstrings	`awq/`, `scripts/`	Written by me and reviewed against the AWQ paper.
30 tests (pack/unpack, exact scaling identity and folding, AWQ beats RTN, end-to-end Llama/Qwen2/Mixtral/plugin, resume, export layout, sizing)	`tests/`, output in `proof/pytest.txt`	Run on CPU. Real output committed.
End-to-end real run on a small real model on CPU: FP32 vs RTN vs AWQ perplexity, size, generation diff	`proof/cpu-qwen2.5-0.5b/` (logs, `results.json`, `generations.md`, `env.json`, git commit hash)	Real numbers from this sandbox. Small model, CPU, so it proves correctness and the pipeline, not speed.
Hardware report for reference models (8B, 70B, Mixtral, DeepSeek-V3, Kimi-K2 class)	`proof/hardware_reports/`	Generated by `01_inspect_model.py` from real config/headers. Formula output, labelled as estimates.
GPU run (8B → 70B ...)	`proof/gpu-<model>/`	**Your run.** I provide `proof/GPU_RUN_TEMPLATE.md`. You paste `run.log` + `results.json`. I review the numbers when you send them.
This playbook v1.0	`docs/Universal_AWQ_Playbook.pdf`, built by `docs/playbook/build.py`	Sections 4.4, 19, 20 and the Appendix are generated from the repo files, so they can't drift from the code.

BRUTAL TRUTH "Proof" means anyone can clone the repo, run `pytest` and `bash scripts/run_all.sh configs/examples/qwen2.5-0.5b-cpu.yaml` on a laptop and get the same numbers within noise. Screenshots aren't proof. Reproducible commands are.

15. Troubleshooting

Symptom	Cause	Fix
`torch.cuda.is_available() == False`	CPU wheel installed, or the wheel's CUDA is newer than the driver	Reinstall with `--index-url …/cu1xx` ≤ the "CUDA Version" in `nvidia-smi`.
`no kernel image is available for execution`	Old torch on a Blackwell GPU	torch ≥ 2.7 with `cu128`.
CUDA OOM in 03_quantize	Block + activations don't fit	Lower `calib.n_samples` to 64, or rent the next GPU up (Section 4). The run resumes with `--resume`.
CUDA OOM in 02_baseline	FP16 model too big for the GPU(s)	The script uses `device_map=auto` across all GPUs. Rent more, or use `--skip-fp16` and compare against published numbers.
Killed (no message)	Out of CPU RAM	Rent a pod with more RAM (≥ 3 × block), or make sure streaming load is on (it's the default).
PPL = nan/inf	fp16 overflow (common on Gemma), or the wrong tokenizer	`dtype: bfloat16`. Load the tokenizer from the same path as the model.
AWQ PPL ≈ RTN PPL or worse	Scale on the wrong axis, `1/s` missing, or calibration text empty	`pytest tests/`. Check that the `search_log.jsonl` alpha histogram isn't all zeros.
Gated 401	License not accepted / wrong account token	Section 5.1.
WinSCP "Connection refused" / "Server unexpectedly closed"	Proxied SSH (RunPod), wrong port/user, firewall	Use "SSH over exposed TCP". Check the port. User `ubuntu` on VMs. Open port 22 (Hyperstack firewall).
`bad interpreter: ^M`	Windows line endings	`sed -i 's/\r$//' scripts/*.sh`
vLLM refuses the export	Architecture not in the fold list, or a group size the kernel doesn't support	Use the native format, or group_size 128 / 64.
Pod disappeared	Spot/community reclaim	Same volume → new pod → `--resume`.

16. Command master sheet

```
# ── LAPTOP (once) ──────────────────────────────────────────────────────────
ssh-keygen -t ed25519 -C "gpu"                      # paste .pub into provider
git clone https://github.com/kashyapphassan2000-netizen/WEEK--37.git && cd WEEK--37
python -m venv .venv && .venv\Scripts\activate       # mac/linux: source .venv/bin/activate
pip install -e . && python scripts/01_inspect_model.py --model <MODEL>   # sizing, no GPU

# ── RENT (Section 6) → CONNECT ─────────────────────────────────────────────
ssh -i ~/.ssh/id_ed25519 root@<IP> -p <PORT>
tmux new -s awq

# ── SERVER SETUP ───────────────────────────────────────────────────────────
nvidia-smi
apt-get update && apt-get install -y git tmux htop nvtop python3-venv
cd /workspace && git clone https://github.com/kashyapphassan2000-netizen/WEEK--37.git && cd
WEEK--37
python3 -m venv .venv && source .venv/bin/activate && pip install -U pip wheel
pip install torch --index-url https://download.pytorch.org/whl/cu128
pip install -e ".[eval]"
export HF_HOME=/workspace/hf_cache && hf auth login
python scripts/00_env_check.py

# ── RUN ────────────────────────────────────────────────────────────────────
cp configs/examples/<closest>.yaml configs/run.yaml && nano configs/run.yaml
bash scripts/run_all.sh configs/run.yaml 2>&1 | tee run.log
python scripts/06_tune.py   --config configs/run.yaml --space configs/tune_space.yaml   #
optional
python scripts/07_export.py --config configs/run.yaml --format native,autoawq
python scripts/08_publish.py --config configs/run.yaml --repo <you>/<name>-AWQ --private

# ── GIVE BACK + SHUT DOWN ──────────────────────────────────────────────────
# WinSCP: drag /workspace/runs/<run>/{export,results.json,generations.md,run.log} → laptop
# provider dashboard: Stop → Terminate pod → Delete volume → billing shows $0/hr
```

17. Status: what is done, what is not (honest)

Item	Status
Library (`awq/`), scripts 00–08, `run_all.sh` , configs + examples	**Done.** Lint clean (ruff).
Unit and integration tests	**30 pass** on CPU (`proof/pytest.txt`): dense Llama, Qwen2 GQA + tied embeddings, Mixtral fused-MoE, custom plugin model, export, resume, sizing.
Real-model end-to-end run	**Done on CPU** with Qwen2.5-0.5B, all gates and tuning. Numbers in Section 19, including the gates it FAILED.
GPU run at 7B / 70B	**Not done**: no GPU on the build machine. It's your first rented run (≈ $1–3 for 7–8B). Template: `proof/GPU_RUN_TEMPLATE.md` .
vLLM loading of the exported GEMM format	**Not verified here** (needs a GPU). Tensor layout verified bit-exact against an independent decoder.
MoE on real weights (Mixtral, Qwen3-MoE, DeepSeek)	Code path tested on a tiny random Mixtral only. Fused experts stay in the native format (no vLLM export).
Kimi-K2 / other `trust_remote_code` models	Kimi-K2's own modeling code does not import under transformers 5.x, so its sizing used the formula fallback (Section 20). Quantizing it needs either its code fixed for transformers 5 or a transformers-4.x environment. Be honest with yourself: this is a multi-hundred-dollar job.
Fast inference kernels	**Out of scope.** The reference runtime dequantizes in Python and is slower than FP16. For speed, export and serve with vLLM/SGLang.

18. Code walkthrough: every file in plain words

This section explains what each file does, what goes in, what comes out, and why it's written that way. The complete source of every file is printed in the Appendix, and each source file also carries plain-English comments at every step. Read this section first, then the code.

18.1 How the pieces call each other

```
scripts/03_quantize.py
  └─ awq.config.load_config()          read configs/run.yaml (+ --set overrides), validate
  └─ awq.pipeline.quantize_model()
       ├─ awq.model_io.resolve_model_path()   download configs + *.safetensors (or use local
folder)
       ├─ awq.model_io.ModelSource            empty skeleton on "meta" + lazy WeightStore
       │     └─ awq.blocks.find_blocks()      which nn.ModuleList holds the N decoder blocks
       ├─ awq.calib.get_calib_tokens()        (n_samples, seq_len) token windows, no padding
       ├─ capture_block0_inputs()             embeddings only, stop at block 0 (Catcher)
       └─ for each block i:
            ModelSource.load_block(i)         only THIS block's tensors are read from disk
            quantize_block()
              ├─ awq.blocks.quantizable_linears / fused_experts
              ├─ collect()                    hooks record every Linear's input, group shared
inputs
              ├─ awq.search.search_scale()    AWQ alpha grid search per group
              ├─ awq.blocks.fold_target() + _fold()   put 1/s into the previous norm/linear
              ├─ awq.search.search_clip()     clip outlier weights per group
              └─ awq.awq_linear.AwqLinear.from_float()   pack to 4-bit
            save block-xxxxx.safetensors + progress.json   (this is the --resume checkpoint)
       └─ save non_block.safetensors, config, tokenizer, quant_config.json
scripts/04_verify.py, 05_eval.py → awq.model_io.load_quantized() → awq.evaluate.*
scripts/07_export.py → awq.export.export_autoawq()      scripts/01_inspect_model.py →
awq.hardware.*
```

18.2 awq/config.py

Job: turn `configs/run.yaml` into Python objects and refuse bad values before you pay for a GPU.

- `QuantConfig`, `SearchConfig`, `CalibConfig`, `EvalConfig`, `IOConfig`, `RunConfig`: one dataclass per YAML section. Every field has a default and a comment saying what it does.
- `apply_overrides()`: makes `--set quant.group_size=64` work. The value is parsed with YAML rules, so `64` becomes an int and `false` becomes a bool.
- `validate()`: fails fast with the fix in the message. Examples: 4-bit only, group size must be one of 32/64/128/256/-1, float32 on CPU, n_samples divisible by batch_size.
- An unknown key (a typo like `groupsize`) is an error, not silently ignored.

18.3 awq/quantizer.py

Job: the arithmetic of 4-bit numbers. It doesn't know about AWQ.

- `compute_qparams(w)`: for every group of 128 weights in a row, compute `scale` and `zero`. Asymmetric uses min/max; symmetric uses max|w| with zero fixed at 8.
- `quantize()`: `q = round(w/scale) + zero`, clamped to 0..15. `dequantize()`: `(q − zero)·scale`.
- `fake_quant()`: quantize, then immediately dequantize. The result is still float but carries the rounding error. The search measures that error.

- `pack_int4()` / `unpack_int4()`: two 4-bit codes per byte (even column in the low nibble, odd column in the high nibble). This is where the 4× storage saving comes from.

18.4 awq/awq_linear.py

- `AwqLinear`: stands in for `nn.Linear`. Buffers: `qweight` (packed codes), `scales`, `qzeros`, optional `inv_s`, optional bias. `forward(x)`: multiply by `inv_s` if present → dequantize → ordinary matmul. `from_float()` builds it from a Linear whose weight is already scaled and clipped.
- `AwqExpertsWeight`: stands in for a fused MoE parameter of shape (experts, out, in). The model's own expert loop asks for `W[e]`; `__getitem__` returns expert e's dequantized matrix with `1/s` already applied to its columns. The original model code runs unchanged.

18.5 awq/blocks.py

- `find_blocks(model)`: the decoder blocks are the biggest `nn.ModuleList` whose children are all the same class. No model names are hard-coded, which is why it works on Llama, Qwen, Mixtral, DeepSeek or your own model.
- `quantizable_linears(block)`: every `nn.Linear` in the block, minus routers, tiny layers, layers whose width isn't divisible by the group size, and your `skip_modules`.
- `fused_experts(block)`: finds 3-D expert weights named `gate_up_proj` / `down_proj` (the transformers ≥ 5 layout).
- `fold_target(block, group)`: answers "which earlier op can absorb 1/s?" using the table in docs/derivation.md §5. It returns None for unknown families or GQA `o_proj`; then the scale is kept inside the layer (always correct) or skipped (if you want the vLLM export).
- `allow_clip()`: no clipping for q/k projections, same rule as the official code.

18.6 awq/model_io.py

- `resolve_model_path()`: an HF id becomes a local folder, downloading only what's needed (configs, tokenizer, weights; `*.py` only if `trust_remote_code`).
- `build_skeleton()`: builds the model with `init_empty_weights()`, so every parameter is a zero-byte meta tensor. A 1T-parameter skeleton takes a few seconds and no memory.
- `WeightStore`: reads one tensor at a time from safetensors shards. It handles: FP8 + `weight_scale_inv` (DeepSeek block format) → float; renamed keys (`block_sparse_moe` ↔ `mlp`); per-expert checkpoint keys stacked into the fused 3-D layout.
- `ModelSource`: `load_block(i)` fills block i from disk onto the GPU and `unload_block(i)` turns it back into meta. If a checkpoint can't be mapped key by key, it falls back to a normal full CPU load and logs that loudly (then you need RAM ≥ model size).
- `load_quantized(dir)`: skeleton → swap in empty `AwqLinear`s → load only packed tensors. Peak memory equals the real quantized footprint, which is why the VRAM numbers in verify/eval are honest.

18.7 awq/calib.py

- `get_calib_tokens()`: shuffle documents (seeded), tokenize, join with end-of-text tokens, cut into exactly `n_samples × seq_len`. No padding: that bug in the original guide is gone.
- `get_eval_tokens()`: non-overlapping 2048-token windows of WikiText-2 *test*, WikiText-2 *validation* (tuning only) or C4 validation.

18.8 awq/search.py: the AWQ algorithm

- `search_scale(weights, x)`: candidate 0 is s = 1 (plain RTN). Then for α = 0, 0.05, ..., 0.95: `s = x_mean^α / w_mean^(1−α)`, normalised. Error = `‖x·(W − Q(W·s)/s)ᵀ‖²`, one matmul per candidate, chunked so memory stays bounded. It returns the best s, its α, and the RTN loss for comparison (both are logged per group in `search_log.jsonl`).
- `search_clip(w, x)`: for every (output row, group), try shrinking the max to 100%...55% and keep the clip with the lowest partial-output error. Written as a batched einsum, which made it 30× faster than the element-wise version on CPU.
- `apply_clip()`: clamps the weights to the chosen limits.

18.9 awq/pipeline.py: the main loop

- `_Catcher`: temporarily replaces block 0, stores its inputs and keyword arguments (attention mask, rotary position embeddings), then raises to stop the forward pass. Any architecture gives us exactly what its blocks expect.
- `collect()`: runs the FP block once. Forward hooks save each Linear's input. Linears whose input is the *same tensor in memory* form a group (q/k/v, gate/up). `_ExpertRecorder` temporarily wraps `F.linear` to catch which tokens each expert actually received.
- `quantize_block()`: (1) search scales for all groups, (2) apply them and fold 1/s, (3) clip, (4) pack into `AwqLinear`, (5) experts per expert, (6) run the quantized block and log its relative error vs FP. It returns the next block's input (FP outputs by default, like the official code).
- `quantize_model()`: the whole run, writing per-block checkpoints and `progress.json`; `resume=True` continues from the last finished block. `tests/test_pipeline.py` proves a crash plus resume gives byte-identical output.

18.10 awq/evaluate.py, awq/export.py, awq/hardware.py, awq/plugin.py

- **evaluate**: `perplexity()` (sum of token NLL over windows), `apply_rtn_()` (the naive baseline, applied to exactly the weights AWQ touches), `generations()`, `speed()`, `model_bytes()`, `run_lm_eval()` (passes the in-memory model to lm-evaluation-harness).
- **export**: `to_gemm()` repacks codes into int32 words along output channels in AWQ's order [0,2,4,6,1,3,5,7]; `gemm_dequant()` is an independent reference decoder used by the tests and by `07_export.py` to prove both formats decode to the same weights.
- **hardware**: `count_params()` counts exact parameters on the meta skeleton (tied embeddings once, experts included); `estimate()` applies the Section 4 formulas; `_pick()` chooses the cheapest GPU × count per phase (quantize = 1 GPU; serve = 1/2/4/8 for vLLM).
- **plugin**: lets a non-transformers model go through the same pipeline via three functions (`load`, `blocks`, `calib_batches`). `tests/tiny_plugin.py` is a working template.

18.11 scripts/

Each numbered script is thin: parse `--config` / `--set`, call the library, write one JSON file, print PASS/FAIL, and exit non-zero on failure so `run_all.sh` stops at the first broken gate. Section 10 lists what each produces.

18.12 tests/

File	What it proves
test_quantizer.py	pack/unpack is lossless; error ≤ half a quantization step; AwqLinear equals fake-quant math; 0.5 byte/weight storage
test_scaling.py	the AWQ identity is exact; every fold (norm, Gemma (1+w) norm, up→down, v→o) leaves the block output unchanged to 1e-9
test_search.py	AWQ beats RTN when salient channels exist (also on fresh data); never worse than RTN; clipping never increases error
test_pipeline.py	end-to-end Llama (all folded), AWQ < RTN error end-to-end, never-fold mode, Qwen2 GQA + tied embeddings, crash + resume = identical bytes
test_moe.py	Mixtral fused experts are quantized per expert, reload works, output close to FP
test_export.py	GEMM pack roundtrip, exact AWQ nibble order, GEMM decode = native decode, export refuses unfolded models
test_plugin.py / test_hardware.py / test_config.py	custom model path; Llama-3.1-8B sizing equals the hand calculation in Section 4.3; config typing and validation

19. Proof: measured results

Everything on this page was produced by `bash scripts/run_all.sh configs/examples/qwen2.5-0.5b-cpu.yaml` on the build machine: 4 CPU cores, 16.9 GB RAM, no GPU, torch 2.14.1+cpu, transformers 5.20.1. The raw files are in `proof/cpu-qwen2.5-0.5b/`. On CPU the "original" model runs in **FP32** (CPU half-precision is slow/unsupported), so "original" below means FP32.

SETUP Model `Qwen/Qwen2.5-0.5B` · AWQ 4-bit, group 128, zero-point=True, fold_scales=auto · calibration pileval 64×512 tokens · eval: first 20 windows of 2048 tokens per dataset (CPU budget) · 168 Linear layers quantized, 0 with runtime 1/s.

19.1 Perplexity (lower is better)

Dataset	Original (FP32)	Naive RTN 4-bit	**AWQ 4-bit**	RTN gap	AWQ gap	Gap closed by AWQ
wikitext2	12.701	15.081	**14.733**	+18.7%	+16.0%	15%
c4	19.156	22.987	**21.999**	+20.0%	+14.8%	26%

19.2 Downstream accuracy (lm-eval, first 300 examples per task)

Task	Metric	Original	RTN	**AWQ**
arc_easy	acc	0.597	0.590	**0.573**
arc_easy	acc_norm	0.570	0.593	**0.563**
piqa	acc	0.710	0.663	**0.680**
piqa	acc_norm	0.717	0.697	**0.687**

300 examples per task means a ±2.8-point standard error. Differences under ~3 points are noise at this sample size.

19.3 Memory, agreement, speed

Measure	Value
Weights in memory, original (FP32)	1.976 GB
Weights in memory, AWQ	0.738 GB (2.679× smaller vs FP32)
AWQ on disk / formula for this dtype	0.7378 GB / 0.7377 GB (ratio 1.0)
AWQ size if saved in 16-bit (formula)	0.46 GB vs 0.99 GB FP16 checkpoint
Next-token agreement with original (10 probe prompts, all positions)	0.786
Mean relative logit error	0.279
Speed original (CPU, greedy)	9.23 tok/s
Speed AWQ reference kernel (CPU)	1.64 tok/s

19.4 Gates

Gate	Result
verify: all_layers_finite	PASS
verify: size_matches_formula	PASS
verify: top1_agreement>=0.75	PASS
wikitext2: AWQ ppl within 10% of original	FAIL
wikitext2: AWQ ppl < RTN ppl	PASS
c4: AWQ ppl within 10% of original	FAIL
c4: AWQ ppl < RTN ppl	PASS
lm-eval avg drop <= 3.0 points	PASS
no empty generations	PASS

19.5 What the search chose, block by block

Block	qkv α	gate/up α	down α	AWQ/RTN loss (qkv, gate/up, down)	block rel. MSE	s
0	0.20	0.20	0.10	0.82, 0.81, 0.91	1.22e-02	35.47
1	0.25	0.15	0.20	0.79, 0.86, 0.97	6.59e-03	31.83
2	0.30	0.20	0.70	0.84, 0.86, 0.03	2.93e-03	35.88
3	RTN	0.20	0.45	1.00, 0.78, 0.09	1.87e-04	35.18
4	0.25	0.20	0.40	0.63, 0.85, 0.99	2.10e-04	33.45
5	0.25	0.20	0.35	0.73, 0.77, 0.89	1.88e-04	30.61
6	0.25	0.25	0.35	0.67, 0.82, 0.96	1.86e-04	30.84
7	0.30	0.25	0.30	0.76, 0.81, 0.97	2.21e-04	32.58
8	0.40	0.25	0.15	0.81, 0.81, 0.93	2.41e-04	35.28
9	0.30	0.25	0.10	0.81, 0.82, 0.93	2.22e-04	30.63
10	0.30	0.20	0.20	0.63, 0.81, 0.93	1.84e-04	33.36
11	RTN	0.25	0.10	1.00, 0.84, 0.93	2.69e-04	34.91
12	0.25	0.25	0.20	0.71, 0.78, 0.90	1.82e-04	29.06
13	0.30	0.30	0.20	0.87, 0.77, 0.90	2.21e-04	31.2
14	0.20	0.30	0.10	0.69, 0.75, 0.89	3.57e-04	30.62
15	0.25	0.25	0.25	0.70, 0.74, 0.89	3.45e-04	29.51
16	0.30	0.30	RTN	0.68, 0.73, 1.00	1.03e-03	29.41
17	0.25	0.25	0.25	0.72, 0.69, 0.90	6.12e-04	29.27
18	0.30	0.25	0.30	0.80, 0.75, 0.91	5.70e-04	29.27
19	0.30	0.30	0.15	0.58, 0.69, 0.87	9.01e-04	31.24
20	0.35	0.30	0.20	0.81, 0.73, 0.80	1.88e-03	29.44
21	0.25	0.25	0.35	0.85, 0.81, 0.25	4.98e-03	30.41
22	0.25	0.25	0.20	0.73, 0.81, 0.59	2.76e-03	32.29
23	0.35	0.25	0.25	0.63, 0.74, 0.24	4.74e-03	31.06

α = 0 trusts weights only, α → 1 trusts activations only. Loss ratio < 1 means AWQ's scale beat plain rounding on that group's calibration output. The first and last blocks are the hardest (largest error), the classic outlier pattern.

19.6 Generations (first 3 of 10 prompts, greedy; all 10 in generations.md)

````
## Prompt
```
The capital of France is
```
**Original (float32):**
```
Paris. It is the largest city in Europe and the second largest in the world. It is also the capital of France,
the second largest country in Europe, and the third largest country in the world. It is the seat of the French
```
**AWQ:**
```
Paris. It is the seat of the government of France. It is also the capital of the French Republic. The city is
located in the North of the country. It is the largest city in France by population. It is the second largest
```

## Prompt
```
Explain in one sentence why the sky is blue:
```
**Original (float32):**
```
The sky is blue because it reflects sunlight.
```
**AWQ:**
```
The sky is blue because it is made up of sunlight reflected off the water surface.
```

## Prompt
```
def fibonacci(n):
    """Return the n-th Fibonacci number."""

```
**Original (float32):**
```
if n == 0:
        return 0
    elif n == 1:
        return 1
    else:
        return fibonacci(n-1) + fibonacci(n-2)
````

19.7 Tuning (06_tune.py, metric = WikiText-2 VALIDATION ppl, never the test set)

trial	minutes	quant.fold_scales	quant.group_size	search.propagate	tune_ppl
2	20.36		64		13.19804
3	20.12		64	quantized	13.21944
1	18.98	never			13.82911
0	18.37				13.84401

```
[  2253.3s] [tune] stage kept quant.fold_scales=auto
[  3475.6s] [tune] stage winner: quant.group_size=64  ppl 13.84401 -> 13.19804
[  4683.5s] [tune] stage kept search.propagate=fp
[  4683.5s] [tune] best overrides {'quant.group_size': 64} -> configs/examples/qwen2.5-0.5b-
cpu.best.yaml  (tune ppl 13.19804)
```
```

19.8 Final run with the tuned config (group 64), evaluated ONCE on the test windows

Dataset	Original	RTN g128	AWQ g128	RTN g64	**AWQ g64 (tuned)**	AWQ g64 gap
wikitext2	12.701	15.081	14.733	14.481	**14.057**	+10.7%
c4	19.156	22.987	21.999	21.894	**21.149**	+10.4%

Tuned model: next-token agreement 0.830 (was 0.786), weights 0.7518 GB, lm-eval arc_easy acc 0.570, piqa acc 0.683. Gates: wikitext2: AWQ ppl within 10% of original: FAIL, wikitext2: AWQ ppl < RTN ppl: PASS, c4: AWQ ppl within 10% of original: FAIL, c4: AWQ ppl < RTN ppl: PASS, lm-eval avg drop <= 3.0 points: PASS, no empty generations: PASS.

BRUTAL TRUTH Most of the tuning gain came from the smaller group size, which helps plain RTN too (RTN g64 is already 14.48). At equal group size, AWQ removes a further ~24–27% of the remaining gap. The 0.5B model still misses the strict ≤10% gate by 0.4–0.7 percentage points. The vLLM-format export of this model passed its layout check (worst rel. diff 3.3e-4, fp16 scale rounding) and the model card was generated with `08_publish.py --dry-run`.

20. Hardware reports for reference models

Produced by `python scripts/01_inspect_model.py --model <id>` (only `config.json` is downloaded). Full reports: `proof/hardware_reports/`. Llama models are read from the ungated `unsloth/` mirrors (same configs). Kimi-K2 used the formula fallback because its remote code does not build under transformers 5.x.

Model	Type	Params	FP16	AWQ	A. Quantize on (VRAM)	B. FP16 eval on	C. Serve AWQ on (8k ctx)	A hours*	A cost*
Qwen/Qwen2.5-0.5B	dense	0.5 B	1 GB	0.5 GB	1× RTX 4090 (5.9 GB)	1× RTX 4090 (1.2 GB)	1× RTX 4090 (0.6 GB)	0.54	$0.23
Qwen/Qwen2.5-7B-Instruct	dense	7.6 B	15 GB	5.6 GB	1× RTX 4090 (17.8 GB)	1× RTX 4090 (18.3 GB)	1× RTX 4090 (7.0 GB)	1.09	$0.46
unsloth/Meta-Llama-3.1-8B-Instruct	dense	8.0 B	16 GB	5.7 GB	1× RTX 4090 (14.1 GB)	1× RTX 4090 (19.3 GB)	1× RTX 4090 (7.8 GB)	1.12	$0.48
Qwen/Qwen3-30B-A3B	MoE 128e top-8	30.5 B	61 GB	16.8 GB	1× RTX 4090 (9.3 GB)	4× RTX 4090 (73.3 GB)	1× RTX 4090 (20.3 GB)	2.86	$1.22
Qwen/Qwen2.5-32B-Instruct	dense	32.8 B	66 GB	19.3 GB	1× L40S (25.7 GB)	4× RTX 4090 (78.6 GB)	2× RTX 4090 (24.7 GB)	2.64	$2.61
unsloth/Llama-3.3-70B-Instruct	dense	70.6 B	141 GB	39.8 GB	1× L40S (28.0 GB)	8× RTX 4090 (169.3 GB)	1× A100 80GB (48.9 GB)	5.11	$5.04
mistralai/Mixtral-8×22B-v0.1	MoE 8e top-2	140.6 B	281 GB	73.7 GB	1× L40S (24.9 GB)	5× A100 80GB (337.5 GB)	2× A100 80GB (86.9 GB)	9.68	$9.56
deepseek-ai/DeepSeek-V3	MoE 256e top-8, FP8 ckpt	671.0 B	1342 GB	351.8 GB	1× A100 80GB (62.5 GB)	multi-node	8× A100 80GB (405.3 GB)	35.01	$52.07
moonshotai/Kimi-K2-Instruct	MoE 384e top-8, FP8 ckpt	1046.5 B	2093 GB	547.6 GB	1× H200 (85.0 GB)	multi-node	4× B200 (630.5 GB)	31.0	$139.11

Appendix: full source code

Every file in the repository, exactly as committed. The comments in each file explain each step in plain words.

pyproject.toml (43 lines)

```
[project]
name = "awq-from-scratch"
version = "1.0.0"
description = "Universal, block-streaming AWQ (Activation-aware Weight Quantization) for any causal LM — from
scratch in pure PyTorch."
readme = "README.md"
license = { text = "Apache-2.0" }
requires-python = ">=3.10"
dependencies = [
    "torch>=2.4",
    "transformers>=4.51",
    "accelerate>=0.30",
    "datasets>=2.18",
    "safetensors>=0.4.2",
    "huggingface_hub>=0.34",
    "zstandard>=0.22",
    "pyyaml>=6.0",
    "tqdm>=4.66",
    "numpy>=1.26",
]

[project.optional-dependencies]
eval = ["lm-eval>=0.4.4"]
dev = ["pytest>=8.0", "ruff>=0.4"]

[build-system]
requires = ["setuptools>=68", "wheel"]
build-backend = "setuptools.build_meta"

[tool.setuptools.packages.find]
include = ["awq*"]

[tool.ruff]
line-length = 120
target-version = "py310"

[tool.pytest.ini_options]
testpaths = ["tests"]

[tool.ruff.lint]
select = ["E", "F", "W"]

[tool.ruff.lint.per-file-ignores]
"docs/playbook/build.py" = ["E501"]   # long HTML strings
```

awq/__init__.py (7 lines)

```
"""awq-from-scratch: universal, block-streaming AWQ 4-bit quantization in pure PyTorch."""
from .awq_linear import AwqExpertsWeight, AwqLinear
from .config import RunConfig, load_config
from .model_io import load_quantized

__all__ = ["AwqLinear", "AwqExpertsWeight", "RunConfig", "load_config", "load_quantized"]
__version__ = "1.0.0"
```

awq/awq_linear.py (149 lines)

```
"""
awq_linear.py — the modules that hold 4-bit weights inside the model.

AwqLinear
    Drop-in replacement for nn.Linear. Stores packed 4-bit codes + per-group scales/zeros.
    forward(x): (optionally x * inv_s) -> dequantize weight -> ordinary matmul.
    `inv_s` (= 1/s) is only present when the AWQ scale could NOT be folded into the
```

```
    previous layer; then the layer divides its own input by s at runtime.

AwqExpertsWeight
    Replacement for a fused 3-D Mixture-of-Experts weight (num_experts, out, in), the way
    transformers >= 5 stores experts. The experts' forward code does `W[e]` to get expert e's
    matrix; this module answers `W[e]` with the dequantized matrix, so the original
    (eager) expert code runs unchanged.

Honest note: dequantizing in Python on every forward is the slow-but-correct reference.
Fast inference uses fused int4 kernels (vLLM / SGLang) via scripts/07_export.py.
"""
from __future__ import annotations

import torch
import torch.nn as nn
import torch.nn.functional as F

from .quantizer import compute_qparams, dequantize, pack_int4, quantize, unpack_int4


class AwqLinear(nn.Module):
    def __init__(self, in_features: int, out_features: int, bias: bool, w_bit: int = 4,
                 group_size: int = 128, has_inv_s: bool = False, dtype=torch.float16, device=None):
        super().__init__()
        self.in_features = in_features
        self.out_features = out_features
        self.w_bit = w_bit
        self.group_size = in_features if group_size <= 0 else group_size
        n_groups = in_features // self.group_size
        f = dict(device=device)
        self.register_buffer("qweight", torch.zeros(out_features, in_features // 2, dtype=torch.uint8, **f))
        self.register_buffer("qzeros", torch.zeros(out_features, n_groups, dtype=torch.uint8, **f))
        self.register_buffer("scales", torch.zeros(out_features, n_groups, dtype=dtype, **f))
        if has_inv_s:
            self.register_buffer("inv_s", torch.ones(in_features, dtype=dtype, **f))
        else:
            self.inv_s = None
        if bias:
            self.bias = nn.Parameter(torch.zeros(out_features, dtype=dtype, **f), requires_grad=False)
        else:
            self.register_parameter("bias", None)

    @classmethod
    @torch.no_grad()
    def from_float(cls, linear: nn.Linear, w_bit: int, group_size: int, zero_point: bool,
                   inv_s: torch.Tensor | None = None) -> "AwqLinear":
        """`linear.weight` must ALREADY be W * diag(s) (scaled) and clipped."""
        w = linear.weight.data
        m = cls(linear.in_features, linear.out_features, linear.bias is not None, w_bit, group_size,
                has_inv_s=inv_s is not None, dtype=w.dtype, device=w.device)
        scales, zeros = compute_qparams(w, w_bit, m.group_size, zero_point)
        q = quantize(w, scales, zeros, w_bit, m.group_size)
        m.qweight.copy_(pack_int4(q))
        m.qzeros.copy_(zeros.to(torch.uint8))
        m.scales.copy_(scales.to(w.dtype))
        if inv_s is not None:
            m.inv_s.copy_(inv_s.to(w.dtype))
        if linear.bias is not None:
            m.bias.data.copy_(linear.bias.data)
        return m

    def dequant_weight(self, dtype=None) -> torch.Tensor:
        q = unpack_int4(self.qweight)
        return dequantize(q, self.scales.float(), self.qzeros, self.group_size,
                          dtype=dtype or self.scales.dtype)

    @property
    def weight(self) -> torch.Tensor:
        """Some model code reads `.weight` (e.g. for its dtype). Return the dequantized matrix."""
        return self.dequant_weight()

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        if self.inv_s is not None:
            x = x * self.inv_s.to(x.dtype)            # the "x / s" half of the AWQ identity
        w = self.dequant_weight(dtype=x.dtype)
        return F.linear(x, w, None if self.bias is None else self.bias.to(x.dtype))

    def extra_repr(self) -> str:
        return (f"in={self.in_features}, out={self.out_features}, bits={self.w_bit}, "
```

```
                f"group={self.group_size}, inv_s={self.inv_s is not None}, bias={self.bias is not None}")


class AwqExpertsWeight(nn.Module):
    """Quantized stand-in for a fused expert parameter of shape (E, out, in)."""

    def __init__(self, num_experts: int, out_features: int, in_features: int, w_bit: int = 4,
                 group_size: int = 128, dtype=torch.float16, device=None):
        super().__init__()
        self.num_experts, self.out_features, self.in_features = num_experts, out_features, in_features
        self.w_bit = w_bit
        self.group_size = in_features if group_size <= 0 else group_size
        ng = in_features // self.group_size
        f = dict(device=device)
        self.register_buffer("qweight", torch.zeros(num_experts, out_features, in_features // 2,
                                                    dtype=torch.uint8, **f))
        self.register_buffer("qzeros", torch.zeros(num_experts, out_features, ng, dtype=torch.uint8, **f))
        self.register_buffer("scales", torch.zeros(num_experts, out_features, ng, dtype=dtype, **f))
        self.register_buffer("inv_s", torch.ones(num_experts, in_features, dtype=dtype, **f))

    @torch.no_grad()
    def set_expert(self, e: int, w_scaled: torch.Tensor, inv_s: torch.Tensor | None, zero_point: bool):
        scales, zeros = compute_qparams(w_scaled, self.w_bit, self.group_size, zero_point)
        q = quantize(w_scaled, scales, zeros, self.w_bit, self.group_size)
        self.qweight[e].copy_(pack_int4(q))
        self.qzeros[e].copy_(zeros.to(torch.uint8))
        self.scales[e].copy_(scales.to(self.scales.dtype))
        if inv_s is not None:
            self.inv_s[e].copy_(inv_s.to(self.inv_s.dtype))

    def expert_weight(self, e: int, dtype=None) -> torch.Tensor:
        """Dequantized W_e with 1/s folded into its input columns:  Q(W s) diag(1/s)."""
        w = dequantize(unpack_int4(self.qweight[e]), self.scales[e].float(), self.qzeros[e],
                       self.group_size, dtype=torch.float32)
        w = w * self.inv_s[e].float().unsqueeze(0)
        return w.to(dtype or self.scales.dtype)

    # --- tensor-like interface used by eager expert forward code -----------------
    def __getitem__(self, e):
        return self.expert_weight(int(e))

    @property
    def shape(self):
        return torch.Size((self.num_experts, self.out_features, self.in_features))

    def size(self, dim=None):
        return self.shape if dim is None else self.shape[dim]

    @property
    def dtype(self):
        return self.scales.dtype

    @property
    def device(self):
        return self.scales.device

    @property
    def ndim(self):
        return 3

    def extra_repr(self) -> str:
        return (f"experts={self.num_experts}, out={self.out_features}, in={self.in_features}, "
                f"bits={self.w_bit}, group={self.group_size}")
```

awq/blocks.py (138 lines)

```
"""
blocks.py — find the repeated decoder blocks and the weights inside them, for ANY architecture.

Plain words:
  * Every LLM is "embedding -> N identical blocks -> norm -> output head".
    `find_blocks` finds that list of N blocks by looking for the biggest nn.ModuleList
    whose children are all the same class. No model names are hard-coded.
  * Only weights INSIDE blocks are quantized. The embedding and the output head stay 16-bit
    (that is the AWQ standard, and it is why we never need to know the head's name).
  * `quantizable_linears` lists every nn.Linear in a block, skipping MoE routers, tiny layers,
    and anything you listed in `quant.skip_modules`.
```

```
  * `fused_experts` finds transformers>=5 style MoE weights: one 3-D tensor (experts, out, in).
  * `fold_target` answers: "after scaling this group's weights by s, which earlier layer can
    absorb the 1/s for free?"  (RMSNorm/LayerNorm before attention or MLP, or up_proj before
    down_proj.) If none, the scale is stored inside the quantized layer instead.
"""
from __future__ import annotations

import re

import torch.nn as nn

ROUTER_NAMES = {"gate", "router", "shared_expert_gate", "gate_proj_router"}
# Linears whose outputs feed attention scores: the official AWQ skips weight clipping for them.
NO_CLIP_PATTERNS = ("q_", "k_", "query", "key", "Wqkv", "qkv", "q_a_proj", "q_b_proj", "kv_a_proj",
"kv_b_proj")
FOLD_FAMILIES = {"llama", "mistral", "qwen2", "qwen3", "gemma", "gemma2", "gemma3_text", "granite",
                 "mixtral", "qwen2_moe", "qwen3_moe", "olmo", "smollm3", "ministral"}


def _numel(module: nn.Module) -> int:
    return sum(p.numel() for p in module.parameters())


def find_blocks(model: nn.Module) -> tuple[str, nn.ModuleList]:
    """Return (dotted_name, ModuleList) of the repeated decoder blocks."""
    best, best_n = None, -1
    for name, mod in model.named_modules():
        if isinstance(mod, nn.ModuleList) and len(mod) >= 1:
            classes = {type(c).__name__ for c in mod}
            if len(classes) != 1:
                continue
            n = _numel(mod)
            if n > best_n:
                best, best_n = (name, mod), n
    if best is None:
        raise RuntimeError("Could not find a list of repeated decoder blocks in this model.")
    return best


def model_type(model_or_config) -> str:
    cfg = getattr(model_or_config, "config", model_or_config)
    text = cfg.get_text_config() if hasattr(cfg, "get_text_config") else cfg
    return str(getattr(text, "model_type", getattr(cfg, "model_type", "")))


def quantizable_linears(block: nn.Module, group_size: int, skip_patterns=()) -> tuple[dict, dict]:
    """Return ({name: nn.Linear} to quantize, {name: reason} skipped)."""
    keep, skipped = {}, {}
    for name, m in block.named_modules():
        if not isinstance(m, nn.Linear):
            continue
        leaf = name.split(".")[-1]
        g = m.in_features if group_size <= 0 else group_size
        if any(p in name for p in skip_patterns):
            skipped[name] = "listed in quant.skip_modules"
        elif leaf in ROUTER_NAMES or m.out_features < 32:
            skipped[name] = "MoE router / tiny layer (kept 16-bit on purpose)"
        elif m.in_features % g != 0 or m.in_features % 2 != 0:
            skipped[name] = f"in_features={m.in_features} not divisible by group_size={g}"
        else:
            keep[name] = m
    return keep, skipped


def fused_experts(block: nn.Module) -> list[tuple[str, str, nn.Module]]:
    """[(module_name, param_name, module)] for 3-D expert weights used as W[e] in eager forward."""
    out = []
    for name, m in block.named_modules():
        if getattr(m, "is_transposed", False):
            continue          # (E, in, out) layout: not supported, stays 16-bit
        for pname, p in m.named_parameters(recurse=False):
            if p.ndim == 3 and pname in ("gate_up_proj", "down_proj", "up_proj"):
                out.append((name, pname, m))
    return out


def allow_clip(name: str) -> bool:
    leaf = name.split(".")[-1]
    return not any(leaf.startswith(p) or p == leaf for p in NO_CLIP_PATTERNS)
```

```
def _get(block: nn.Module, dotted: str):
    m = block
    for p in dotted.split("."):
        if not hasattr(m, p):
            return None
        m = getattr(m, p)
    return m


def fold_target(block: nn.Module, group: list[str], family: str):
    """Where can 1/s be absorbed for this group of linears (names relative to block)?

    Returns (kind, module_name) with kind in {"norm", "norm_offset", "linear_out"} or None.
    Only for families whose block structure we know (FOLD_FAMILIES); everything else keeps 1/s
    inside the quantized layer, which is always correct.
    """
    if family not in FOLD_FAMILIES:
        return None
    leaves = {g.split(".")[-1] for g in group}
    parent = group[0].rsplit(".", 1)[0] if "." in group[0] else ""

    def norm(name):
        m = _get(block, name)
        if m is None or not hasattr(m, "weight") or m.weight is None:
            return None
        offset = "Gemma" in type(m).__name__ or family.startswith("gemma")
        return ("norm_offset" if offset else "norm", name)

    if leaves == {"q_proj", "k_proj", "v_proj"}:
        return norm("input_layernorm")
    if leaves == {"gate_proj", "up_proj"} and re.fullmatch(r"(mlp|feed_forward)", parent or ""):
        # Gemma2/3 put a norm AFTER attention ("post_attention_layernorm") and a separate
        # "pre_feedforward_layernorm" before the MLP — the MLP input comes from the latter.
        return norm("pre_feedforward_layernorm") or norm("post_attention_layernorm")
    if leaves == {"down_proj"}:
        up = _get(block, f"{parent}.up_proj") if parent else None
        gate = _get(block, f"{parent}.gate_proj") if parent else None
        if isinstance(up, nn.Linear) and isinstance(gate, nn.Linear):
            return ("linear_out", f"{parent}.up_proj")
        return None
    if leaves == {"o_proj"}:
        v = _get(block, f"{parent}.v_proj")
        o = _get(block, group[0])
        if isinstance(v, nn.Linear) and v.out_features == o.in_features:
            return ("linear_out", f"{parent}.v_proj")
        return None
    return None
```

awq/calib.py (102 lines)

```
"""
calib.py — text in, exact-length token windows out.

Plain words:
  AWQ needs to *watch* the model work on real text to learn which input channels matter.
  We take documents, tokenize them, glue them together with an end-of-text token, and cut
  the stream into `n_samples` windows of exactly `seq_len` tokens.

  NO padding is used. (The original Week-37 guide used padding="max_length", which crashes on
  Llama tokenizers — they have no pad token — and pollutes the statistics with pad tokens.)

Datasets:
  calibration : pileval (official AWQ), c4, wikitext2, or your own .jsonl with {"text": ...}
  evaluation  : wikitext2 (test), wikitext2_val (validation, used only for tuning), c4 (validation)
"""
from __future__ import annotations

import json
import random
from pathlib import Path

import torch


def _iter_texts(name: str, split_hint: str = "train"):
```

```
    from datasets import load_dataset

    if name == "pileval":
        ds = load_dataset("mit-han-lab/pile-val-backup", split="validation", streaming=True)
        for ex in ds:
            yield ex["text"]
    elif name == "c4":
        f = "en/c4-train.00000-of-01024.json.gz" if split_hint == "train" else "en/c4-validation.00000-of-
00008.json.gz"
        ds = load_dataset("allenai/c4", data_files={"x": f}, split="x", streaming=True)
        for ex in ds:
            yield ex["text"]
    elif name in ("wikitext2", "wikitext2_val", "wikitext2_train"):
        split = {"wikitext2": "test", "wikitext2_val": "validation", "wikitext2_train": "train"}[name]
        if name == "wikitext2" and split_hint == "train":
            split = "train"
        ds = load_dataset("Salesforce/wikitext", "wikitext-2-raw-v1", split=split)
        for ex in ds:
            yield ex["text"]
    elif Path(name).exists():
        with open(name) as fh:
            for line in fh:
                line = line.strip()
                if line:
                    yield json.loads(line)["text"]
    else:
        raise ValueError(f"Unknown dataset {name!r}. Use pileval | c4 | wikitext2 | path/to/file.jsonl")


def get_calib_tokens(tokenizer, dataset: str, n_samples: int, seq_len: int, seed: int = 42) -> torch.Tensor:
    """Return LongTensor (n_samples, seq_len) of calibration tokens."""
    rng = random.Random(seed)
    need = n_samples * seq_len
    eos = tokenizer.eos_token_id if tokenizer.eos_token_id is not None else 0
    # Take a pool of documents, shuffle it with the seed, then fill the token budget.
    pool = []
    for i, text in enumerate(_iter_texts(dataset, "train")):
        if text and len(text.strip()) > 20:
            pool.append(text)
        if len(pool) >= max(4 * n_samples, 512) and sum(len(t) for t in pool) > need * 6:
            break
    rng.shuffle(pool)
    ids: list[int] = []
    for text in pool:
        ids.extend(tokenizer(text, add_special_tokens=False)["input_ids"])
        ids.append(eos)
        if len(ids) >= need:
            break
    if len(ids) < need:
        raise RuntimeError(f"Calibration data too small: got {len(ids)} tokens, need {need}.")
    return torch.tensor(ids[:need], dtype=torch.long).view(n_samples, seq_len)


def get_eval_tokens(tokenizer, dataset: str, seq_len: int, max_windows: int = 0) -> torch.Tensor:
    """Return LongTensor (n_windows, seq_len) of non-overlapping evaluation windows."""
    if dataset in ("wikitext2", "wikitext2_val"):
        texts = list(_iter_texts(dataset, "eval"))
        text = "\n\n".join(texts)
    elif dataset == "c4":
        texts = []
        for i, t in enumerate(_iter_texts("c4", "validation")):
            texts.append(t)
            if i >= 1100:            # same budget as the GPTQ/AWQ papers' c4 eval slice
                break
        text = "\n\n".join(texts)
    elif Path(dataset).exists():
        text = "\n\n".join(_iter_texts(dataset))
    else:
        raise ValueError(f"Unknown eval dataset {dataset!r}")
    ids = tokenizer(text, return_tensors="pt", add_special_tokens=False).input_ids[0]
    n = ids.numel() // seq_len
    if dataset == "c4":
        n = min(n, 256)
    if max_windows and max_windows > 0:
        n = min(n, max_windows)
    if n == 0:
        raise RuntimeError(f"{dataset}: fewer than seq_len={seq_len} tokens")
    return ids[: n * seq_len].view(n, seq_len)
```

awq/cli.py (18 lines)

```
"""cli.py — the shared command-line flags every script accepts: --config and --set key=value."""
from __future__ import annotations

import argparse

from .config import load_config


def parser(description: str, need_config: bool = True) -> argparse.ArgumentParser:
    p = argparse.ArgumentParser(description=description)
    p.add_argument("--config", required=need_config, help="path to configs/run.yaml")
    p.add_argument("--set", action="append", default=[], metavar="KEY=VALUE",
                   help="override a config value, e.g. --set quant.group_size=64 (repeatable)")
    return p


def run_config(args):
    return load_config(args.config, overrides=args.set)
```

awq/config.py (181 lines)

```
"""
config.py — read configs/run.yaml into typed Python objects.

Plain words: every script in this repo calls `load_config("configs/run.yaml")`.
This file fills in defaults for anything you did not write, refuses obviously
wrong values *before* you spend GPU money, and lets you override any value
from the command line with `--set quant.group_size=64`.
"""
from __future__ import annotations

import copy
from dataclasses import MISSING, asdict, dataclass, field, fields, is_dataclass
from pathlib import Path
from typing import Any

import yaml


@dataclass
class QuantConfig:
    w_bit: int = 4                      # bits per weight. v1 packs 4-bit only.
    group_size: int = 128               # weights sharing one scale (128 = standard AWQ)
    zero_point: bool = True             # True = asymmetric (standard AWQ format), False = symmetric
    skip_modules: list[str] = field(default_factory=list)  # name substrings kept in 16-bit
    fold_scales: str = "auto"           # auto = fold 1/s into previous op when the family is known
                                        # never = keep 1/s inside every layer (any architecture)
    quantize_experts: bool = True       # quantize MoE expert weights (fused 3D or nn.Linear)


@dataclass
class SearchConfig:
    n_grid: int = 20                    # alpha values tried in [0, 1)
    duo_scaling: bool = True            # s = x_mean^a / w_mean^(1-a)  (official AWQ default)
    max_search_tokens: int = 0          # 0 = use every calibration token; >0 = random subsample (CPU speed)
    clip: bool = True                   # AWQ weight-clipping search
    clip_grid: int = 20
    max_shrink: float = 0.5             # clip search tries max|w| * (1 .. 1-max_shrink)
    clip_tokens: int = 512              # tokens used by the clip search (official value)
    propagate: str = "fp"               # next block input: fp (official AWQ) | quantized (GPTQ-style)


@dataclass
class CalibConfig:
    dataset: str = "pileval"            # pileval | c4 | wikitext2 | /path/to/file.jsonl ({"text": ...})
    n_samples: int = 128
    seq_len: int = 512
    batch_size: int = 8                 # sequences per forward pass through a block


@dataclass
class EvalConfig:
    ppl_datasets: list[str] = field(default_factory=lambda: ["wikitext2", "c4"])
    ppl_seq_len: int = 2048
```

```
    max_windows: int = 0                # 0 = full dataset; >0 = first N windows (CPU / quick checks)
    lm_eval_tasks: list[str] = field(
        default_factory=lambda: ["arc_easy", "arc_challenge", "hellaswag", "piqa", "winogrande"])
    lm_eval_limit: int | None = None    # docs per task (None = all)
    lm_eval_batch_size: str = "auto"
    gen_max_new_tokens: int = 48
    custom_jsonl: str | None = None     # your own {"prompt":..., "expected":...} lines
    tune_dataset: str = "wikitext2_val"  # held-out split used ONLY by 06_tune.py
    tune_max_windows: int = 40


@dataclass
class IOConfig:
    work_dir: str = "runs/default"


@dataclass
class RunConfig:
    model: str = ""
    revision: str = "main"
    trust_remote_code: bool = False
    dtype: str = "bfloat16"             # compute dtype (use float32 on CPU)
    device: str = "cuda"                # cuda | cpu
    plugin: str | None = None           # path to your own-model plugin .py (see awq/plugin.py)
    seed: int = 42
    quant: QuantConfig = field(default_factory=QuantConfig)
    search: SearchConfig = field(default_factory=SearchConfig)
    calib: CalibConfig = field(default_factory=CalibConfig)
    eval: EvalConfig = field(default_factory=EvalConfig)
    io: IOConfig = field(default_factory=IOConfig)

    # ---- helpers -----------------------------------------------------------
    def to_dict(self) -> dict:
        return asdict(self)

    def save(self, path: str | Path) -> None:
        Path(path).parent.mkdir(parents=True, exist_ok=True)
        Path(path).write_text(yaml.safe_dump(self.to_dict(), sort_keys=False))

    @property
    def work_dir(self) -> Path:
        return Path(self.io.work_dir)

    @property
    def torch_dtype(self):
        import torch
        return {"float32": torch.float32, "float16": torch.float16, "bfloat16": torch.bfloat16}[self.dtype]


def _build(cls, data: dict | None):
    """Turn a plain dict into the dataclass `cls`, recursing into nested dataclasses."""
    data = dict(data or {})
    known = {f.name: f for f in fields(cls)}
    unknown = set(data) - set(known)
    if unknown:
        raise ValueError(f"Unknown config key(s) in [{cls.__name__}]: {sorted(unknown)}")
    kwargs = {}
    for name, f in known.items():
        if name not in data:
            continue
        default = f.default_factory() if f.default_factory is not MISSING else f.default
        if is_dataclass(default):
            kwargs[name] = _build(type(default), data[name])
        else:
            kwargs[name] = data[name]
    return cls(**kwargs)


def _parse_value(text: str) -> Any:
    """'64' -> 64, 'true' -> True, '[a,b]' -> ['a','b'] (YAML rules)."""
    return yaml.safe_load(text)


def apply_overrides(cfg_dict: dict, overrides: dict[str, Any] | list[str] | None) -> dict:
    """Apply {'quant.group_size': 64} or ['quant.group_size=64'] onto a nested dict."""
    out = copy.deepcopy(cfg_dict)
    if not overrides:
        return out
    if isinstance(overrides, list):
```

```
        pairs = {}
        for item in overrides:
            if "=" not in item:
                raise ValueError(f"--set expects key=value, got {item!r}")
            k, v = item.split("=", 1)
            pairs[k.strip()] = _parse_value(v)
        overrides = pairs
    for dotted, value in overrides.items():
        node = out
        parts = dotted.split(".")
        for p in parts[:-1]:
            node = node.setdefault(p, {})
        node[parts[-1]] = value
    return out


def validate(cfg: RunConfig) -> RunConfig:
    """Fail fast, with a message that says what to change."""
    if not cfg.model:
        raise ValueError("config: `model:` is empty. Put a Hugging Face id or a local folder path.")
    if cfg.quant.w_bit != 4:
        raise ValueError("config: quant.w_bit must be 4 (this implementation packs 4-bit only).")
    if cfg.quant.group_size not in (-1, 32, 64, 128, 256):
        raise ValueError("config: quant.group_size must be one of -1 (per-channel), 32, 64, 128, 256.")
    if cfg.quant.fold_scales not in ("auto", "never"):
        raise ValueError("config: quant.fold_scales must be 'auto' or 'never'.")
    if cfg.search.propagate not in ("fp", "quantized"):
        raise ValueError("config: search.propagate must be 'fp' or 'quantized'.")
    if cfg.dtype not in ("float32", "float16", "bfloat16"):
        raise ValueError("config: dtype must be float32, float16 or bfloat16.")
    if cfg.device not in ("cuda", "cpu"):
        raise ValueError("config: device must be cuda or cpu.")
    if cfg.device == "cpu" and cfg.dtype != "float32":
        raise ValueError("config: on CPU use dtype: float32 (half-precision matmuls on CPU are
slow/unsupported).")
    if cfg.calib.n_samples % cfg.calib.batch_size != 0:
        raise ValueError("config: calib.n_samples must be divisible by calib.batch_size.")
    if not 0 < cfg.search.max_shrink < 1:
        raise ValueError("config: search.max_shrink must be in (0, 1).")
    return cfg


def load_config(path: str | Path | None = None, overrides=None, **top_level) -> RunConfig:
    """Load YAML -> apply --set overrides -> build dataclasses -> validate."""
    raw: dict = {}
    if path:
        raw = yaml.safe_load(Path(path).read_text()) or {}
    raw.update({k: v for k, v in top_level.items() if v is not None})
    raw = apply_overrides(raw, overrides)
    return validate(_build(RunConfig, raw))
```

awq/evaluate.py (152 lines)

```
"""
evaluate.py — measure what quantization cost you, honestly.

  perplexity()      : how surprised the model is by real text (lower = better). Standard metric.
  generations()     : same prompts, greedy decoding, FP16 vs AWQ side by side (catches broken output).
  speed()           : tokens/second for a fixed prompt (this repo's reference kernels are SLOW by design;
                      fast serving = export to vLLM).
  model_bytes()     : real memory footprint of all weights/buffers actually allocated.
  run_lm_eval()     : downstream accuracy through EleutherAI lm-evaluation-harness (optional dep).
  apply_rtn_()      : naive round-to-nearest baseline (no AWQ) — the number AWQ must beat.
"""
from __future__ import annotations

import math
import time
from pathlib import Path

import torch
import torch.nn.functional as F

from .blocks import find_blocks, fused_experts, quantizable_linears
from .model_io import checkpoint_is_fp8, force_eager_experts, load_hf_config
from .quantizer import fake_quant
```

```
DEFAULT_PROMPTS = [
    "The capital of France is",
    "Explain in one sentence why the sky is blue:",
    "def fibonacci(n):\n    \"\"\"Return the n-th Fibonacci number.\"\"\"\n",
    "Translate to French: 'Good morning, how are you?' ->",
    "Q: If a train travels 60 km in 1.5 hours, what is its average speed?\nA:",
    "The three primary colors are",
    "Write a haiku about the ocean:\n",
    "In machine learning, overfitting means",
    "import numpy as np\n# compute the mean of each column of matrix X\n",
    "The theory of relativity was proposed by",
]


def input_device(model) -> torch.device:
    return model.get_input_embeddings().weight.device


def load_fp_model(path, dtype, device: str, trust_remote_code: bool = False):
    from transformers import AutoModelForCausalLM

    kwargs = dict(dtype=dtype, trust_remote_code=trust_remote_code, low_cpu_mem_usage=True)
    if device == "cuda":
        kwargs["device_map"] = "auto"
    if checkpoint_is_fp8(load_hf_config(path, trust_remote_code)):
        from transformers import FineGrainedFP8Config
        kwargs["quantization_config"] = FineGrainedFP8Config(dequantize=True)
    model = AutoModelForCausalLM.from_pretrained(str(path), **kwargs)
    force_eager_experts(model)
    return model.eval()


@torch.no_grad()
def apply_rtn_(model, w_bit: int, group_size: int, zero_point: bool, skip_patterns=()) -> int:
    """In-place naive RTN on exactly the weights AWQ would quantize. Returns #tensors changed."""
    _, blocks = find_blocks(model)
    n = 0
    for block in blocks:
        lin, _ = quantizable_linears(block, group_size, skip_patterns)
        for m in lin.values():
            m.weight.data = fake_quant(m.weight.data, w_bit, group_size, zero_point)
            n += 1
        for _, pname, mod in fused_experts(block):
            p = getattr(mod, pname)
            for e in range(p.shape[0]):
                p.data[e] = fake_quant(p.data[e], w_bit, group_size, zero_point)
            n += 1
    return n


@torch.no_grad()
def perplexity(model, tokens: torch.Tensor, log=None, desc="ppl") -> float:
    dev = input_device(model)
    nll, count = 0.0, 0
    for i in range(tokens.shape[0]):
        ids = tokens[i:i + 1].to(dev)
        logits = model(input_ids=ids, use_cache=False).logits[:, :-1].float()
        target = ids[:, 1:].to(logits.device)
        nll += F.cross_entropy(logits.reshape(-1, logits.shape[-1]), target.reshape(-1),
reduction="sum").item()
        count += target.numel()
        if log and (i + 1) % max(1, tokens.shape[0] // 5) == 0:
            log(f"  [{desc}] {i + 1}/{tokens.shape[0]} windows  running ppl={math.exp(nll / count):.4f}")
    return math.exp(nll / count)


@torch.no_grad()
def generations(model, tok, prompts=None, max_new_tokens: int = 48) -> list[str]:
    prompts = prompts or DEFAULT_PROMPTS
    dev = input_device(model)
    outs = []
    for p in prompts:
        ids = tok(p, return_tensors="pt").input_ids.to(dev)
        g = model.generate(ids, max_new_tokens=max_new_tokens, do_sample=False,
                           pad_token_id=tok.pad_token_id if tok.pad_token_id is not None else
tok.eos_token_id)
        outs.append(tok.decode(g[0, ids.shape[1]:], skip_special_tokens=True))
    return outs
```

```
@torch.no_grad()
def speed(model, tok, new_tokens: int = 64) -> dict:
    dev = input_device(model)
    ids = tok("The history of computing", return_tensors="pt").input_ids.to(dev)
    pad = tok.pad_token_id if tok.pad_token_id is not None else tok.eos_token_id
    model.generate(ids, max_new_tokens=4, do_sample=False, pad_token_id=pad)        # warm-up
    if dev.type == "cuda":
        torch.cuda.synchronize()
    t = time.time()
    g = model.generate(ids, max_new_tokens=new_tokens, min_new_tokens=new_tokens, do_sample=False,
pad_token_id=pad)
    if dev.type == "cuda":
        torch.cuda.synchronize()
    dt = time.time() - t
    n = g.shape[1] - ids.shape[1]
    return {"new_tokens": int(n), "seconds": round(dt, 3), "tokens_per_s": round(n / dt, 2)}


def model_bytes(model) -> int:
    seen, total = set(), 0
    for t in list(model.parameters()) + list(model.buffers()):
        if t.data_ptr() in seen or t.device.type == "meta":
            continue
        seen.add(t.data_ptr())
        total += t.numel() * t.element_size()
    return total


def first_token_agreement(logits_a: torch.Tensor, logits_b: torch.Tensor) -> float:
    return (logits_a.argmax(-1) == logits_b.argmax(-1)).float().mean().item()


def run_lm_eval(model, tok, tasks, limit=None, batch_size="auto") -> dict:
    """Downstream accuracy via lm-evaluation-harness. Returns {task: {metric: value}}."""
    try:
        from lm_eval import simple_evaluate
        from lm_eval.models.huggingface import HFLM
    except ImportError as e:
        raise RuntimeError("lm-eval not installed: pip install -e '.[eval]'") from e
    lm = HFLM(pretrained=model, tokenizer=tok, batch_size=batch_size)
    res = simple_evaluate(model=lm, tasks=list(tasks), limit=limit)
    out = {}
    for task, metrics in res["results"].items():
        out[task] = {k.split(",")[0]: v for k, v in metrics.items()
                     if isinstance(v, (int, float)) and not k.startswith("alias") and "stderr" not in k}
    return out


def read_custom_jsonl(path: str | Path) -> list[dict]:
    import json
    return [json.loads(line) for line in Path(path).read_text().splitlines() if line.strip()]
```

awq/export.py (149 lines)

```
"""
export.py — turn our native output into the STANDARD AWQ "GEMM" format that vLLM / SGLang load.

Plain words:
  Our native format stores codes as (out, in/2) uint8. The ecosystem format (AutoAWQ "GEMM",
  understood by vLLM's awq / awq_marlin kernels) stores, per Linear:
      qweight : int32 (in, out/8)   eight 4-bit codes per int32, packed along OUTPUT channels,
                                    in the interleaved order [0, 2, 4, 6, 1, 3, 5, 7]
      qzeros  : int32 (in/g, out/8) zero points, packed the same way
      scales  : fp16  (in/g, out)
  plus `quantization_config` in config.json. Dequant rule:  W[o, i] = (q[i, o] - z[i/g, o]) * s[i/g, o].

  This only works when every AWQ scale was folded into the previous layer (no `inv_s` left),
  because vLLM has no slot for a per-input-channel 1/s. Use `quant.fold_scales: auto` on a
  supported family. Fused MoE experts are not exported (stay native).
"""
from __future__ import annotations

import json
import shutil
from pathlib import Path
```

```
import torch

from .quantizer import unpack_int4

AWQ_ORDER = [0, 2, 4, 6, 1, 3, 5, 7]
AWQ_REVERSE_ORDER = [0, 4, 1, 5, 2, 6, 3, 7]


def _to_int32(x64: torch.Tensor) -> torch.Tensor:
    x64 = x64 & 0xFFFFFFFF
    return torch.where(x64 >= 2 ** 31, x64 - 2 ** 32, x64).to(torch.int32)


def gemm_pack_codes(codes_in_out: torch.Tensor) -> torch.Tensor:
    """(rows, out) integer codes 0..15 -> (rows, out/8) int32 in AWQ interleaved order."""
    rows, out_f = codes_in_out.shape
    if out_f % 8:
        raise ValueError(f"out_features={out_f} must be divisible by 8 for the AWQ GEMM format")
    c = codes_in_out.to(torch.int64).view(rows, out_f // 8, 8)[..., AWQ_ORDER]
    shifts = torch.arange(0, 32, 4, dtype=torch.int64)
    return _to_int32((c << shifts).sum(-1))


def gemm_unpack_codes(packed: torch.Tensor) -> torch.Tensor:
    """Inverse of gemm_pack_codes: (rows, out/8) int32 -> (rows, out) int64 codes."""
    rows, cols = packed.shape
    p = packed.to(torch.int64) & 0xFFFFFFFF
    shifts = torch.arange(0, 32, 4, dtype=torch.int64)
    nib = (p.unsqueeze(-1) >> shifts) & 0xF                      # (rows, cols, 8) in packed order
    return nib[..., AWQ_REVERSE_ORDER].reshape(rows, cols * 8)


def to_gemm(qweight_native: torch.Tensor, qzeros_native: torch.Tensor, scales_native: torch.Tensor):
    """Native (out, in/2) uint8 / (out, ng) / (out, ng) -> GEMM qweight, qzeros, scales."""
    codes = unpack_int4(qweight_native)                          # (out, in)
    qweight = gemm_pack_codes(codes.t().contiguous())            # (in, out/8)
    qzeros = gemm_pack_codes(qzeros_native.t().contiguous())     # (ng, out/8)
    scales = scales_native.t().contiguous().to(torch.float16)    # (ng, out)
    return qweight, qzeros, scales


def gemm_dequant(qweight: torch.Tensor, qzeros: torch.Tensor, scales: torch.Tensor, group_size: int):
    """Reference dequantization of the GEMM format -> (out, in) float32. Used by tests."""
    codes = gemm_unpack_codes(qweight).float()                   # (in, out)
    zeros = gemm_unpack_codes(qzeros).float()                    # (ng, out)
    in_f = codes.shape[0]
    g = in_f if group_size <= 0 else group_size
    idx = torch.arange(in_f) // g
    w = (codes - zeros[idx]) * scales.float()[idx]
    return w.t().contiguous()


def export_autoawq(qdir: str | Path, out_dir: str | Path, shard_gb: float = 4.0, log=print) -> Path:
    from safetensors import safe_open
    from safetensors.torch import save_file

    qdir, out_dir = Path(qdir), Path(out_dir)
    qc = json.loads((qdir / "quant_config.json").read_text())
    if qc.get("n_linears_with_inv_s", 0):
        raise RuntimeError(
            f"{qc['n_linears_with_inv_s']} layers keep a runtime 1/s (scale could not be folded). The standard
"
            "AWQ format has no place for it. Re-quantize with quant.fold_scales=auto on a supported family "
            "(llama, mistral, qwen2/3, gemma*, ...) or ship the native format.")
    if qc.get("n_expert_tensors_quantized", 0):
        raise RuntimeError("Fused MoE experts were quantized natively; the GEMM export does not cover them. "
                           "Ship the native format, or re-run with quant.quantize_experts=false.")
    if qc.get("group_size", 128) <= 0:
        raise RuntimeError("Per-channel (group_size=-1) is not supported by the GEMM export.")
    out_dir.mkdir(parents=True, exist_ok=True)
    quant_names = set(qc["linears"])
    tensors, shards, cur_bytes = {}, [], 0
    limit = int(shard_gb * 1e9)

    def flush():
        nonlocal tensors, cur_bytes
        if tensors:
            shards.append(dict(tensors))
            tensors, cur_bytes = {}, 0
```

```
    for f in sorted(qdir.glob("*.safetensors")):
        with safe_open(str(f), framework="pt") as h:
            keys = list(h.keys())
            done = set()
            for k in keys:
                base, _, leaf = k.rpartition(".")
                if base in quant_names:
                    if base in done:
                        continue
                    qw, qz, sc = to_gemm(h.get_tensor(base + ".qweight"), h.get_tensor(base + ".qzeros"),
                                         h.get_tensor(base + ".scales"))
                    new = {base + ".qweight": qw, base + ".qzeros": qz, base + ".scales": sc}
                    if base + ".bias" in keys:
                        new[base + ".bias"] = h.get_tensor(base + ".bias").to(torch.float16)
                    done.add(base)
                else:
                    t = h.get_tensor(k)
                    new = {k: t.to(torch.float16) if t.is_floating_point() else t}
                for nk, nv in new.items():
                    tensors[nk] = nv.contiguous()
                    cur_bytes += nv.numel() * nv.element_size()
                if cur_bytes > limit:
                    flush()
    flush()
    weight_map = {}
    for i, sd in enumerate(shards):
        name = "model.safetensors" if len(shards) == 1 else f"model-{i + 1:05d}-of-
{len(shards):05d}.safetensors"
        save_file(sd, str(out_dir / name), metadata={"format": "pt"})
        weight_map.update({k: name for k in sd})
    if len(shards) > 1:
        index = {"metadata": {}, "weight_map": weight_map}
        (out_dir / "model.safetensors.index.json").write_text(json.dumps(index, indent=2))

    for f in qdir.iterdir():
        if f.is_file() and not f.name.endswith(".safetensors") and f.name not in ("quant_config.json",
"config.json"):
            shutil.copy2(f, out_dir / f.name)
    cfg = json.loads((qdir / "config.json").read_text())
    skipped = sorted({k.rsplit(".", 1)[-1] for k in qc.get("skipped", {})})
    cfg["quantization_config"] = {
        "quant_method": "awq", "bits": qc["w_bit"], "group_size": qc["group_size"],
        "zero_point": qc["zero_point"], "version": "gemm",
        "modules_to_not_convert": skipped or None,
    }
    cfg["torch_dtype"] = "float16"
    cfg["dtype"] = "float16"
    (out_dir / "config.json").write_text(json.dumps(cfg, indent=2))
    log(f"[export] AutoAWQ-GEMM format -> {out_dir}  ({len(quant_names)} quantized linears, {len(shards)}
shard(s))")
    return out_dir
```

awq/hardware.py (302 lines)

```
"""
hardware.py — "which GPU, how many, how long, how much?" (Option B of the playbook, as code).

Plain words:
  1. Read the model's config.json (a few KB — no weights are downloaded).
  2. Build the model on the "meta" device to count EXACTLY how many parameters sit in
     quantizable Linear layers, in MoE experts, and outside the blocks (embeddings/head).
     If the skeleton cannot be built, fall back to formulas from config fields.
  3. Apply the sizing formulas for the three phases:
        A. quantize      -> one block + calibration activations
        B. FP16 baseline -> the whole 16-bit model
        C. serve AWQ     -> the 4-bit model + KV cache
  4. For each phase pick the cheapest (GPU type x count) from configs/gpus.yaml.

Every number printed is an ESTIMATE from formulas; the quantize-speed constant is a guess
until calibrated by a real GPU run (see docs/).
"""
from __future__ import annotations

import math
from pathlib import Path
```

```
import yaml

ROOT = Path(__file__).resolve().parents[1]
QUANT_MIN_PER_BPARAM_A100 = 3.0     # ESTIMATE: minutes of quantization per billion params on an A100
DOWNLOAD_MB_S = 200.0               # typical cloud download speed from the HF Hub


def load_gpus(path: str | Path | None = None) -> dict:
    return yaml.safe_load(Path(path or ROOT / "configs" / "gpus.yaml").read_text())


def _cfg_get(cfg, *names, default=None):
    for n in names:
        v = getattr(cfg, n, None)
        if v is not None:
            return v
    return default


def config_dims(config) -> dict:
    t = config.get_text_config() if hasattr(config, "get_text_config") else config
    h = _cfg_get(t, "hidden_size", "d_model", "n_embd")
    heads = _cfg_get(t, "num_attention_heads", "n_head")
    d = {
        "model_type": getattr(t, "model_type", None),
        "layers": _cfg_get(t, "num_hidden_layers", "n_layer", "num_layers"),
        "hidden": h,
        "intermediate": _cfg_get(t, "intermediate_size", "ffn_dim", "n_inner", default=4 * (h or 0)),
        "heads": heads,
        "kv_heads": _cfg_get(t, "num_key_value_heads", "num_kv_heads", default=heads),
        "head_dim": _cfg_get(t, "head_dim", default=(h // heads) if h and heads else None),
        "vocab": _cfg_get(t, "vocab_size"),
        "tie_word_embeddings": bool(getattr(t, "tie_word_embeddings", getattr(config, "tie_word_embeddings",
False))),
        "experts": _cfg_get(t, "num_local_experts", "num_experts", "n_routed_experts", default=0) or 0,
        "experts_per_tok": _cfg_get(t, "num_experts_per_tok", "moe_topk", default=0) or 0,
        "moe_intermediate": _cfg_get(t, "moe_intermediate_size", default=0) or 0,
        "max_context": _cfg_get(t, "max_position_embeddings", "n_positions", "seq_length"),
        "dtype": str(_cfg_get(t, "dtype", "torch_dtype", default=_cfg_get(config, "dtype", "torch_dtype"))),
    }
    # kv_lora (DeepSeek MLA) caches a compressed latent instead of full K/V
    if getattr(t, "kv_lora_rank", None):
        d["kv_cache_width"] = int(t.kv_lora_rank + getattr(t, "qk_rope_head_dim", 0))
    return d


def count_params(model) -> dict:
    """Exact parameter accounting from a (meta) skeleton."""
    import torch.nn as nn

    from .blocks import find_blocks, fused_experts, quantizable_linears

    name, blocks = find_blocks(model)
    seen = set()
    total = 0
    for p in model.parameters():
        if id(p) in seen:
            continue
        seen.add(id(p))
        total += p.numel()
    per_block, lin_total, exp_total, n_lin, transposed_experts = [], 0, 0, 0, 0
    for b in blocks:
        lin, _ = quantizable_linears(b, 128)
        exps = fused_experts(b)
        lp = sum(m.weight.numel() for m in lin.values())
        ep = sum(getattr(m, pn).numel() for _, pn, m in exps)
        transposed_experts += sum(1 for _, m in b.named_modules() if getattr(m, "is_transposed", False))
        per_block.append(sum(p.numel() for p in b.parameters()))
        lin_total += lp
        exp_total += ep
        n_lin += len(lin)
    block_total = sum(per_block)
    # tied embeddings: the head re-uses the embedding matrix -> count it once
    tie = getattr(model.config.get_text_config() if hasattr(model.config, "get_text_config") else
model.config,
                  "tie_word_embeddings", False)
    out_emb, in_emb = model.get_output_embeddings(), model.get_input_embeddings()
    if tie and out_emb is not None and in_emb is not None and out_emb.weight is not in_emb.weight:
        total -= out_emb.weight.numel()
```

```
    return {
        "blocks_path": name,
        "n_blocks": len(blocks),
        "total_params": total,
        "block_params": block_total,
        "non_block_params": total - block_total,
        "max_block_params": max(per_block),
        "quant_linear_params": lin_total,
        "quant_expert_params": exp_total,
        "quantizable_params": lin_total + exp_total,
        "n_quant_linears": n_lin,
        "coverage_of_blocks": (lin_total + exp_total) / max(1, block_total),
        "transposed_expert_modules": transposed_experts,
        "has_nn_linear": any(isinstance(m, nn.Linear) for m in blocks[0].modules()),
    }


def formula_params(d: dict) -> dict:
    """Fallback when no skeleton can be built: standard decoder formulas from config fields."""
    h, L, inter = d["hidden"], d["layers"], d["intermediate"]
    hd = d["head_dim"] or h // d["heads"]
    attn = h * d["heads"] * hd * 2 + h * d["kv_heads"] * hd * 2
    if d["experts"]:
        mlp = d["experts"] * 3 * h * (d["moe_intermediate"] or inter)
    else:
        mlp = 3 * h * inter
    block = attn + mlp
    emb = d["vocab"] * h * (1 if d["tie_word_embeddings"] else 2)
    return {"blocks_path": "?", "n_blocks": L, "total_params": block * L + emb, "block_params": block * L,
            "non_block_params": emb, "max_block_params": block,
            "quant_linear_params": attn * L + (0 if d["experts"] else mlp * L),
            "quant_expert_params": mlp * L if d["experts"] else 0, "quantizable_params": block * L,
            "n_quant_linears": None, "coverage_of_blocks": 1.0, "transposed_expert_modules": 0,
            "has_nn_linear": True, "estimated_from_formula": True}


def _pick(vram_gb: float, gpus: list[dict], allowed=(1, 2, 3, 4, 5, 6, 7, 8)):
    """Cheapest (gpu, count) whose usable memory (90%) holds vram_gb, with count in `allowed`."""
    options = []
    for g in gpus:
        need = max(1, math.ceil(vram_gb / (0.9 * g["vram_gb"])))
        n = next((a for a in allowed if a >= need), None)
        if n is not None:
            options.append({"gpu": g["name"], "count": n, "usd_per_hr": round(n * g["usd_per_hr"], 2),
                            "speed": g.get("speed", 1.0)})
    options.sort(key=lambda o: (o["usd_per_hr"], o["count"]))
    return options


def estimate(counts: dict, dims: dict, ckpt_gb: float | None = None, n_samples: int = 128,
             seq_len: int = 512, context: int = 8192, batch: int = 1, gpus_cfg: dict | None = None) -> dict:
    gpus_cfg = gpus_cfg or load_gpus()
    gpus = gpus_cfg["gpus"]
    P = counts["total_params"] / 1e9
    Pq = counts["quantizable_params"] / 1e9
    fp16_gb = 2 * P
    awq_gb = 0.52 * Pq + 2 * (P - Pq)
    block_gb = 2 * counts["max_block_params"] / 1e9
    width = max(dims["intermediate"] or 0, (dims["experts_per_tok"] or 0) * (dims["moe_intermediate"] or 0),
                dims["hidden"] or 0)
    calib_gb = n_samples * seq_len * width * 2 / 1e9
    vram_quant = 2 * block_gb + 6 * calib_gb + 2
    vram_fp16 = 1.2 * fp16_gb
    kv_width = dims.get("kv_cache_width") or 2 * (dims["kv_heads"] or 0) * (dims["head_dim"] or 0)
    kv_gb = (dims["layers"] or 0) * kv_width * context * batch * 2 / 1e9
    vram_serve = 1.15 * (awq_gb + kv_gb)
    ckpt_gb = ckpt_gb if ckpt_gb is not None else fp16_gb
    disk_gb = ckpt_gb + awq_gb + 20
    ram_gb = max(16, 3 * block_gb + 2 * calib_gb + 8)
    download_h = ckpt_gb * 1000 / DOWNLOAD_MB_S / 3600

    phases = {}
    # A: one block must fit ONE GPU (the pipeline does not split a block across cards).
    # B: transformers device_map="auto" spreads layers over any number of GPUs.
    # C: vLLM tensor parallelism wants 1, 2, 4 or 8 GPUs.
    allowed = {"A_quantize": (1,), "B_fp16_eval": (1, 2, 3, 4, 5, 6, 7, 8), "C_serve_awq": (1, 2, 4, 8)}
    for key, vram in (("A_quantize", vram_quant), ("B_fp16_eval", vram_fp16), ("C_serve_awq", vram_serve)):
        opts = _pick(vram, gpus, allowed[key])
        best = opts[0] if opts else None
```

```
        if key == "A_quantize" and best:
            hours = P * QUANT_MIN_PER_BPARAM_A100 / 60 / best["speed"] + download_h + 0.5
        elif key == "B_fp16_eval" and best:
            hours = 0.3 + P / 100 + download_h * 0          # weights already on disk from phase A
        else:
            hours = 0.3
        phases[key] = {
            "vram_gb": round(vram, 1),
            "best": best,
            "alternatives": opts[1:4],
            "hours_est": round(hours, 2) if best else None,
            "cost_usd_est": round(best["usd_per_hr"] * hours * 1.25, 2) if best else None,
            "feasible_single_node": bool(opts),
        }
    return {
        "params_b": round(P, 3),
        "quantizable_params_b": round(Pq, 3),
        "fp16_gb": round(fp16_gb, 2),
        "awq_gb": round(awq_gb, 2),
        "compression_x": round(fp16_gb / awq_gb, 2) if awq_gb else None,
        "checkpoint_gb": round(ckpt_gb, 2),
        "block_gb": round(block_gb, 3),
        "calib_act_gb": round(calib_gb, 3),
        "kv_cache_gb": round(kv_gb, 3),
        "context": context,
        "disk_gb": round(disk_gb, 1),
        "cpu_ram_gb": round(ram_gb, 1),
        "download_hours": round(download_h, 2),
        "phases": phases,
        "prices_checked": gpus_cfg.get("checked"),
        "price_source": gpus_cfg.get("source"),
        "note": "ESTIMATES. Quantize speed constant is uncalibrated until a real GPU run; prices change
monthly.",
    }


def native_size_gb(counts: dict, dtype_bytes: int, group_size: int = 128) -> float:
    """Exact size of OUR native output: 4-bit codes + per-group (scale in run dtype + 1-byte zero),
    everything outside the quantized weights kept in the run dtype (fp32 on CPU, bf16/fp16 on GPU)."""
    pq, p = counts["quantizable_params"], counts["total_params"]
    g = group_size if group_size > 0 else 1
    return (pq * 0.5 + pq / g * (dtype_bytes + 1) + (p - pq) * dtype_bytes) / 1e9


def checkpoint_size_gb(model_id: str, revision: str = "main") -> float | None:
    p = Path(model_id).expanduser()
    try:
        if p.is_dir():
            files = list(p.glob("*.safetensors")) or list(p.glob("*.bin"))
            return sum(f.stat().st_size for f in files) / 1e9 if files else None
        from huggingface_hub import HfApi
        info = HfApi().model_info(model_id, revision=revision, files_metadata=True)
        sizes = [s.size for s in info.siblings if s.rfilename.endswith(".safetensors") and s.size]
        if not sizes:
            sizes = [s.size for s in info.siblings if s.rfilename.endswith(".bin") and s.size]
        return sum(sizes) / 1e9 if sizes else None
    except Exception:
        return None


def analyze_model(model_id: str, revision: str = "main", trust_remote_code: bool = False,
                  n_samples: int = 128, seq_len: int = 512, context: int = 8192, log=print) -> dict:
    import torch

    from .model_io import already_quantized_method, build_skeleton, checkpoint_is_fp8, load_hf_config

    config = load_hf_config(model_id, trust_remote_code)
    dims = config_dims(config)
    try:
        model = build_skeleton(model_id, torch.bfloat16, trust_remote_code)
        counts = count_params(model)
        del model
    except Exception as e:                                  # custom code that will not build, etc.
        log(f"[inspect] skeleton build failed ({type(e).__name__}: {str(e)[:160]}) -> formula estimate")
        counts = formula_params(dims)
    est = estimate(counts, dims, checkpoint_size_gb(model_id, revision), n_samples, seq_len, context)

    verdict, reasons = "PASS", []
    aq = already_quantized_method(config)
```

```
    if aq:
        verdict, reasons = "FAIL", [f"checkpoint is already quantized ({aq}). Use the original 16-bit/FP8
weights."]
    if counts["coverage_of_blocks"] < 0.5 and verdict != "FAIL":
        verdict = "WARN"
        reasons.append(f"only {counts['coverage_of_blocks']:.0%} of block weights are quantizable "
                       "(non-Linear compute or transposed experts) -> small memory saving")
    if counts.get("transposed_expert_modules"):
        reasons.append("some expert modules store transposed weights; they stay 16-bit")
    if checkpoint_is_fp8(config):
        reasons.append("FP8 checkpoint: dequantized to 16-bit block by block while loading (supported)")
    if not est["phases"]["B_fp16_eval"]["feasible_single_node"]:
        reasons.append("FP16 baseline does not fit one 8-GPU node -> skip it (--skip-fp16) and compare "
                       "against published scores")
    return {"model": model_id, "revision": revision, "verdict": verdict, "reasons": reasons,
            "dims": dims, "counts": counts, "estimate": est}


def format_report(r: dict) -> str:
    e, c, d = r["estimate"], r["counts"], r["dims"]
    lines = [
        f"MODEL      {r['model']}   [{d['model_type']}]   verdict: {r['verdict']}",
        *[f"           - {x}" for x in r["reasons"]],
        f"PARAMS     total {e['params_b']} B | quantizable {e['quantizable_params_b']} B "
        f"({c['coverage_of_blocks']:.0%} of block weights) | blocks {c['n_blocks']} | "
        f"{'MoE ' + str(d['experts']) + ' experts, top-' + str(d['experts_per_tok']) if d['experts'] else
'dense'}",
        f"SIZE       FP16 {e['fp16_gb']} GB -> AWQ-4bit {e['awq_gb']} GB  ({e['compression_x']}x)   "
        f"checkpoint on disk {e['checkpoint_gb']} GB",
        f"MEMORY     largest block {e['block_gb']} GB | calib activations {e['calib_act_gb']} GB | "
        f"KV@{e['context']} {e['kv_cache_gb']} GB | CPU RAM >= {e['cpu_ram_gb']} GB | disk >= {e['disk_gb']}
GB",
        "",
        f"{'PHASE':<14}{'VRAM need':>11}  {'RENT (cheapest)':<24}{'$/hr':>7}{'hours*':>8}{'cost*':>9}
alternatives",
    ]
    for k, label in (("A_quantize", "A quantize"), ("B_fp16_eval", "B FP16 eval"), ("C_serve_awq", "C serve
AWQ")):
        p = r["estimate"]["phases"][k]
        if p["best"]:
            b = p["best"]
            alts = ", ".join(f"{a['count']}x {a['gpu']} (${a['usd_per_hr']})" for a in p["alternatives"])
            lines.append(f"{label:<14}{p['vram_gb']:>9} GB  {str(b['count']) + 'x ' + b['gpu']:<24}"
                         f"{b['usd_per_hr']:>7}{p['hours_est']:>8}{'$' + str(p['cost_usd_est']):>9}   {alts}")
        else:
            why = ("one block does not fit the largest single GPU" if k == "A_quantize"
                   else "needs > 8 GPUs of the largest type (multi-node)")
            lines.append(f"{label:<14}{p['vram_gb']:>9} GB  {why}")
    lines += ["", f"* ESTIMATES. Prices: {e['price_source']} (checked {e['prices_checked']}). "
              "Edit configs/gpus.yaml for your provider."]
    return "\n".join(lines)
```

awq/model_io.py (457 lines)

```
"""
model_io.py — getting weights in and out WITHOUT ever needing the whole model in memory.

Plain words:
  1. `resolve_model_path`  turns "Qwen/Qwen2.5-7B" or "/workspace/models/x" into a local folder
     (downloading only the files we need: configs, tokenizer, *.safetensors).
  2. `build_skeleton`      builds the model's Python structure on the "meta" device: every layer
     exists, but no weight memory is allocated. A 1-trillion-parameter skeleton costs ~0 bytes.
  3. `WeightStore`         opens the safetensors shards lazily and reads ONE tensor at a time.
     It also handles: FP8 checkpoints (DeepSeek-style block scales), renamed keys, and the
     per-expert -> fused-experts layout that transformers >= 5 uses for MoE models.
  4. `ModelSource`         ties it together: load block i to the GPU, quantize it, throw it away.
     If the checkpoint layout cannot be streamed, it falls back to a normal full CPU load
     (then you need CPU RAM >= model size) and says so loudly.
  5. `load_quantized`      rebuilds a quantized model from our native output folder.
"""
from __future__ import annotations

import glob
import json
import re
from pathlib import Path
```

```
import torch
import torch.nn as nn

from .blocks import find_blocks

WEIGHT_PATTERNS = ["*.json", "*.safetensors", "tokenizer*", "*.model", "*.tiktoken", "*.txt",
                   "LICENSE*", "*.md", "NOTICE*", "USE_POLICY*"]


# ---------------------------------------------------------------------------
# 1. where are the files?
# ---------------------------------------------------------------------------
def resolve_model_path(model: str, revision: str = "main", trust_remote_code: bool = False,
                       weights: bool = True) -> Path:
    p = Path(model).expanduser()
    if p.is_dir():
        return p
    from huggingface_hub import list_repo_files, snapshot_download

    patterns = list(WEIGHT_PATTERNS) if weights else ["*.json", "tokenizer*", "*.model", "*.tiktoken",
"*.txt"]
    if trust_remote_code:
        patterns.append("*.py")
    if weights:
        files = list_repo_files(model, revision=revision)
        if not any(f.endswith(".safetensors") for f in files):
            patterns.append("*.bin")                      # old checkpoints without safetensors
    return Path(snapshot_download(model, revision=revision, allow_patterns=patterns))


# ---------------------------------------------------------------------------
# 2. the empty skeleton
# ---------------------------------------------------------------------------
def load_hf_config(path: Path | str, trust_remote_code: bool = False):
    from transformers import AutoConfig

    config = AutoConfig.from_pretrained(str(path), trust_remote_code=trust_remote_code)
    return config


def checkpoint_is_fp8(config) -> bool:
    qc = getattr(config, "quantization_config", None)
    if qc is None:
        return False
    method = qc.get("quant_method") if isinstance(qc, dict) else getattr(qc, "quant_method", None)
    return str(method).lower() == "fp8"


def already_quantized_method(config) -> str | None:
    """Return 'awq'/'gptq'/'bitsandbytes'/... if the checkpoint is already low-bit (wrong source)."""
    qc = getattr(config, "quantization_config", None)
    if qc is None:
        return None
    method = qc.get("quant_method") if isinstance(qc, dict) else getattr(qc, "quant_method", None)
    method = str(method).lower()
    return None if method == "fp8" else method


def _strip_quant_config(config):
    if hasattr(config, "quantization_config"):
        try:
            delattr(config, "quantization_config")
        except AttributeError:
            config.quantization_config = None
    return config


def force_eager_experts(model: nn.Module) -> None:
    """Fused-expert models must run the plain per-expert loop so quantized weights can be served."""
    for cfg in {id(m.config): m.config for m in model.modules() if hasattr(m, "config")}.values():
        if hasattr(cfg, "_experts_implementation"):
            cfg._experts_implementation = "eager"


def build_skeleton(path: Path | str, dtype: torch.dtype, trust_remote_code: bool = False):
    """Model structure on the meta device (no weight memory). Buffers like rotary tables are real."""
    from accelerate import init_empty_weights
    from transformers import AutoModelForCausalLM
```

```
    config = _strip_quant_config(load_hf_config(path, trust_remote_code))
    with init_empty_weights(include_buffers=False):
        model = AutoModelForCausalLM.from_config(config, dtype=dtype, trust_remote_code=trust_remote_code)
    model.eval()
    force_eager_experts(model)
    return model


# ---------------------------------------------------------------------------
# 3. lazy weight reader
# ---------------------------------------------------------------------------
_EXPERT_PAIRS = [("w1", "w3", "w2"), ("gate_proj", "up_proj", "down_proj")]
_RENAMES = [(".mlp.", ".block_sparse_moe."), (".mlp.", ".feed_forward.")]


class WeightStore:
    """Reads individual tensors from a checkpoint folder on demand."""

    def __init__(self, path: Path | str):
        self.path = Path(path)
        self.key_to_file: dict[str, str] = {}
        self._handles: dict[str, object] = {}
        self._bin_cache: dict[str, dict] = {}
        idx = self.path / "model.safetensors.index.json"
        st_files = sorted(glob.glob(str(self.path / "*.safetensors")))
        if idx.exists():
            self.key_to_file = {k: str(self.path / v) for k, v in json.loads(idx.read_text())
["weight_map"].items()}
            self.kind = "safetensors"
        elif st_files:
            from safetensors import safe_open
            for f in st_files:
                with safe_open(f, framework="pt") as h:
                    for k in h.keys():
                        self.key_to_file[k] = f
            self.kind = "safetensors"
        else:
            bidx = self.path / "pytorch_model.bin.index.json"
            if bidx.exists():
                wmap = json.loads(bidx.read_text())["weight_map"]
                self.key_to_file = {k: str(self.path / v) for k, v in wmap.items()}
            else:
                for f in sorted(glob.glob(str(self.path / "*.bin"))):
                    sd = torch.load(f, map_location="cpu", mmap=True, weights_only=True)
                    self._bin_cache[f] = sd
                    for k in sd:
                        self.key_to_file[k] = f
            self.kind = "bin"
        if not self.key_to_file:
            raise FileNotFoundError(f"No *.safetensors or *.bin weights found in {self.path}")

    def keys(self):
        return self.key_to_file.keys()

    def has(self, key: str) -> bool:
        return key in self.key_to_file

    def _raw(self, key: str) -> torch.Tensor:
        f = self.key_to_file[key]
        if self.kind == "safetensors":
            if f not in self._handles:
                from safetensors import safe_open
                self._handles[f] = safe_open(f, framework="pt")
            return self._handles[f].get_tensor(key)
        if f not in self._bin_cache:
            self._bin_cache[f] = torch.load(f, map_location="cpu", mmap=True, weights_only=True)
        return self._bin_cache[f][key]

    def get(self, key: str) -> torch.Tensor:
        """One tensor, FP8 block-dequantized to float32 if needed."""
        t = self._raw(key)
        if t.dtype in (getattr(torch, "float8_e4m3fn", None), getattr(torch, "float8_e5m2", None)):
            scale_key = key + "_scale_inv" if key.endswith("weight") else key.replace(".weight",
".weight_scale_inv")
            t = t.float()
            if self.has(scale_key):
                s = self._raw(scale_key).float()
                bo = -(-t.shape[0] // s.shape[0])          # block size along rows (128 for DeepSeek)
                bi = -(-t.shape[1] // s.shape[1])
```

```
                s = s.repeat_interleave(bo, 0)[: t.shape[0]].repeat_interleave(bi, 1)[:, : t.shape[1]]
                t = t * s
        return t

    # --- map a model parameter name to checkpoint tensor(s) -------------------
    def resolve(self, model_key: str) -> torch.Tensor:
        for cand in self._candidates(model_key):
            if self.has(cand):
                return self.get(cand)
        fused = self._fused_expert(model_key)
        if fused is not None:
            return fused
        raise KeyError(model_key)

    def _candidates(self, key: str):
        yield key
        for new, old in _RENAMES:
            if new in key:
                yield key.replace(new, old)

    def _fused_expert(self, key: str) -> torch.Tensor | None:
        m = re.match(r"(.*\.experts)\.(gate_up_proj|down_proj|up_proj)$", key)
        if not m:
            return None
        base, kind = m.groups()
        for prefix in self._candidates(base):
            for g_name, u_name, d_name in _EXPERT_PAIRS:
                first = f"{prefix}.0.{g_name}.weight"
                if not self.has(first):
                    continue
                n = 0
                while self.has(f"{prefix}.{n}.{g_name}.weight"):
                    n += 1
                if kind == "gate_up_proj":
                    gate = torch.stack([self.get(f"{prefix}.{e}.{g_name}.weight") for e in range(n)])
                    up = torch.stack([self.get(f"{prefix}.{e}.{u_name}.weight") for e in range(n)])
                    return torch.cat([gate, up], dim=1)
                name = d_name if kind == "down_proj" else u_name
                return torch.stack([self.get(f"{prefix}.{e}.{name}.weight") for e in range(n)])
        return None


def _set_tensor(module: nn.Module, name: str, value: torch.Tensor, device, dtype) -> None:
    from accelerate.utils import set_module_tensor_to_device

    if value.is_floating_point():
        value = value.to(dtype)
    set_module_tensor_to_device(module, name, device, value=value)


def materialize(module: nn.Module, prefix: str, store: WeightStore, device, dtype,
                skip_prefixes: tuple[str, ...] = ()) -> list[str]:
    """Fill every meta parameter of `module` from the checkpoint. Returns names that were missing."""
    missing = []
    for name, p in list(module.named_parameters()) + list(module.named_buffers()):
        full = prefix + name
        if any(full.startswith(s) for s in skip_prefixes):
            continue
        is_meta = p.device.type == "meta"
        try:
            value = store.resolve(full)
        except KeyError:
            if is_meta:
                missing.append(full)
            elif p.device != torch.device(device):
                _set_tensor(module, name, p.data, device, p.dtype if not p.is_floating_point() else dtype)
            continue
        _set_tensor(module, name, value, device, dtype)
    return missing


def _module_to_meta(module: nn.Module) -> None:
    """Release memory: turn every tensor of `module` back into a meta tensor."""
    module.to_empty(device="meta")


# ---------------------------------------------------------------------------
# 4. the streaming source used by the pipeline
# ---------------------------------------------------------------------------
```

```
class ModelSource:
    """Hands out one decoder block at a time.

    mode='stream': skeleton on meta + per-tensor reads (RAM ~ one block).
    mode='full'  : transformers.from_pretrained on CPU, blocks moved to GPU one by one
                   (fallback when the checkpoint layout cannot be mapped; RAM ~ model size).
    """

    def __init__(self, path: Path, dtype: torch.dtype, trust_remote_code: bool = False,
                 mode: str = "auto", log=print):
        self.path, self.dtype, self.trust_remote_code, self.log = Path(path), dtype, trust_remote_code, log
        self.config = load_hf_config(path, trust_remote_code)
        self.is_fp8 = checkpoint_is_fp8(self.config)
        self.mode = mode
        if mode in ("auto", "stream"):
            self.model = build_skeleton(path, dtype, trust_remote_code)
            self.store = WeightStore(path)
            self.blocks_name, self.blocks = find_blocks(self.model)
            if mode == "auto" and not self._layout_streamable():
                self.log("[model_io] checkpoint keys do not match the model layout -> FULL CPU LOAD fallback "
                         "(needs CPU RAM >= model size).")
                self.mode = "full"
            else:
                self.mode = "stream"
        if self.mode == "full":
            self.model = self._full_load()
            self.store = None
            self.blocks_name, self.blocks = find_blocks(self.model)
        self.log(f"[model_io] mode={self.mode}  blocks={self.blocks_name} x{len(self.blocks)}  fp8_ckpt=
{self.is_fp8}")

    def _layout_streamable(self) -> bool:
        """Check that every tensor of block 0 and the non-block parts can be found in the checkpoint."""
        try:
            for name, p in self.model.named_parameters():
                if name.startswith(self.blocks_name + ".") and not name.startswith(self.blocks_name + ".0."):
                    continue
                if self.store.has(name):
                    continue
                if self.tied and name.endswith("lm_head.weight"):
                    continue
                self.store.resolve(name)              # raises KeyError if not resolvable
            return True
        except KeyError as e:
            self.log(f"[model_io] unresolvable key: {e}")
            return False

    @property
    def tied(self) -> bool:
        cfg = self.config.get_text_config() if hasattr(self.config, "get_text_config") else self.config
        return bool(getattr(cfg, "tie_word_embeddings", False))

    def _full_load(self):
        from transformers import AutoModelForCausalLM

        kwargs = dict(dtype=self.dtype, trust_remote_code=self.trust_remote_code, low_cpu_mem_usage=True)
        if self.is_fp8:
            from transformers import FineGrainedFP8Config
            kwargs["quantization_config"] = FineGrainedFP8Config(dequantize=True)
        model = AutoModelForCausalLM.from_pretrained(str(self.path), **kwargs)
        model.eval()
        force_eager_experts(model)
        return model

    # non-block = embeddings, final norm, lm_head, rotary tables ...
    def _non_block_modules(self):
        b = self.blocks_name
        for name, mod in self.model.named_modules():
            if name == b or name.startswith(b + "."):
                continue
            yield name, mod

    def load_non_block(self, device) -> None:
        if self.mode == "stream":
            missing = materialize(self.model, "", self.store, device, self.dtype,
                                  skip_prefixes=(self.blocks_name + ".",))
            if self.tied:
                self._retie()
                missing = [m for m in missing if not m.endswith("lm_head.weight")]
```

```
            if missing:
                raise KeyError(f"Missing non-block weights in checkpoint: {missing[:5]}")
        for _, mod in self._non_block_modules():
            for n, p in list(mod._parameters.items()):
                if p is not None and p.device.type != "meta":
                    mod._parameters[n] = nn.Parameter(p.data.to(device), requires_grad=False)
            for n, buf in list(mod._buffers.items()):
                if buf is not None:
                    mod._buffers[n] = buf.to(device)
        if self.tied:
            self._retie()

    def _retie(self) -> None:
        out = self.model.get_output_embeddings()
        inp = self.model.get_input_embeddings()
        if out is not None and inp is not None:
            out.weight = inp.weight

    def unload_non_block(self) -> None:
        """Free embeddings/lm_head memory while blocks are being processed."""
        target = "meta" if self.mode == "stream" else "cpu"
        for _, mod in self._non_block_modules():
            for n, p in list(mod._parameters.items()):
                if p is None:
                    continue
                t = torch.empty_like(p, device="meta") if target == "meta" else p.data.to("cpu")
                mod._parameters[n] = nn.Parameter(t, requires_grad=False)

    def load_block(self, i: int, device) -> nn.Module:
        block = self.blocks[i]
        if self.mode == "stream":
            missing = materialize(block, f"{self.blocks_name}.{i}.", self.store, device, self.dtype)
            if missing:
                raise KeyError(f"Block {i}: weights missing from checkpoint: {missing[:5]}")
        block.to(device)
        return block

    def unload_block(self, i: int) -> None:
        if self.mode == "stream":
            _module_to_meta(self.blocks[i])
        else:
            self.blocks[i].to("cpu")

    def non_block_state_dict(self) -> dict[str, torch.Tensor]:
        """Original (unquantized) weights outside the blocks, as CPU tensors, for saving."""
        out = {}
        for name, p in self.model.named_parameters():
            if name.startswith(self.blocks_name + "."):
                continue
            if self.mode == "stream":
                try:
                    t = self.store.resolve(name)
                except KeyError:
                    if self.tied and name.endswith("lm_head.weight"):
                        continue
                    raise
            else:
                t = p.detach().cpu()
            out[name] = (t.to(self.dtype) if t.is_floating_point() else t).contiguous()
        return out


# ---------------------------------------------------------------------------
# 5. load our native quantized folder back
# ---------------------------------------------------------------------------
def _get_parent(root: nn.Module, dotted: str):
    parts = dotted.split(".")
    parent = root
    for p in parts[:-1]:
        parent = getattr(parent, p)
    return parent, parts[-1]


def load_quantized(qdir: str | Path, device="cpu", dtype: torch.dtype | None = None):
    """Rebuild the model from <work_dir>/quantized/. Only packed weights are ever allocated,
    so peak memory == real quantized footprint (this is how VRAM is measured honestly)."""
    from safetensors import safe_open

    from .awq_linear import AwqExpertsWeight, AwqLinear
```

```
    qdir = Path(qdir)
    qc = json.loads((qdir / "quant_config.json").read_text())
    if qc.get("plugin"):
        from .plugin import load_quantized_plugin
        return load_quantized_plugin(qdir, qc["plugin"], device)
    dtype = dtype or {"float32": torch.float32, "float16": torch.float16,
                      "bfloat16": torch.bfloat16}[qc["dtype"]]
    model = build_skeleton(qdir, dtype, trust_remote_code=qc.get("trust_remote_code", False))
    for name, info in qc["linears"].items():
        parent, attr = _get_parent(model, name)
        old = getattr(parent, attr)
        new = AwqLinear(old.in_features, old.out_features, old.bias is not None, qc["w_bit"],
                        qc["group_size"], has_inv_s=info["inv_s"], dtype=dtype, device="meta")
        setattr(parent, attr, new)
    for name, info in qc.get("experts", {}).items():
        parent, attr = _get_parent(model, name)
        e, o, i = info["shape"]
        parent._parameters.pop(attr, None)
        setattr(parent, attr, AwqExpertsWeight(e, o, i, qc["w_bit"], qc["group_size"], dtype=dtype,
device="meta"))

    from accelerate.utils import set_module_tensor_to_device

    loaded = set()
    for f in sorted(qdir.glob("*.safetensors")):
        with safe_open(str(f), framework="pt") as h:
            for k in h.keys():
                t = h.get_tensor(k)
                if t.is_floating_point():
                    t = t.to(dtype)
                parent, attr = _get_parent(model, k)
                set_module_tensor_to_device(parent, attr, device, value=t)
                loaded.add(k)
    if qc.get("tie_word_embeddings"):
        model.get_output_embeddings().weight = model.get_input_embeddings().weight
    still_meta = [n for n, p in list(model.named_parameters()) + list(model.named_buffers())
                  if p.device.type == "meta"]
    if still_meta:
        raise RuntimeError(f"Quantized folder is incomplete, missing: {still_meta[:5]}")
    model.to(device)
    force_eager_experts(model)
    model.eval()
    return model
```

awq/pipeline.py (431 lines)

```
"""
pipeline.py — quantize a whole model, ONE decoder block at a time.

Plain words, the loop:
  1. Tokenize calibration text -> (n_samples, seq_len) tokens.
  2. Run embeddings and stop right before block 0 ("Catcher") -> inputs of block 0.
  3. For block i = 0 .. L-1:
       a. load block i's weights (only this block is on the GPU)
       b. run it once with hooks: record each Linear's input; Linears that receive the very
          same tensor form a "group" (q/k/v, gate/up, ...). MoE experts record their routed tokens.
       c. AWQ scale search per group, apply the scales (fold 1/s upstream when possible)
       d. AWQ clip search per Linear
       e. replace each Linear with a 4-bit AwqLinear (experts -> AwqExpertsWeight)
       f. run the quantized block -> block error; outputs become inputs of block i+1
       g. save block i to disk + progress file  (so a killed pod can --resume here)
  4. Save embeddings/head (16-bit), config, tokenizer and quant_config.json.
"""
from __future__ import annotations

import json
import shutil
import time
from collections import OrderedDict, defaultdict
from pathlib import Path

import torch
import torch.nn as nn
import torch.nn.functional as F

from .awq_linear import AwqExpertsWeight, AwqLinear
```

```
from .blocks import (FOLD_FAMILIES, allow_clip, fold_target, fused_experts, model_type,
                     quantizable_linears)
from .calib import get_calib_tokens
from .config import RunConfig
from .model_io import ModelSource, resolve_model_path
from .search import apply_clip, search_clip, search_scale
from .utils import Logger, git_commit, now_iso, write_json


class _Stop(Exception):
    pass


class _Catcher(nn.Module):
    """Stands in for block 0: stores its inputs, then stops the forward pass."""

    def __init__(self, block: nn.Module):
        super().__init__()
        self.block = block
        self.hidden: list[torch.Tensor] = []
        self.kwargs: dict | None = None

    def __getattr__(self, name):
        try:
            return super().__getattr__(name)
        except AttributeError:
            return getattr(self._modules["block"], name)   # e.g. `.attention_type` read by the model

    def forward(self, hidden_states, *args, **kwargs):
        self.hidden.append(hidden_states.detach())
        if self.kwargs is None:
            self.kwargs = kwargs
        raise _Stop


def _clean_kwargs(kw: dict) -> dict:
    kw = dict(kw)
    for k in ("past_key_values", "past_key_value", "layer_past"):
        kw.pop(k, None)
    kw["use_cache"] = False
    return kw


@torch.no_grad()
def capture_block0_inputs(model: nn.Module, blocks: nn.ModuleList, batches, device):
    """batches: iterable of model-input dicts. Returns (inputs (N, T, h), layer_kwargs)."""
    is_hf = hasattr(model, "config") and hasattr(model, "generate")
    extra = {"use_cache": False} if is_hf else {}
    catcher = _Catcher(blocks[0])
    blocks[0] = catcher
    try:
        for batch in batches:
            batch = {k: (v.to(device) if torch.is_tensor(v) else v) for k, v in batch.items()}
            try:
                model(**batch, **extra)
            except _Stop:
                pass
    finally:
        blocks[0] = catcher.block
    kw = _clean_kwargs(catcher.kwargs or {}) if is_hf else dict(catcher.kwargs or {})
    return torch.cat(catcher.hidden, dim=0), kw


@torch.no_grad()
def run_block(block: nn.Module, inps: torch.Tensor, kwargs: dict, bs: int) -> torch.Tensor:
    outs = []
    for i in range(0, inps.shape[0], bs):
        o = block(inps[i:i + bs], **kwargs)
        outs.append(o[0] if isinstance(o, (tuple, list)) else o)
    return torch.cat(outs, dim=0)


class _ExpertRecorder:
    """While active, F.linear calls whose weight is expert e of a fused 3-D parameter are recorded."""

    def __init__(self, experts):
        self.map = {}
        self.feats = defaultdict(list)
        for mod_name, pname, mod in experts:
```

```
            p = getattr(mod, pname)
            for e in range(p.shape[0]):
                self.map[p[e].data_ptr()] = (f"{mod_name}.{pname}", e, tuple(p.shape[1:]))
        self._orig = None

    def __enter__(self):
        self._orig = F.linear
        rec = self

        def linear(input, weight, bias=None):
            hit = rec.map.get(weight.data_ptr()) if weight.dim() == 2 else None
            if hit is not None and tuple(weight.shape) == hit[2]:
                rec.feats[(hit[0], hit[1])].append(input.detach().reshape(-1, input.shape[-1]))
            return rec._orig(input, weight, bias)

        torch.nn.functional.linear = linear
        return self

    def __exit__(self, *exc):
        torch.nn.functional.linear = self._orig


@torch.no_grad()
def collect(block, inps, kwargs, bs, linears: dict, experts):
    """Run the FP block once. Returns (fp_outputs, {name: input tensor list}, groups, expert_feats)."""
    feats, first_key, handles = defaultdict(list), {}, []

    def make_hook(name):
        def hook(mod, args, kw, out):
            x = args[0] if args else kw.get("input")
            if name not in first_key:
                first_key[name] = (x.data_ptr(), tuple(x.shape), x.stride())
            feats[name].append(x.detach().reshape(-1, x.shape[-1]))
        return hook

    for name, m in linears.items():
        handles.append(m.register_forward_hook(make_hook(name), with_kwargs=True))
    rec = _ExpertRecorder(experts)
    try:
        with rec:
            outs = run_block(block, inps, kwargs, bs)
    finally:
        for h in handles:
            h.remove()
    groups: OrderedDict[tuple, list[str]] = OrderedDict()
    for name in linears:                      # module order = execution order for standard blocks
        if name in first_key:
            groups.setdefault(first_key[name], []).append(name)
    return outs, feats, list(groups.values()), rec.feats


def _fold(block, target, s: torch.Tensor):
    """Absorb x/s into the op that produces x (so no runtime cost)."""
    kind, name = target
    mod = block.get_submodule(name)
    w = mod.weight.data
    hp = torch.float64 if w.dtype == torch.float64 else torch.float32     # compute in >= fp32
    s = s.to(w.device, hp)
    if kind == "norm":
        mod.weight.data = (w.to(hp) / s).to(w.dtype)
    elif kind == "norm_offset":                    # Gemma RMSNorm multiplies by (1 + w)
        mod.weight.data = ((1 + w.to(hp)) / s - 1).to(w.dtype)
    elif kind == "linear_out":                     # previous Linear: scale its output rows
        mod.weight.data = (w.to(hp) / s.view(-1, 1)).to(w.dtype)
    if getattr(mod, "bias", None) is not None:
        mod.bias.data = (mod.bias.data.to(hp) / s).to(mod.bias.dtype)


@torch.no_grad()
def quantize_block(block: nn.Module, idx: int, inps: torch.Tensor, kwargs: dict, rc: RunConfig,
                   family: str, log) -> tuple[torch.Tensor, dict, dict]:
    q, sc = rc.quant, rc.search
    bs = rc.calib.batch_size
    linears, skipped = quantizable_linears(block, q.group_size, q.skip_modules)
    experts = fused_experts(block) if q.quantize_experts else []
    t0 = time.time()
    outs_fp, feats, groups, expert_feats = collect(block, inps, kwargs, bs, linears, experts)
    report = {"block": idx, "groups": [], "experts": {}, "skipped": skipped}
```

```
    # ---- (c) scale search for every group, then apply ------------------------------------
    inv_s_of: dict[str, torch.Tensor] = {}
    group_x: dict[str, torch.Tensor] = {}
    plans = []
    for names in groups:
        x = torch.cat(feats[names[0]], dim=0)
        for n in names:
            feats.pop(n, None)
        target = fold_target(block, names, family) if q.fold_scales == "auto" else None
        searchable = target is not None or q.fold_scales == "never" or family not in FOLD_FAMILIES
        ws = [linears[n].weight.data for n in names]
        if searchable:
            res = search_scale(ws, [linears[n].bias for n in names], x, q.w_bit, q.group_size,
                               q.zero_point, sc.n_grid, sc.duo_scaling, sc.max_search_tokens)
        else:
            res = None
        plans.append((names, target, res, x))
        report["groups"].append({
            "linears": names,
            "fold": None if target is None else f"{target[0]}:{target[1]}",
            "mode": "folded" if target else ("inv_s" if searchable else "no-scale (export-compatible)"),
            "ratio": None if res is None or res["ratio"] is None else round(res["ratio"], 3),
            "loss_awq": None if res is None else res["loss"],
            "loss_rtn": None if res is None else res["loss_rtn"],
        })
    for names, target, res, x in plans:
        if res is not None and res["ratio"] is not None:
            s = res["scales"].to(x.device)
            for n in names:
                w = linears[n].weight
                w.data = (w.data.float() * s.view(1, -1)).to(w.dtype)
            if target is not None:
                _fold(block, target, s)
            else:
                for n in names:
                    inv_s_of[n] = (1.0 / s)
            x = (x.float() / s.view(1, -1)).to(x.dtype)
        for n in names:
            group_x[n] = x

    # ---- (d) clip search + (e) quantize -------------------------------------------------------
    for name, lin in linears.items():
        if name not in group_x:
            skipped[name] = "never executed during calibration (kept 16-bit)"
            continue
        if sc.clip and allow_clip(name):
            mv = search_clip(lin.weight.data, group_x[name], q.w_bit, q.group_size, q.zero_point,
                             sc.clip_grid, sc.max_shrink, sc.clip_tokens)
            lin.weight.data = apply_clip(lin.weight.data, mv)
        new = AwqLinear.from_float(lin, q.w_bit, q.group_size, q.zero_point, inv_s_of.get(name))
        parent_name, _, attr = name.rpartition(".")
        setattr(block.get_submodule(parent_name) if parent_name else block, attr, new)
    group_x.clear()

    # ---- MoE fused experts: per-expert AWQ (scale kept inside, never folded) ------------------
    for mod_name, pname, mod in experts:
        key = f"{mod_name}.{pname}"
        p = getattr(mod, pname)
        E, out_f, in_f = p.shape
        if in_f % (in_f if q.group_size <= 0 else q.group_size) != 0:
            skipped[key] = "expert in_features not divisible by group_size"
            continue
        qmod = AwqExpertsWeight(E, out_f, in_f, q.w_bit, q.group_size, dtype=p.dtype, device=p.device)
        stats = {"experts": E, "with_tokens": 0, "tokens_min": None, "tokens_max": 0}
        for e in range(E):
            w = p[e].data
            xs = expert_feats.get((key, e))
            if xs:
                x = torch.cat(xs, dim=0)
                n_tok = x.shape[0]
                stats["with_tokens"] += 1
                stats["tokens_max"] = max(stats["tokens_max"], n_tok)
                stats["tokens_min"] = n_tok if stats["tokens_min"] is None else min(stats["tokens_min"],
n_tok)
                res = search_scale([w], [None], x, q.w_bit, q.group_size, q.zero_point, sc.n_grid,
                                   sc.duo_scaling, sc.max_search_tokens)
                inv = None
                if res["ratio"] is not None:
                    s = res["scales"].to(w.device)
```

```
                    w = (w.float() * s.view(1, -1)).to(w.dtype)
                    x = (x.float() / s.view(1, -1)).to(x.dtype)
                    inv = 1.0 / s
                if sc.clip:
                    w = apply_clip(w, search_clip(w, x, q.w_bit, q.group_size, q.zero_point,
                                                  sc.clip_grid, sc.max_shrink, sc.clip_tokens))
                qmod.set_expert(e, w, inv, q.zero_point)
            else:                                   # expert never routed to: plain RTN
                qmod.set_expert(e, w, None, q.zero_point)
        mod._parameters.pop(pname, None)      # a Parameter slot cannot hold a Module
        setattr(mod, pname, qmod)
        report["experts"][key] = stats
    expert_feats.clear()

    # ---- (f) quantized forward -> block error + next inputs ---------------------------------
    outs_q = run_block(block, inps, kwargs, bs)
    denom = outs_fp.float().pow(2).mean().item() + 1e-12
    report["block_rel_mse"] = (outs_q.float() - outs_fp.float()).pow(2).mean().item() / denom
    report["seconds"] = round(time.time() - t0, 2)
    meta = {
        "linears": {n: {"inv_s": n in inv_s_of} for n, m in block.named_modules() if isinstance(m,
AwqLinear)},
        "experts": {n: {"shape": [m.num_experts, m.out_features, m.in_features]}
                    for n, m in block.named_modules() if isinstance(m, AwqExpertsWeight)},
        "skipped": skipped,
    }
    nxt = outs_fp if sc.propagate == "fp" else outs_q
    del outs_q
    return nxt, report, meta


# ---------------------------------------------------------------------------
def _copy_side_files(src: Path, dst: Path, trust_remote_code: bool) -> None:
    pats = ["tokenizer*", "*.model", "*.tiktoken", "vocab*", "merges*", "special_tokens_map.json",
            "generation_config.json", "chat_template*", "LICENSE*", "NOTICE*", "USE_POLICY*"]
    if trust_remote_code:
        pats.append("*.py")
    for pat in pats:
        for f in src.glob(pat):
            if f.is_file():
                shutil.copy2(f, dst / f.name)
    cfg = json.loads((src / "config.json").read_text())
    cfg.pop("quantization_config", None)
    (dst / "config.json").write_text(json.dumps(cfg, indent=2))


def quantize_model(rc: RunConfig, resume: bool = False, log: Logger | None = None,
                   calib_tokens: torch.Tensor | None = None) -> Path:
    """Quantize rc.model into <work_dir>/quantized. `calib_tokens` overrides the dataset (tests)."""
    from safetensors.torch import save_file
    from transformers import AutoTokenizer

    work = rc.work_dir
    log = log or Logger(work / "quantize.log")
    torch.manual_seed(rc.seed)
    device = torch.device(rc.device)
    qdir, state = work / "quantized", work / "state"
    qdir.mkdir(parents=True, exist_ok=True)
    state.mkdir(parents=True, exist_ok=True)
    rc.save(work / "run_config.yaml")

    if rc.plugin:
        from .plugin import PluginSource
        src = PluginSource(rc, log)
        path, tok = None, src.tokenizer
    else:
        path = resolve_model_path(rc.model, rc.revision, rc.trust_remote_code)
        log(f"[quantize] model files: {path}")
        src = ModelSource(path, rc.torch_dtype, rc.trust_remote_code, log=log)
        tok = None if calib_tokens is not None else AutoTokenizer.from_pretrained(
            str(path), trust_remote_code=rc.trust_remote_code)
    family = model_type(src.model)
    n_blocks = len(src.blocks)
    log(f"[quantize] family={family} blocks={n_blocks} fold_scales={rc.quant.fold_scales} "
        f"(fold-capable family: {family in FOLD_FAMILIES})")

    progress_f = state / "progress.json"
    start = 0
    if resume and progress_f.exists():
```

```
        start = json.loads(progress_f.read_text())["next_block"]
        inps = torch.load(state / "inps.pt", map_location=device)
        kwargs = torch.load(state / "kwargs.pt", map_location=device, weights_only=False)
        log(f"[quantize] RESUME from block {start}/{n_blocks}")
    else:
        for f in qdir.glob("block-*.safetensors"):
            f.unlink()
        (work / "search_log.jsonl").unlink(missing_ok=True)
        src.load_non_block(device)
        if rc.plugin:
            batches = src.calib_batches(rc.calib.n_samples, rc.calib.seq_len, rc.calib.batch_size)
        else:
            calib = calib_tokens if calib_tokens is not None else get_calib_tokens(
                tok, rc.calib.dataset, rc.calib.n_samples, rc.calib.seq_len, rc.seed)
            src_name = "custom tokens" if calib_tokens is not None else rc.calib.dataset
            log(f"[quantize] calibration: {src_name} {tuple(calib.shape)} tokens")
            batches = ({"input_ids": calib[i:i + rc.calib.batch_size]}
                       for i in range(0, calib.shape[0], rc.calib.batch_size))
        inps, kwargs = capture_block0_inputs(src.model, src.blocks, batches, device)
        src.unload_non_block()
        torch.save(kwargs, state / "kwargs.pt")
        log(f"[quantize] block-0 inputs {tuple(inps.shape)} {inps.dtype}")

    for i in range(start, n_blocks):
        t = time.time()
        block = src.load_block(i, device)
        inps, report, meta = quantize_block(block, i, inps, kwargs, rc, family, log)
        prefix = f"{src.blocks_name}.{i}."
        sd = {prefix + k: v.detach().cpu().contiguous() for k, v in block.state_dict().items()}
        save_file(sd, str(qdir / f"block-{i:05d}.safetensors"))
        meta_full = {"linears": {prefix + k: v for k, v in meta["linears"].items()},
                     "experts": {prefix + k: v for k, v in meta["experts"].items()},
                     "skipped": {prefix + k: v for k, v in meta["skipped"].items()}}
        write_json(state / f"meta-{i:05d}.json", meta_full)
        src.unload_block(i)
        with open(work / "search_log.jsonl", "a") as fh:
            fh.write(json.dumps(report) + "\n")
        torch.save(inps, state / "inps.pt")
        progress_f.write_text(json.dumps({"next_block": i + 1, "n_blocks": n_blocks}))
        ratios = [g["ratio"] for g in report["groups"] if g["ratio"] is not None]
        log(f"[quantize] block {i + 1}/{n_blocks}  rel_mse={report['block_rel_mse']:.2e}  "
            f"alphas={ratios}  experts={len(report['experts'])}  {time.time() - t:.1f}s")
        if device.type == "cuda":
            torch.cuda.empty_cache()

    # ---- non-block weights + metadata ----------------------------------------------------------
    nb = src.non_block_state_dict()
    save_file(nb, str(qdir / "non_block.safetensors"))
    linears, experts, skipped = {}, {}, {}
    for f in sorted(state.glob("meta-*.json")):
        m = json.loads(f.read_text())
        linears.update(m["linears"])
        experts.update(m["experts"])
        skipped.update(m["skipped"])
    if path is not None:
        _copy_side_files(path, qdir, rc.trust_remote_code)
    else:
        src.save_side_files(qdir)
    qc = {
        "format": "awq-from-scratch-native-v1",
        "source_model": rc.model,
        "revision": rc.revision,
        "family": family,
        "w_bit": rc.quant.w_bit,
        "group_size": rc.quant.group_size,
        "zero_point": rc.quant.zero_point,
        "fold_scales": rc.quant.fold_scales,
        "dtype": rc.dtype,
        "trust_remote_code": rc.trust_remote_code,
        "tie_word_embeddings": bool(src.tied),
        "plugin": rc.plugin,
        "calibration": {"dataset": rc.calib.dataset, "n_samples": rc.calib.n_samples,
                        "seq_len": rc.calib.seq_len},
        "search": rc.to_dict()["search"],
        "n_linears_quantized": len(linears),
        "n_linears_with_inv_s": sum(1 for v in linears.values() if v["inv_s"]),
        "n_expert_tensors_quantized": len(experts),
        "linears": linears,
        "experts": experts,
```

```
        "skipped": skipped,
        "created_utc": now_iso(),
        "git_commit": git_commit(),
    }
    write_json(qdir / "quant_config.json", qc)
    log(f"[quantize] DONE -> {qdir}  ({len(linears)} linears, {len(experts)} expert tensors, "
        f"{len(skipped)} skipped)")
    return qdir
```

awq/plugin.py (118 lines)

```
"""
plugin.py — quantize YOUR OWN PyTorch model (not on the Hugging Face Hub, not a transformers class).

Write a small file, e.g. my_plugin.py, with three functions and point `plugin:` in run.yaml at it:

    def load():                                  # -> (model, tokenizer_or_None), FP weights loaded
        ...
    def blocks(model):                           # -> the nn.ModuleList of repeated blocks
        return model.layers
    def calib_batches(tokenizer, n_samples, seq_len, batch_size):
        # yield dicts of keyword arguments for model(**batch), batch_size samples each
        ...

Requirements (honest): the block's heavy compute must be nn.Linear layers, and each block must be
callable as block(hidden_states, **kwargs) returning a tensor (or a tuple whose first item is it).
The plugin model is loaded fully in CPU RAM (no streaming): fine up to ~tens of GB.
See tests/tiny_plugin.py for a complete working example.
"""
from __future__ import annotations

import importlib.util
from pathlib import Path

import torch
import torch.nn as nn


def load_plugin(path: str | Path):
    path = Path(path)
    spec = importlib.util.spec_from_file_location(f"awq_user_plugin_{path.stem}", path)
    mod = importlib.util.module_from_spec(spec)
    spec.loader.exec_module(mod)
    for fn in ("load", "blocks", "calib_batches"):
        if not hasattr(mod, fn):
            raise AttributeError(f"plugin {path} must define `{fn}()`")
    return mod


class PluginSource:
    """Same interface as model_io.ModelSource, backed by the user's plugin."""

    mode = "full"
    tied = False

    def __init__(self, rc, log=print):
        self.rc, self.log = rc, log
        self.mod = load_plugin(rc.plugin)
        self.model, self.tokenizer = self.mod.load()
        self.model.eval()
        self.blocks = self.mod.blocks(self.model)
        self.blocks_name = next(n for n, m in self.model.named_modules() if m is self.blocks)
        self.log(f"[plugin] {rc.plugin}: blocks={self.blocks_name} x{len(self.blocks)}")

    def _non_block(self):
        b = self.blocks_name
        for name, mod in self.model.named_modules():
            if name == b or name.startswith(b + "."):
                continue
            yield mod

    def load_non_block(self, device):
        for mod in self._non_block():
            for n, p in list(mod._parameters.items()):
                if p is not None:
                    mod._parameters[n] = nn.Parameter(p.data.to(device), requires_grad=False)
            for n, b in list(mod._buffers.items()):
```

```
                if b is not None:
                    mod._buffers[n] = b.to(device)

    def unload_non_block(self):
        self.load_non_block("cpu")

    def load_block(self, i, device):
        return self.blocks[i].to(device)

    def unload_block(self, i):
        self.blocks[i].to("cpu")

    def calib_batches(self, n_samples, seq_len, batch_size):
        return self.mod.calib_batches(self.tokenizer, n_samples, seq_len, batch_size)

    def non_block_state_dict(self):
        b = self.blocks_name + "."
        return {k: v.detach().cpu().contiguous() for k, v in self.model.state_dict().items()
                if not k.startswith(b)}

    def save_side_files(self, qdir):
        (Path(qdir) / "PLUGIN.txt").write_text(f"Built with plugin: {Path(self.rc.plugin).resolve()}\n")


def load_quantized_plugin(qdir, plugin_path, device="cpu"):
    """Rebuild a plugin model and swap in the quantized layers from <qdir>."""
    import json

    from safetensors import safe_open

    from .awq_linear import AwqExpertsWeight, AwqLinear
    from .model_io import _get_parent

    qc = json.loads((Path(qdir) / "quant_config.json").read_text())
    model, _ = load_plugin(plugin_path).load()
    dtype = {"float32": torch.float32, "float16": torch.float16, "bfloat16": torch.bfloat16}[qc["dtype"]]
    for name, info in qc["linears"].items():
        parent, attr = _get_parent(model, name)
        old = getattr(parent, attr)
        setattr(parent, attr, AwqLinear(old.in_features, old.out_features, old.bias is not None,
                                        qc["w_bit"], qc["group_size"], info["inv_s"], dtype))
    for name, info in qc.get("experts", {}).items():
        parent, attr = _get_parent(model, name)
        parent._parameters.pop(attr, None)
        setattr(parent, attr, AwqExpertsWeight(*info["shape"], qc["w_bit"], qc["group_size"], dtype))
    sd = {}
    for f in sorted(Path(qdir).glob("*.safetensors")):
        with safe_open(str(f), framework="pt") as h:
            for k in h.keys():
                sd[k] = h.get_tensor(k)
    model.load_state_dict(sd, strict=True)
    return model.to(device).eval()
```

awq/quantizer.py (96 lines)

```
"""
quantizer.py — the arithmetic of 4-bit group quantization. No AWQ yet, just numbers.

Plain words:
  * A weight matrix W has shape (out_features, in_features).
  * We cut every row into groups of `group_size` consecutive input channels.
  * Each group gets one `scale` (and one `zero` in asymmetric mode).
  * Every weight becomes an integer q in [0, 15]:      q = round(w / scale) + zero
  * To get the weight back (dequantize):               w ≈ (q - zero) * scale
  * Two 4-bit integers are packed into one uint8 byte, so storage is 0.5 byte/weight.

`fake_quant` = quantize then immediately dequantize. It returns a float matrix that
already contains the rounding error — that is what the AWQ search measures.
"""
from __future__ import annotations

import torch


def _group(in_features: int, group_size: int) -> int:
    """group_size=-1 means one group per whole row (per-channel)."""
    g = in_features if group_size <= 0 else group_size
```

```
    if in_features % g != 0:
        raise ValueError(f"in_features={in_features} is not divisible by group_size={g}")
    return g


@torch.no_grad()
def compute_qparams(w: torch.Tensor, w_bit: int = 4, group_size: int = 128, zero_point: bool = True):
    """Return (scales, zeros), both float32 with shape (out, n_groups).

    Asymmetric (zero_point=True): use min and max of each group  -> the 16 levels cover [min, max].
    Symmetric  (zero_point=False): use max|w| of each group       -> levels cover [-max, +max], zero fixed at
8.
    """
    out_f, in_f = w.shape
    g = _group(in_f, group_size)
    wg = w.float().reshape(out_f, in_f // g, g)
    qmax = 2 ** w_bit - 1                                  # 15 for 4-bit
    if zero_point:
        w_max = wg.amax(dim=-1)
        w_min = wg.amin(dim=-1)
        scales = (w_max - w_min).clamp(min=1e-5) / qmax
        zeros = (-torch.round(w_min / scales)).clamp(0, qmax)
    else:
        half = 2 ** (w_bit - 1)                            # 8 for 4-bit
        scales = wg.abs().amax(dim=-1).clamp(min=1e-5) / (half - 1)
        zeros = torch.full_like(scales, float(half))
    return scales, zeros


@torch.no_grad()
def quantize(w: torch.Tensor, scales: torch.Tensor, zeros: torch.Tensor,
             w_bit: int = 4, group_size: int = 128) -> torch.Tensor:
    """Float weights -> integer codes q in [0, 2^bits - 1] (uint8, shape (out, in))."""
    out_f, in_f = w.shape
    g = _group(in_f, group_size)
    wg = w.float().reshape(out_f, in_f // g, g)
    q = torch.round(wg / scales.unsqueeze(-1)) + zeros.unsqueeze(-1)
    q = q.clamp(0, 2 ** w_bit - 1)
    return q.reshape(out_f, in_f).to(torch.uint8)


def dequantize(q: torch.Tensor, scales: torch.Tensor, zeros: torch.Tensor,
               group_size: int = 128, dtype: torch.dtype = torch.float16) -> torch.Tensor:
    """Integer codes -> float weights:  w = (q - zero) * scale."""
    out_f, in_f = q.shape
    g = _group(in_f, group_size)
    qg = q.reshape(out_f, in_f // g, g).to(scales.dtype)
    w = (qg - zeros.to(scales.dtype).unsqueeze(-1)) * scales.unsqueeze(-1)
    return w.reshape(out_f, in_f).to(dtype)


@torch.no_grad()
def fake_quant(w: torch.Tensor, w_bit: int = 4, group_size: int = 128, zero_point: bool = True) ->
torch.Tensor:
    """Quantize + dequantize in one go. Same shape and dtype as the input."""
    scales, zeros = compute_qparams(w, w_bit, group_size, zero_point)
    q = quantize(w, scales, zeros, w_bit, group_size)
    return dequantize(q, scales, zeros, group_size, dtype=w.dtype)


# ---------------------------------------------------------------------------
# Packing: two 4-bit codes per byte.  byte = low nibble (even column) | high nibble (odd column) << 4
# ---------------------------------------------------------------------------
def pack_int4(q: torch.Tensor) -> torch.Tensor:
    """(..., in) uint8 codes in [0,15] -> (..., in/2) uint8."""
    if q.shape[-1] % 2:
        raise ValueError("pack_int4 needs an even number of columns")
    q = q.to(torch.uint8)
    return (q[..., 0::2] | (q[..., 1::2] << 4)).contiguous()


def unpack_int4(p: torch.Tensor) -> torch.Tensor:
    """Inverse of pack_int4: (..., in/2) uint8 -> (..., in) uint8 codes."""
    low = p & 0x0F
    high = (p >> 4) & 0x0F
    return torch.stack([low, high], dim=-1).reshape(*p.shape[:-1], p.shape[-1] * 2)
```

awq/search.py (153 lines)

```
"""
search.py — the AWQ algorithm itself (Lin et al. 2023, arXiv:2306.00978).

Plain words, scale search (`search_scale`):
  For a group of layers that read the same input x (e.g. q/k/v):
    1. importance of input channel j:   x_mean[j] = mean |x[:, j]|
    2. try ratio a = 0, 1/n, 2/n, ...:   s = x_mean^a / w_mean^(1-a)        (duo scaling)
                                         s = s / sqrt(max(s) * min(s))      (keep magnitudes sane)
    3. pretend-quantize the scaled weight and undo the scale:  W_test = Q(W * s) / s
    4. error = || x W^T - x W_test^T ||^2   (how much the layer output changed)
    5. keep the s with the smallest error.
  s = 1 (plain round-to-nearest) is always tried first, so on calibration data AWQ can never be
  worse than RTN — it only replaces RTN when some scale measurably lowers the error.

Plain words, clip search (`search_clip`):
  One huge weight in a group stretches that group's scale, wasting the 16 levels.
  Try shrinking each group's max |w| to 100%, 95%, ... 50% and keep the cut that minimises
  the group's output error. Skipped for q/k projections (attention scores are sensitive).
"""
from __future__ import annotations

import math

import torch

from .quantizer import fake_quant


def _chunks(n: int, size: int):
    for i in range(0, n, size):
        yield slice(i, min(i + size, n))


@torch.no_grad()
def _subsample(x: torch.Tensor, max_tokens: int, seed: int = 0) -> torch.Tensor:
    if max_tokens and x.shape[0] > max_tokens:
        g = torch.Generator(device="cpu").manual_seed(seed)
        idx = torch.randperm(x.shape[0], generator=g)[:max_tokens].to(x.device)
        return x.index_select(0, idx)
    return x


@torch.no_grad()
def search_scale(weights: list[torch.Tensor], biases: list[torch.Tensor | None], x: torch.Tensor,
                 w_bit: int, group_size: int, zero_point: bool, n_grid: int = 20,
                 duo_scaling: bool = True, max_tokens: int = 0, token_chunk: int = 4096):
    """weights: list of (out_i, in) matrices sharing input x (n, in).

    Returns dict(scales=(in,) float32, ratio=float or None (None = plain RTN won),
                 loss=float, loss_rtn=float, losses=list).
    """
    x = _subsample(x, max_tokens)
    dev, dt = weights[0].device, weights[0].dtype
    x = x.to(dev, dt)
    in_f = x.shape[-1]
    x_mean = x.abs().float().mean(0)                                    # (in,)

    if duo_scaling:
        wcat = torch.cat(weights, dim=0).float()
        g = in_f if group_size <= 0 else group_size
        wg = wcat.abs().view(-1, g)
        w_scale = (wg / (wg.amax(dim=1, keepdim=True) + 1e-6)).view(wcat.shape)
        w_mean = w_scale.mean(0)                                        # (in,)
        del wcat, wg, w_scale
    else:
        w_mean = None

    # output error of a candidate:  || x W^T - x W_test^T ||^2 = || x (W - W_test)^T ||^2
    # -> ONE matmul per candidate, computed chunk by chunk to bound memory (bias cancels).
    def outputs_error(test_weights):
        diffs = [(w_ref - w_t) for w_ref, w_t in zip(weights, test_weights)]
        err, count = 0.0, 0
        for sl in _chunks(x.shape[0], token_chunk):
            xb = x[sl]
            for d in diffs:
                err += (xb @ d.T).float().pow(2).sum().item()
```

```
                count += xb.shape[0] * d.shape[0]
        return err / max(count, 1)

    # candidate 0: s = 1 (plain round-to-nearest). AWQ only replaces it if a scale is better.
    loss_rtn = outputs_error([fake_quant(w, w_bit, group_size, zero_point) for w in weights])
    best = dict(loss=loss_rtn, ratio=None, scales=torch.ones(in_f, device=dev))
    losses = []
    for i in range(n_grid):
        ratio = i / n_grid
        if w_mean is not None:
            s = (x_mean.pow(ratio) / (w_mean.pow(1 - ratio) + 1e-4)).clamp(min=1e-4)
        else:
            s = x_mean.pow(ratio).clamp(min=1e-4)
        s = s / (s.max() * s.min()).sqrt()
        if not torch.isfinite(s).all():
            losses.append(math.inf)
            continue
        s_dt = s.to(dt)
        tests = [fake_quant(w * s_dt.view(1, -1), w_bit, group_size, zero_point) / s_dt.view(1, -1)
                 for w in weights]
        loss = outputs_error(tests)
        losses.append(loss)
        if loss < best["loss"]:
            best = dict(loss=loss, ratio=ratio, scales=s.clone())
    best["loss_rtn"] = loss_rtn
    best["losses"] = losses
    best["scales"] = best["scales"].float()
    return best


@torch.no_grad()
def search_clip(w: torch.Tensor, x: torch.Tensor, w_bit: int, group_size: int, zero_point: bool,
                clip_grid: int = 20, max_shrink: float = 0.5, n_tokens: int = 512,
                elem_budget: int = 2 ** 27) -> torch.Tensor:
    """Return best max|w| per (out, group) — shape (out, n_groups, 1). `w` is already scaled.
    `x` must already be divided by the AWQ scale (it is the input the scaled weight will see)."""
    out_f, in_f = w.shape
    g = in_f if group_size <= 0 else group_size
    ng = in_f // g
    # evenly spaced token subsample (official AWQ uses 512)
    if x.shape[0] > n_tokens:
        step = max(1, x.shape[0] // n_tokens)
        x = x[::step][:n_tokens]
    x = x.to(w.device, w.dtype)
    n = x.shape[0]
    xg = x.view(n, ng, g)                                                  # (n, ng, g)
    oc_batch = max(1, min(out_f, elem_budget // max(1, n * ng)))
    best_max_all = []
    for sl in _chunks(out_f, oc_batch):
        wb = w[sl].view(-1, ng, g)                                         # (ob, ng, g)
        # partial output of every group: out[o, t, k] = sum_j x[t, k, j] * w[o, k, j]  (batched matmul)
        org_out = torch.einsum("nkg,okg->onk", xg, wb)                     # (ob, n, ng)
        org_max = wb.abs().amax(dim=-1, keepdim=True)                      # (ob, ng, 1)
        best_max = org_max.clone()
        min_err = torch.full((wb.shape[0], ng), float("inf"), device=w.device)
        for i in range(int(max_shrink * clip_grid)):
            max_val = org_max * (1 - i / clip_grid)
            cur = torch.clamp(wb, -max_val, max_val)
            q = fake_quant(cur.reshape(wb.shape[0], in_f), w_bit, g, zero_point).view(-1, ng, g)
            cur_out = torch.einsum("nkg,okg->onk", xg, q)
            err = (cur_out - org_out).float().pow(2).mean(dim=1)           # (ob, ng)
            better = err < min_err
            min_err[better] = err[better]
            best_max[better] = max_val[better]
        best_max_all.append(best_max)                                      # (ob, ng, 1)
        del org_out, wb
    return torch.cat(best_max_all, dim=0)


@torch.no_grad()
def apply_clip(w: torch.Tensor, max_val: torch.Tensor) -> torch.Tensor:
    out_f, in_f = w.shape
    ng = max_val.shape[1]
    g = in_f // ng
    wg = w.view(out_f, ng, g)
    mv = max_val.to(w.dtype)
    return torch.clamp(wg, -mv, mv).view(out_f, in_f)
```

awq/utils.py (88 lines)

```
"""
utils.py — small helpers shared by every script: logging, JSON, environment info, timing.
"""
from __future__ import annotations

import datetime as _dt
import json
import os
import platform
import subprocess
import sys
import time
from pathlib import Path


class Logger:
    """print() that also appends to a log file, with elapsed time."""

    def __init__(self, path: str | Path | None = None):
        self.t0 = time.time()
        self.path = Path(path) if path else None
        if self.path:
            self.path.parent.mkdir(parents=True, exist_ok=True)

    def __call__(self, *msg):
        line = f"[{time.time() - self.t0:8.1f}s] " + " ".join(str(m) for m in msg)
        print(line, flush=True)
        if self.path:
            with open(self.path, "a") as fh:
                fh.write(line + "\n")


def write_json(path: str | Path, obj) -> None:
    Path(path).parent.mkdir(parents=True, exist_ok=True)
    Path(path).write_text(json.dumps(obj, indent=2, default=str))


def read_json(path: str | Path):
    return json.loads(Path(path).read_text())


def now_iso() -> str:
    return _dt.datetime.now(_dt.timezone.utc).replace(microsecond=0).isoformat()


def git_commit() -> str | None:
    try:
        root = Path(__file__).resolve().parents[1]
        return subprocess.check_output(["git", "-C", str(root), "rev-parse", "--short", "HEAD"],
                                       stderr=subprocess.DEVNULL).decode().strip()
    except Exception:
        return None


def env_info() -> dict:
    import torch

    info = {
        "time_utc": now_iso(),
        "git_commit": git_commit(),
        "python": sys.version.split()[0],
        "platform": platform.platform(),
        "cpu_count": os.cpu_count(),
        "torch": torch.__version__,
        "cuda_available": torch.cuda.is_available(),
        "torch_cuda": torch.version.cuda,
    }
    try:
        import transformers
        info["transformers"] = transformers.__version__
    except Exception:
        pass
    try:
        info["ram_gb"] = round(os.sysconf("SC_PAGE_SIZE") * os.sysconf("SC_PHYS_PAGES") / 1e9, 1)
    except Exception:
        pass
```

```
    if torch.cuda.is_available():
        info["gpus"] = [
            {"name": torch.cuda.get_device_name(i),
             "vram_gb": round(torch.cuda.get_device_properties(i).total_memory / 1e9, 1)}
            for i in range(torch.cuda.device_count())
        ]
        info["bf16_supported"] = torch.cuda.is_bf16_supported()
    return info


def dir_size_gb(path: str | Path, pattern: str = "*.safetensors") -> float:
    return round(sum(f.stat().st_size for f in Path(path).glob(pattern)) / 1e9, 4)
```

scripts/00_env_check.py (97 lines)

```
#!/usr/bin/env python
"""
00_env_check.py — GATE 0. Is this machine ready? Prints every version that matters and the exact
fix for anything that is wrong. Ends with "ENV OK" or "ENV FAIL".

    python scripts/00_env_check.py                       # generic check
    python scripts/00_env_check.py --config configs/run.yaml   # also checks disk vs. this model
"""
import shutil
import sys
import time
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parents[1]))
from awq.cli import parser, run_config  # noqa: E402
from awq.utils import env_info, write_json  # noqa: E402


def main():
    p = parser(__doc__, need_config=False)
    args = p.parse_args()
    rc = run_config(args) if args.config else None
    problems, warnings = [], []
    info = env_info()
    for k, v in info.items():
        print(f"  {k:<16} {v}")

    import torch
    want_cuda = rc is None or rc.device == "cuda"
    if want_cuda and not torch.cuda.is_available():
        (problems if rc else warnings).append(
            "CUDA not available. On a GPU box: check `nvidia-smi`, then reinstall torch with "
            "`pip install torch --index-url https://download.pytorch.org/whl/cu128` (cuXYZ <= driver's CUDA
Version).")
    if torch.cuda.is_available():
        x = torch.randn(4096, 4096, device="cuda", dtype=torch.float16)
        torch.cuda.synchronize()
        t = time.time()
        for _ in range(20):
            x @ x
        torch.cuda.synchronize()
        tflops = 20 * 2 * 4096 ** 3 / (time.time() - t) / 1e12
        info["fp16_matmul_tflops"] = round(tflops, 1)
        print(f"  fp16 matmul      {tflops:.1f} TFLOP/s")
        if rc and rc.dtype == "bfloat16" and not torch.cuda.is_bf16_supported():
            problems.append("GPU has no bfloat16 (pre-Ampere, e.g. T4/V100): set dtype: float16 in run.yaml.")

    work = Path(rc.io.work_dir if rc else ".").resolve()
    probe = work if work.exists() else next(p for p in [work, *work.parents] if p.exists())
    free_gb = shutil.disk_usage(probe).free / 1e9
    info["disk_free_gb"] = round(free_gb, 1)
    print(f"  disk free        {free_gb:.1f} GB at {probe}")
    if rc:
        try:
            from awq.hardware import analyze_model
            r = analyze_model(rc.model, rc.revision, rc.trust_remote_code, rc.calib.n_samples,
rc.calib.seq_len,
                              log=lambda *a: None)
            need = r["estimate"]["disk_gb"]
            info["disk_needed_gb_est"] = need
            print(f"  disk needed      ~{need} GB (estimate for {rc.model})")
            if free_gb < need:
```

```
                problems.append(f"Not enough disk: {free_gb:.0f} GB free, ~{need:.0f} GB needed. "
                                "Use a bigger volume and put HF_HOME + work_dir on it.")
        except Exception as e:
            warnings.append(f"could not size the model ({type(e).__name__}: {e})")

    try:
        from huggingface_hub import whoami
        user = whoami()["name"]
        info["hf_user"] = user
        print(f"  HF login         {user}")
    except Exception:
        warnings.append("Not logged in to Hugging Face (fine for public models). Gated models: `hf auth
login`.")

    for mod in ("transformers", "accelerate", "datasets", "safetensors", "huggingface_hub", "yaml"):
        try:
            __import__(mod)
        except ImportError:
            problems.append(f"missing package {mod}: pip install -e .")
    try:
        import lm_eval  # noqa: F401
        info["lm_eval"] = True
    except ImportError:
        warnings.append("lm-eval not installed (downstream tasks skipped): pip install -e '.[eval]'")

    for w in warnings:
        print("  WARN  ", w)
    for pr in problems:
        print("  FAIL  ", pr)
    out = (rc.work_dir if rc else Path(".")) / "env.json"
    info["problems"], info["warnings"] = problems, warnings
    write_json(out, info)
    print("ENV OK" if not problems else "ENV FAIL", f"(wrote {out})")
    sys.exit(1 if problems else 0)


if __name__ == "__main__":
    main()
```

scripts/01_inspect_model.py (44 lines)

```
#!/usr/bin/env python
"""
01_inspect_model.py — GATE 1. Before renting anything: is the model compatible, and what GPU,
how many, how long, how much? Downloads only config.json (+ custom code if trust_remote_code).

    python scripts/01_inspect_model.py --model Qwen/Qwen2.5-7B-Instruct
    python scripts/01_inspect_model.py --config configs/run.yaml
    python scripts/01_inspect_model.py --model deepseek-ai/DeepSeek-V3 --context 32768 --out report.json
"""
import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parents[1]))
from awq.cli import parser, run_config  # noqa: E402
from awq.hardware import analyze_model, format_report  # noqa: E402
from awq.utils import write_json  # noqa: E402


def main():
    p = parser(__doc__, need_config=False)
    p.add_argument("--model", help="HF id or local folder (instead of --config)")
    p.add_argument("--revision", default="main")
    p.add_argument("--trust-remote-code", action="store_true")
    p.add_argument("--context", type=int, default=8192, help="serving context length for the KV-cache
estimate")
    p.add_argument("--out", help="where to write the JSON report")
    args = p.parse_args()
    if args.config:
        rc = run_config(args)
        model, rev, trc, ns, sl = rc.model, rc.revision, rc.trust_remote_code, rc.calib.n_samples,
rc.calib.seq_len
        out = Path(args.out) if args.out else rc.work_dir / "model_report.json"
    elif args.model:
        model, rev, trc, ns, sl = args.model, args.revision, args.trust_remote_code, 128, 512
        out = Path(args.out) if args.out else Path("model_report.json")
    else:
```

```
        p.error("give --model or --config")
    r = analyze_model(model, rev, trc, ns, sl, args.context)
    print(format_report(r))
    write_json(out, r)
    print(f"\nwrote {out}")
    sys.exit(1 if r["verdict"] == "FAIL" else 0)


if __name__ == "__main__":
    main()
```

scripts/02_baseline.py (96 lines)

```
#!/usr/bin/env python
"""
02_baseline.py — the numbers to beat. Measures the ORIGINAL model (16-bit) and naive
round-to-nearest 4-bit (RTN, no AWQ) on the same eval data that 05_eval.py uses.

Writes <work_dir>/baseline.json:  fp16_ppl / rtn_ppl per dataset, fp16 weight bytes, generations,
optional lm-eval scores.

    python scripts/02_baseline.py --config configs/run.yaml
    python scripts/02_baseline.py --config configs/run.yaml --skip-fp16     # model too big for FP16 eval
    python scripts/02_baseline.py --config configs/run.yaml --skip-fp16 --set quant.group_size=64 --out
rtn_g64.json
"""
import gc
import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parents[1]))
import torch  # noqa: E402

from awq.calib import get_eval_tokens  # noqa: E402
from awq.cli import parser, run_config  # noqa: E402
from awq.evaluate import (apply_rtn_, generations, load_fp_model, model_bytes, perplexity,  # noqa: E402
                          run_lm_eval, speed)
from awq.model_io import resolve_model_path  # noqa: E402
from awq.utils import Logger, env_info, write_json  # noqa: E402


def main():
    p = parser(__doc__)
    p.add_argument("--skip-fp16", action="store_true")
    p.add_argument("--skip-rtn", action="store_true")
    p.add_argument("--no-gen", action="store_true", help="skip generations / speed")
    p.add_argument("--no-lm-eval", action="store_true")
    p.add_argument("--out", default="baseline.json", help="file name inside work_dir (e.g. rtn_g64.json)")
    args = p.parse_args()
    rc = run_config(args)
    log = Logger(rc.work_dir / "baseline.log")
    from transformers import AutoTokenizer

    path = resolve_model_path(rc.model, rc.revision, rc.trust_remote_code)
    tok = AutoTokenizer.from_pretrained(str(path), trust_remote_code=rc.trust_remote_code)
    data = {d: get_eval_tokens(tok, d, rc.eval.ppl_seq_len, rc.eval.max_windows) for d in
rc.eval.ppl_datasets}
    for d, t in data.items():
        log(f"[baseline] eval set {d}: {t.shape[0]} windows x {t.shape[1]} tokens")
    out = {"model": rc.model, "env": env_info(), "eval_windows": {d: int(t.shape[0]) for d, t in
data.items()},
           "ppl_seq_len": rc.eval.ppl_seq_len}
    do_lm = bool(rc.eval.lm_eval_tasks) and not args.no_lm_eval

    for variant in ("fp16", "rtn"):
        if (variant == "fp16" and args.skip_fp16) or (variant == "rtn" and args.skip_rtn):
            continue
        log(f"[baseline] loading {variant} model ...")
        if rc.device == "cuda":
            torch.cuda.reset_peak_memory_stats()
        model = load_fp_model(path, rc.torch_dtype, rc.device, rc.trust_remote_code)
        if rc.device == "cpu":
            model.to("cpu")
        if variant == "fp16":
            out["fp16_weight_bytes"] = model_bytes(model)
        if variant == "rtn":
            n = apply_rtn_(model, rc.quant.w_bit, rc.quant.group_size, rc.quant.zero_point,
```

```
rc.quant.skip_modules)
            log(f"[baseline] RTN applied to {n} weight tensors")
        for d, t in data.items():
            ppl = perplexity(model, t, log, desc=f"{variant} {d}")
            out[f"{variant}_ppl_{d}"] = ppl
            log(f"[baseline] {variant} {d} ppl = {ppl:.4f}")
        if rc.device == "cuda":
            out[f"{variant}_peak_vram_gb"] = round(torch.cuda.max_memory_allocated() / 1e9, 3)
        if variant == "fp16" and not args.no_gen:
            out["fp16_generations"] = generations(model, tok, max_new_tokens=rc.eval.gen_max_new_tokens)
            out["fp16_speed"] = speed(model, tok)
            log(f"[baseline] fp16 speed {out['fp16_speed']}")
        if variant == "fp16" and rc.eval.custom_jsonl:
            from awq.evaluate import read_custom_jsonl
            rows = read_custom_jsonl(rc.eval.custom_jsonl)
            gens = generations(model, tok, [r["prompt"] for r in rows], max_new_tokens=64)
            hits = sum(r["expected"].lower() in g.lower() for r, g in zip(rows, gens))
            out["fp16_custom_accuracy"] = hits / len(rows)
        if do_lm:
            try:
                out[f"{variant}_lm_eval"] = run_lm_eval(model, tok, rc.eval.lm_eval_tasks,
rc.eval.lm_eval_limit,
                                                        rc.eval.lm_eval_batch_size)
                log(f"[baseline] {variant} lm-eval {out[f'{variant}_lm_eval']}")
            except Exception as e:
                log(f"[baseline] lm-eval skipped: {type(e).__name__}: {e}")
                do_lm = False
        del model
        gc.collect()
        if rc.device == "cuda":
            torch.cuda.empty_cache()
        write_json(rc.work_dir / args.out, out)
    log(f"[baseline] wrote {rc.work_dir / args.out}")


if __name__ == "__main__":
    main()
```

scripts/03_quantize.py (29 lines)

```
#!/usr/bin/env python
"""
03_quantize.py — run AWQ, block by block. Output: <work_dir>/quantized/ (native format) and
<work_dir>/search_log.jsonl (per-block alphas, losses, block error).

    python scripts/03_quantize.py --config configs/run.yaml
    python scripts/03_quantize.py --config configs/run.yaml --resume      # after a crash / reclaimed pod
"""
import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parents[1]))
from awq.cli import parser, run_config  # noqa: E402
from awq.pipeline import quantize_model  # noqa: E402
from awq.utils import Logger  # noqa: E402


def main():
    p = parser(__doc__)
    p.add_argument("--resume", action="store_true")
    args = p.parse_args()
    rc = run_config(args)
    log = Logger(rc.work_dir / "quantize.log")
    log(f"[quantize] config: {rc.to_dict()}")
    quantize_model(rc, resume=args.resume, log=log)


if __name__ == "__main__":
    main()
```

scripts/04_verify.py (107 lines)

```
#!/usr/bin/env python
"""
04_verify.py — GATE 4. Is the quantized model sane before we spend time on full evaluation?
```

```
  * loads the quantized model ALONE first and measures its real footprint (weights actually allocated,
    and peak VRAM on GPU) — the honest version of the "VRAM drop" claim
  * size on disk vs. the formula estimate from 01_inspect_model
  * logits on 10 probe prompts vs. the original model: relative error + top-1 agreement
  * every packed layer decodes to finite numbers

    python scripts/04_verify.py --config configs/run.yaml
"""
import gc
import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parents[1]))
import torch  # noqa: E402

from awq.cli import parser, run_config  # noqa: E402
from awq.evaluate import DEFAULT_PROMPTS, input_device, load_fp_model, model_bytes  # noqa: E402
from awq.awq_linear import AwqLinear  # noqa: E402
from awq.hardware import native_size_gb  # noqa: E402
from awq.model_io import load_quantized, resolve_model_path  # noqa: E402
from awq.utils import Logger, dir_size_gb, read_json, write_json  # noqa: E402

GATE_TOP1 = 0.75          # >= 75% of next-token predictions identical to the original model
GATE_SIZE_TOL = 0.15      # on-disk size within 15% of the formula estimate


@torch.no_grad()
def logits_for(model, tok):
    dev = input_device(model)
    outs = []
    for pr in DEFAULT_PROMPTS:
        ids = tok(pr, return_tensors="pt").input_ids.to(dev)
        outs.append(model(input_ids=ids, use_cache=False).logits[0].float().cpu())
    return outs


def main():
    p = parser(__doc__)
    p.add_argument("--skip-fp16", action="store_true", help="no FP16 comparison (model too big)")
    args = p.parse_args()
    rc = run_config(args)
    log = Logger(rc.work_dir / "verify.log")
    from transformers import AutoTokenizer

    qdir = rc.work_dir / "quantized"
    path = resolve_model_path(rc.model, rc.revision, rc.trust_remote_code)
    tok = AutoTokenizer.from_pretrained(str(qdir))
    out, gates = {}, {}

    if rc.device == "cuda":
        torch.cuda.empty_cache()
        torch.cuda.reset_peak_memory_stats()
    qm = load_quantized(qdir, rc.device)
    out["awq_weight_bytes"] = model_bytes(qm)
    if rc.device == "cuda":
        out["awq_load_peak_vram_gb"] = round(torch.cuda.max_memory_allocated() / 1e9, 3)
    out["awq_disk_gb"] = dir_size_gb(qdir)
    bad = [n for n, m in qm.named_modules()
           if isinstance(m, AwqLinear) and not torch.isfinite(m.dequant_weight()).all()]
    gates["all_layers_finite"] = not bad
    report = rc.work_dir / "model_report.json"
    if report.exists():
        counts = read_json(report)["counts"]
        dtype_bytes = torch.tensor([], dtype=rc.torch_dtype).element_size()
        est = native_size_gb(counts, dtype_bytes, rc.quant.group_size)   # formula for THIS run's dtype
        out["awq_gb_formula"] = round(est, 4)
        out["awq_gb_formula_16bit"] = read_json(report)["estimate"]["awq_gb"]
        out["size_vs_formula"] = round(out["awq_disk_gb"] / est, 3)
        gates["size_matches_formula"] = abs(out["size_vs_formula"] - 1) <= GATE_SIZE_TOL
    log(f"[verify] AWQ weights {out['awq_weight_bytes'] / 1e9:.3f} GB in memory, {out['awq_disk_gb']} GB on
disk")
    q_logits = logits_for(qm, tok)
    del qm
    gc.collect()

    if not args.skip_fp16:
        fp = load_fp_model(path, rc.torch_dtype, rc.device, rc.trust_remote_code)
        if rc.device == "cpu":
```

```
            fp.to("cpu")
        out["fp16_weight_bytes"] = model_bytes(fp)
        f_logits = logits_for(fp, tok)
        del fp
        rel, agree, n = [], 0, 0
        for a, b in zip(f_logits, q_logits):
            rel.append(((a - b).norm() / a.norm()).item())
            agree += (a.argmax(-1) == b.argmax(-1)).sum().item()
            n += a.shape[0]
        out["logit_rel_err_mean"] = sum(rel) / len(rel)
        out["top1_agreement"] = agree / n
        out["memory_reduction_x"] = round(out["fp16_weight_bytes"] / out["awq_weight_bytes"], 3)
        gates["top1_agreement>=0.75"] = out["top1_agreement"] >= GATE_TOP1
        log(f"[verify] logit rel err {out['logit_rel_err_mean']:.4f} | top-1 agreement
{out['top1_agreement']:.3f} "
            f"| weights {out['memory_reduction_x']}x smaller in memory")
    out["gates"] = gates
    out["pass"] = all(gates.values())
    write_json(rc.work_dir / "verify.json", out)
    for k, v in gates.items():
        log(f"[verify] {'PASS' if v else 'FAIL'}  {k}")
    log("[verify] PASS" if out["pass"] else "[verify] FAIL")
    sys.exit(0 if out["pass"] else 1)


if __name__ == "__main__":
    main()
```

scripts/05_eval.py (136 lines)

```
#!/usr/bin/env python
"""
05_eval.py — the headline numbers for the quantized model + pass/fail gates vs. baseline.json.

  perplexity on eval.ppl_datasets (same windows as 02_baseline), generations side by side
  (generations.md), tokens/s, weight footprint, optional lm-eval tasks and your custom prompts.

    python scripts/05_eval.py --config configs/run.yaml
"""
import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parents[1]))
import torch  # noqa: E402

from awq.calib import get_eval_tokens  # noqa: E402
from awq.cli import parser, run_config  # noqa: E402
from awq.evaluate import (DEFAULT_PROMPTS, generations, model_bytes, perplexity, read_custom_jsonl,  # noqa:
E402
                          run_lm_eval, speed)
from awq.model_io import load_quantized  # noqa: E402
from awq.utils import Logger, env_info, read_json, write_json  # noqa: E402


def gate_limits(params_b: float):
    if params_b <= 3.5:
        return 0.10, 3.0, "<=3B"
    if params_b < 60:
        return 0.05, 1.5, "7-34B"
    return 0.03, 1.0, ">=70B"


def custom_accuracy(model, tok, path):
    rows = read_custom_jsonl(path)
    gens = generations(model, tok, [r["prompt"] for r in rows], max_new_tokens=64)
    hits = sum(1 for r, g in zip(rows, gens) if r["expected"].strip().lower() in g.lower())
    return hits / max(1, len(rows))


def main():
    p = parser(__doc__)
    p.add_argument("--no-lm-eval", action="store_true")
    p.add_argument("--no-gen", action="store_true")
    args = p.parse_args()
    rc = run_config(args)
    log = Logger(rc.work_dir / "eval.log")
    from transformers import AutoTokenizer
```

```
    qdir = rc.work_dir / "quantized"
    tok = AutoTokenizer.from_pretrained(str(qdir))
    if rc.device == "cuda":
        torch.cuda.reset_peak_memory_stats()
    model = load_quantized(qdir, rc.device)
    res = {"model": rc.model, "env": env_info(), "quant_config": {
        k: v for k, v in read_json(qdir / "quant_config.json").items() if k not in ("linears", "experts",
"skipped")}}
    res["awq_weight_bytes"] = model_bytes(model)
    for d in rc.eval.ppl_datasets:
        t = get_eval_tokens(tok, d, rc.eval.ppl_seq_len, rc.eval.max_windows)
        res[f"awq_ppl_{d}"] = perplexity(model, t, log, desc=f"awq {d}")
        res[f"eval_windows_{d}"] = int(t.shape[0])
        log(f"[eval] awq {d} ppl = {res[f'awq_ppl_{d}']:.4f}  ({t.shape[0]} windows)")
    if rc.device == "cuda":
        res["awq_peak_vram_gb"] = round(torch.cuda.max_memory_allocated() / 1e9, 3)
    if not args.no_gen:
        res["awq_generations"] = generations(model, tok, max_new_tokens=rc.eval.gen_max_new_tokens)
        res["awq_speed"] = speed(model, tok)
        log(f"[eval] awq speed {res['awq_speed']}  (reference dequant-per-call kernels; export for fast
serving)")
    if rc.eval.custom_jsonl:
        res["awq_custom_accuracy"] = custom_accuracy(model, tok, rc.eval.custom_jsonl)
    if rc.eval.lm_eval_tasks and not args.no_lm_eval:
        try:
            res["awq_lm_eval"] = run_lm_eval(model, tok, rc.eval.lm_eval_tasks, rc.eval.lm_eval_limit,
                                             rc.eval.lm_eval_batch_size)
            log(f"[eval] awq lm-eval {res['awq_lm_eval']}")
        except Exception as e:
            log(f"[eval] lm-eval skipped: {type(e).__name__}: {e}")

    # ---- compare with baseline + gates ------------------------------------------------------
    base_f = rc.work_dir / "baseline.json"
    gates = {}
    if base_f.exists():
        base = read_json(base_f)
        params_b = (base.get("fp16_weight_bytes") or 0) / 2 / 1e9
        mr = rc.work_dir / "model_report.json"
        if mr.exists():
            params_b = read_json(mr)["estimate"]["params_b"]
        ppl_gate, acc_gate, cls = gate_limits(params_b)
        res["size_class"] = cls
        for d in rc.eval.ppl_datasets:
            fp, rtn, awq = base.get(f"fp16_ppl_{d}"), base.get(f"rtn_ppl_{d}"), res[f"awq_ppl_{d}"]
            if fp:
                res[f"awq_vs_fp16_{d}"] = (awq - fp) / fp
                gates[f"{d}: AWQ ppl within {ppl_gate:.0%} of original"] = res[f"awq_vs_fp16_{d}"] <= ppl_gate
            if rtn:
                res[f"rtn_vs_fp16_{d}"] = (rtn - fp) / fp if fp else None
                gates[f"{d}: AWQ ppl < RTN ppl"] = awq < rtn
        if base.get("fp16_weight_bytes"):
            res["weight_memory_reduction_x"] = round(base["fp16_weight_bytes"] / res["awq_weight_bytes"], 3)
        if base.get("fp16_lm_eval") and res.get("awq_lm_eval"):
            drops = []
            for task, m in res["awq_lm_eval"].items():
                key = "acc_norm" if "acc_norm" in m else "acc"
                if task in base["fp16_lm_eval"] and key in m:
                    drops.append(100 * (base["fp16_lm_eval"][task][key] - m[key]))
            if drops:
                res["lm_eval_avg_drop_points"] = sum(drops) / len(drops)
                gates[f"lm-eval avg drop <= {acc_gate} points"] = res["lm_eval_avg_drop_points"] <= acc_gate
        if not args.no_gen:
            gates["no empty generations"] = all(g.strip() for g in res["awq_generations"])
            fp_g = base.get("fp16_generations") or [""] * len(DEFAULT_PROMPTS)
            orig = f"Original ({rc.dtype})"
            md = [f"# Generations: {orig} vs AWQ (greedy, same prompts)\n",
                  f"Model: `{rc.model}` | AWQ: {rc.quant.w_bit}-bit g{rc.quant.group_size} | "
                  f"max_new_tokens={rc.eval.gen_max_new_tokens}\n"]
            for pr, a, b in zip(DEFAULT_PROMPTS, fp_g, res["awq_generations"]):
                md += [f"## Prompt\n```\n{pr}\n```", f"**{orig}:**\n```\n{a.strip()}\n```",
                       f"**AWQ:**\n```\n{b.strip()}\n```\n"]
            (rc.work_dir / "generations.md").write_text("\n".join(md))
    else:
        log("[eval] no baseline.json -> run 02_baseline.py for comparisons and gates")
    res["gates"] = gates
    res["pass"] = bool(gates) and all(gates.values())
    write_json(rc.work_dir / "results.json", res)
    log("[eval] ---------------- RESULTS ----------------")
```

```
    for d in rc.eval.ppl_datasets:
        b = read_json(base_f) if base_f.exists() else {}
        fp, rtn = b.get(f"fp16_ppl_{d}", float("nan")), b.get(f"rtn_ppl_{d}", float("nan"))
        log(f"[eval] {d:<10} original {fp:.4f} | RTN {rtn:.4f}"
            f" | AWQ {res[f'awq_ppl_{d}']:.4f}")
    for k, v in gates.items():
        log(f"[eval] {'PASS' if v else 'FAIL'}  {k}")
    log("[eval] OVERALL " + ("PASS" if res["pass"] else "FAIL"))
    sys.exit(0 if res["pass"] else 1)


if __name__ == "__main__":
    main()
```

scripts/06_tune.py (130 lines)

```
#!/usr/bin/env python
"""
06_tune.py — stage-wise tuning of the quantization knobs ("hyperparameters").

Rules (strict):
  * the tuning metric is perplexity on eval.tune_dataset (default: WikiText-2 VALIDATION split),
    which is never used by 05_eval.py — the final test numbers are not tuned on;
  * one knob at a time (configs/tune_space.yaml), the winner is carried into the next stage;
  * a change must improve the metric by >= min_improvement (default 0.5%) to be accepted;
  * stops at budget_trials or --budget-hours.

Outputs: <work_dir>/tuning/leaderboard.csv and configs/run.best.yaml (use it for the final run).

    python scripts/06_tune.py --config configs/run.yaml --space configs/tune_space.yaml --budget-hours 3
"""
import csv
import gc
import shutil
import sys
import time
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parents[1]))
import yaml  # noqa: E402

from awq.calib import get_eval_tokens  # noqa: E402
from awq.cli import parser  # noqa: E402
from awq.config import load_config  # noqa: E402
from awq.evaluate import perplexity  # noqa: E402
from awq.model_io import load_quantized  # noqa: E402
from awq.pipeline import quantize_model  # noqa: E402
from awq.utils import Logger  # noqa: E402


def get_value(rc, dotted):
    node = rc
    for p in dotted.split("."):
        node = getattr(node, p)
    return node


def main():
    p = parser(__doc__)
    p.add_argument("--space", default="configs/tune_space.yaml")
    p.add_argument("--budget-hours", type=float, default=None)
    p.add_argument("--keep-trials", action="store_true", help="keep every trial's quantized weights")
    p.add_argument("--out-config", default="configs/run.best.yaml")
    args = p.parse_args()
    base_rc = load_config(args.config, overrides=args.set)
    space = yaml.safe_load(Path(args.space).read_text())
    budget = space.get("budget_trials", 8)
    min_imp = space.get("min_improvement", 0.005)
    tdir = base_rc.work_dir / "tuning"
    tdir.mkdir(parents=True, exist_ok=True)
    log = Logger(tdir / "tune.log")
    from transformers import AutoTokenizer

    from awq.model_io import resolve_model_path
    path = resolve_model_path(base_rc.model, base_rc.revision, base_rc.trust_remote_code)
    tok = AutoTokenizer.from_pretrained(str(path), trust_remote_code=base_rc.trust_remote_code)
    tune_tokens = get_eval_tokens(tok, base_rc.eval.tune_dataset, base_rc.eval.ppl_seq_len,
```

```
                                  base_rc.eval.tune_max_windows)
    log(f"[tune] metric: ppl on {base_rc.eval.tune_dataset} ({tune_tokens.shape[0]} windows) — NOT the test
set")
    t_start, rows, cache = time.time(), [], {}
    best_over: dict = {}

    def trial(over: dict):
        key = tuple(sorted((k, str(v)) for k, v in over.items()))
        if key in cache:
            return cache[key]
        k = len(rows)
        wd = tdir / f"trial_{k:02d}"
        rc = load_config(args.config, overrides=list(args.set) + [f"{kk}={vv}" for kk, vv in over.items()]
                         + [f"io.work_dir={wd}"])
        t = time.time()
        quantize_model(rc, log=Logger(wd / "quantize.log"))
        model = load_quantized(wd / "quantized", rc.device)
        ppl = perplexity(model, tune_tokens)
        del model
        gc.collect()
        row = {"trial": k, **{kk: str(vv) for kk, vv in over.items()}, "tune_ppl": round(ppl, 5),
               "minutes": round((time.time() - t) / 60, 2)}
        rows.append(row)
        cache[key] = ppl
        log(f"[tune] trial {k}: {over or '(base config)'} -> ppl {ppl:.5f}  ({row['minutes']} min)")
        if not args.keep_trials:
            shutil.rmtree(wd / "quantized", ignore_errors=True)
            shutil.rmtree(wd / "state", ignore_errors=True)
        return ppl

    best = trial({})
    stalls = 0
    for stage in space["stages"]:
        cur = best_over.get(stage["key"], get_value(base_rc, stage["key"]))
        log(f"[tune] stage '{stage['name']}': {stage['key']} in {stage['values']} (current {cur})")
        stage_best, stage_val = best, cur
        for v in stage["values"]:
            if str(v) == str(cur):
                continue
            if len(rows) >= budget or (args.budget_hours and (time.time() - t_start) / 3600 >
args.budget_hours):
                log("[tune] budget reached")
                break
            ppl = trial({**best_over, stage["key"]: v})
            if ppl < stage_best * (1 - min_imp):
                stage_best, stage_val = ppl, v
        if stage_val != cur:
            best_over[stage["key"]] = stage_val
            log(f"[tune] stage winner: {stage['key']}={stage_val}  ppl {best:.5f} -> {stage_best:.5f}")
            best, stalls = stage_best, 0
        else:
            stalls += 1
            log(f"[tune] stage kept {stage['key']}={cur}")
        if stalls >= space.get("patience_stages", 3) or len(rows) >= budget:
            break

    fields = sorted({k for r in rows for k in r}, key=lambda x: (x not in ("trial",), x))
    with open(tdir / "leaderboard.csv", "w", newline="") as fh:
        w = csv.DictWriter(fh, fieldnames=fields)
        w.writeheader()
        w.writerows(sorted(rows, key=lambda r: r["tune_ppl"]))
    final = load_config(args.config, overrides=list(args.set) + [f"{kk}={vv}" for kk, vv in
best_over.items()])
    final.save(args.out_config)
    log(f"[tune] best overrides {best_over} -> {args.out_config}  (tune ppl {best:.5f})")
    log(f"[tune] leaderboard -> {tdir / 'leaderboard.csv'}")
    log("[tune] NEXT: run 03_quantize.py + 05_eval.py with the best config for the final, untuned-on test
numbers.")



if __name__ == "__main__":
    main()
```

scripts/07_export.py (77 lines)

```
#!/usr/bin/env python
"""
07_export.py — package the result.

  native  : our format (any architecture; load with awq.load_quantized)       -> <work_dir>/export/native
  autoawq : standard AWQ GEMM format for vLLM / SGLang (needs folded scales)  -> <work_dir>/export/autoawq

    python scripts/07_export.py --config configs/run.yaml --format native,autoawq
"""
import shutil
import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parents[1]))
from safetensors import safe_open  # noqa: E402

from awq.cli import parser, run_config  # noqa: E402
from awq.export import export_autoawq, gemm_dequant  # noqa: E402
from awq.quantizer import dequantize, unpack_int4  # noqa: E402
from awq.utils import Logger, read_json  # noqa: E402


def check_autoawq(qdir: Path, out: Path, log, n: int = 4) -> float:
    """Decode a few layers from both formats; they must give identical weights."""
    qc = read_json(qdir / "quant_config.json")
    names = list(qc["linears"])[:n]
    worst = 0.0
    nat, gem = {}, {}
    for f in qdir.glob("*.safetensors"):
        with safe_open(str(f), framework="pt") as h:
            for k in h.keys():
                nat[k] = f
    for f in out.glob("*.safetensors"):
        with safe_open(str(f), framework="pt") as h:
            for k in h.keys():
                gem[k] = f
    for name in names:
        with safe_open(str(nat[name + ".qweight"]), framework="pt") as h:
            w_nat = dequantize(unpack_int4(h.get_tensor(name + ".qweight")), h.get_tensor(name +
".scales").float(),
                               h.get_tensor(name + ".qzeros"), qc["group_size"],
dtype=__import__("torch").float32)
        with safe_open(str(gem[name + ".qweight"]), framework="pt") as h:
            w_gem = gemm_dequant(h.get_tensor(name + ".qweight"), h.get_tensor(name + ".qzeros"),
                                 h.get_tensor(name + ".scales"), qc["group_size"])
        d = (w_nat - w_gem).abs().max().item() / (w_nat.abs().max().item() + 1e-12)
        worst = max(worst, d)
        log(f"[export] check {name}: max rel diff native vs GEMM = {d:.2e}")
    return worst


def main():
    p = parser(__doc__)
    p.add_argument("--format", default="native,autoawq")
    args = p.parse_args()
    rc = run_config(args)
    log = Logger(rc.work_dir / "export.log")
    qdir = rc.work_dir / "quantized"
    fmts = [f.strip() for f in args.format.split(",") if f.strip()]
    ok = True
    if "native" in fmts:
        dst = rc.work_dir / "export" / "native"
        shutil.copytree(qdir, dst, dirs_exist_ok=True)
        log(f"[export] native -> {dst}")
    if "autoawq" in fmts:
        dst = rc.work_dir / "export" / "autoawq"
        try:
            export_autoawq(qdir, dst, log=log)
            worst = check_autoawq(qdir, dst, log)
            ok &= worst < 1e-2           # fp16 rounding of scales only
            log(f"[export] GEMM layout check {'PASS' if worst < 1e-2 else 'FAIL'} (worst rel diff
{worst:.2e})")
        except RuntimeError as e:
            log(f"[export] autoawq NOT produced: {e}")
            ok = False
    sys.exit(0 if ok else 1)
```

```
if __name__ == "__main__":
    main()
```

scripts/08_publish.py (110 lines)

```
#!/usr/bin/env python
"""
08_publish.py — write a model card with ONLY measured numbers and upload to the Hugging Face Hub.

    python scripts/08_publish.py --config configs/run.yaml --repo you/Model-AWQ            # private (default)
    python scripts/08_publish.py --config configs/run.yaml --repo you/Model-AWQ --public
    python scripts/08_publish.py --config configs/run.yaml --repo you/Model-AWQ --dry-run  # card only

Needs a WRITE token: `hf auth login`. The base model's LICENSE file travels with the weights
(it was copied into the quantized folder) — a quantized model is a derivative work.
"""
import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parents[1]))
from awq.cli import parser, run_config  # noqa: E402
from awq.utils import Logger, read_json  # noqa: E402


def fmt(x, nd=4):
    return "n/a" if x is None else (f"{x:.{nd}f}" if isinstance(x, float) else str(x))


def model_card(rc, folder: Path) -> str:
    wd = rc.work_dir
    res = read_json(wd / "results.json") if (wd / "results.json").exists() else {}
    base = read_json(wd / "baseline.json") if (wd / "baseline.json").exists() else {}
    ver = read_json(wd / "verify.json") if (wd / "verify.json").exists() else {}
    is_gemm = (folder / "config.json").exists() and "quantization_config" in (folder /
"config.json").read_text()
    rows = []
    for d in rc.eval.ppl_datasets:
        rows.append(f"| {d} perplexity (lower is better) | {fmt(base.get(f'fp16_ppl_{d}'))} | "
                    f"{fmt(base.get(f'rtn_ppl_{d}'))} | {fmt(res.get(f'awq_ppl_{d}'))} |")
    gates = "\n".join(f"- {'PASS' if v else 'FAIL'}: {k}" for k, v in res.get("gates", {}).items()) or "- not
run"
    usage = ("""```bash
vllm serve <this-repo> --quantization awq_marlin --dtype float16
```""" if is_gemm else """```python
pip install git+https://github.com/kashyapphassan2000-netizen/WEEK--37
from awq import load_quantized
model = load_quantized("<local snapshot of this repo>", device="cuda")
```""")
    zp = ", zero-point" if rc.quant.zero_point else ""
    mem_fp = fmt((base.get("fp16_weight_bytes") or 0) / 1e9, 3)
    mem_q = fmt((res.get("awq_weight_bytes") or 0) / 1e9, 3)
    return f"""---
base_model: {rc.model}
tags: [awq, quantization, int4, awq-from-scratch]
---

# {rc.model.split('/')[-1]} — AWQ {rc.quant.w_bit}-bit (group {rc.quant.group_size}{zp})

4-bit AWQ (Lin et al. 2023, arXiv:2306.00978) quantization of [`{rc.model}`]
(https://huggingface.co/{rc.model}),
produced with the from-scratch, block-streaming implementation at
https://github.com/kashyapphassan2000-netizen/WEEK--37 . Every number below was measured, not copied.

## Results (measured)

| metric | original 16-bit | naive RTN 4-bit | **AWQ 4-bit (this)** |
|---|---|---|---|
{chr(10).join(rows)}
| weight memory (GB) | {mem_fp} | — | {mem_q} |

Next-token agreement with the original on probe prompts: {fmt(ver.get('top1_agreement'), 3)}.
Eval: {rc.eval.ppl_seq_len}-token windows; windows per dataset: {base.get('eval_windows', 'n/a')}.

Gates:
{gates}
```

```
## Method
- Calibration: {rc.calib.dataset}, {rc.calib.n_samples} x {rc.calib.seq_len} tokens.
- Per-group activation-aware scale search ({rc.search.n_grid} grid points, duo scaling=
{rc.search.duo_scaling}),
  weight-clip search={rc.search.clip}, scales folded into preceding norms/linears where possible.
- Embeddings and output head kept in 16-bit (standard AWQ practice).

## Use
{usage}

## License
Same license and use policy as the base model `{rc.model}` (see LICENSE / USE_POLICY files in this repo).
"""


def main():
    p = parser(__doc__)
    p.add_argument("--repo", required=True)
    p.add_argument("--folder", help="folder to upload (default: export/autoawq, else export/native)")
    p.add_argument("--public", action="store_true")
    p.add_argument("--dry-run", action="store_true")
    args = p.parse_args()
    rc = run_config(args)
    log = Logger(rc.work_dir / "publish.log")
    folder = Path(args.folder) if args.folder else next(
        (f for f in (rc.work_dir / "export" / "autoawq", rc.work_dir / "export" / "native", rc.work_dir /
"quantized")
         if f.exists()), None)
    if folder is None:
        sys.exit("nothing to publish: run 03_quantize.py / 07_export.py first")
    (folder / "README.md").write_text(model_card(rc, folder))
    log(f"[publish] model card written: {folder / 'README.md'}")
    if args.dry_run:
        log("[publish] dry run — nothing uploaded")
        return
    from huggingface_hub import HfApi
    api = HfApi()
    api.create_repo(args.repo, private=not args.public, exist_ok=True)
    api.upload_folder(folder_path=str(folder), repo_id=args.repo, commit_message="AWQ 4-bit (awq-from-
scratch)")
    log(f"[publish] uploaded -> https://huggingface.co/{args.repo}  ({'public' if args.public else
'private'})")


if __name__ == "__main__":
    main()
```

scripts/run_all.sh (15 lines)

```
#!/usr/bin/env bash
# run_all.sh — gates 0 -> 5 for one config. Stops at the first failure.
#   bash scripts/run_all.sh configs/run.yaml 2>&1 | tee run.log
set -euo pipefail
CFG="${1:?usage: bash scripts/run_all.sh configs/run.yaml [extra --set args]}"
shift || true
cd "$(dirname "$0")/.."
step() { echo; echo "================ $* ================"; }
step "00 env check";      python scripts/00_env_check.py   --config "$CFG" "$@"
step "01 inspect model";  python scripts/01_inspect_model.py --config "$CFG" "$@"
step "02 baseline";       python scripts/02_baseline.py    --config "$CFG" "$@"
step "03 quantize";       python scripts/03_quantize.py    --config "$CFG" "$@"
step "04 verify";         python scripts/04_verify.py      --config "$CFG" "$@"
step "05 eval";           python scripts/05_eval.py        --config "$CFG" "$@"
echo; echo "ALL GATES PASSED. Next: 06_tune.py (optional), 07_export.py, 08_publish.py"
```

configs/run.yaml (13 lines)

```
# THE FILE YOU EDIT. Copy any configs/examples/*.yaml here, then change model + work_dir.
# 1x RTX 4090 / L40S. Ungated — no license click needed.
model: Qwen/Qwen2.5-7B-Instruct
dtype: bfloat16
device: cuda
quant: {w_bit: 4, group_size: 128, zero_point: true, fold_scales: auto}
search: {n_grid: 20, clip: true}
```

```
calib: {dataset: pileval, n_samples: 128, seq_len: 512, batch_size: 8}
eval:
  ppl_datasets: [wikitext2, c4]
  ppl_seq_len: 2048
  lm_eval_tasks: [arc_easy, arc_challenge, hellaswag, piqa, winogrande]
io: {work_dir: /workspace/runs/qwen2.5-7b-awq}
```

configs/gpus.yaml (14 lines)

```
# GPU catalog used by scripts/01_inspect_model.py to pick GPU type + count + cost.
# Prices: RunPod COMMUNITY cloud, on-demand USD per GPU-hour, read from runpod.io/gpu-cloud/pricing
# (page dated 2026-10-08). Secure cloud is ~1.3-1.6x. Other providers: see docs/ playbook Section 6.
# PRICES CHANGE MONTHLY — edit this file before trusting a cost estimate.
checked: "2026-10-08"
source: "https://www.runpod.io/gpu-cloud/pricing"
gpus:
  - {name: "RTX 4090",  vram_gb: 24,  usd_per_hr: 0.34, speed: 0.67}
  - {name: "L40S",      vram_gb: 48,  usd_per_hr: 0.79, speed: 0.80}
  - {name: "A100 80GB", vram_gb: 80,  usd_per_hr: 1.19, speed: 1.00}
  - {name: "H100 80GB", vram_gb: 80,  usd_per_hr: 1.99, speed: 1.60}
  - {name: "H200",      vram_gb: 141, usd_per_hr: 3.59, speed: 1.80}
  - {name: "B200",      vram_gb: 180, usd_per_hr: 5.98, speed: 2.50}
# speed = rough throughput relative to A100 for this pure-PyTorch pipeline (ESTIMATE).
```

configs/tune_space.yaml (10 lines)

```
# Search space for scripts/06_tune.py — one stage at a time, winner carried forward.
budget_trials: 8          # max quantize+eval runs
min_improvement: 0.005    # a change must lower tune-ppl by >= 0.5% to be accepted
patience_stages: 3        # stop after 3 stages in a row without an accepted change
stages:
  - {name: calibration data,  key: calib.dataset,     values: [pileval, c4, wikitext2]}
  - {name: group size,        key: quant.group_size,  values: [128, 64]}
  - {name: propagation,       key: search.propagate,  values: [fp, quantized]}
  - {name: calibration size,  key: calib.n_samples,   values: [128, 256]}
  - {name: scale placement,   key: quant.fold_scales, values: [auto, never]}
```

configs/tune_space_cpu.yaml (8 lines)

```
# Small search space used for the CPU proof (each trial = one full quantization of Qwen2.5-0.5B on CPU).
budget_trials: 5
min_improvement: 0.005
patience_stages: 3
stages:
  - {name: scale placement,  key: quant.fold_scales, values: [auto, never]}
  - {name: group size,       key: quant.group_size,  values: [128, 64]}
  - {name: propagation,      key: search.propagate,  values: [fp, quantized]}
```

configs/examples/deepseek-v3.yaml (15 lines)

```
# 671B MoE, FP8 checkpoint (~700 GB download). Quantize on 1x H200/B200 with a 2-3 TB volume and
# >= 256 GB CPU RAM. FP16 baseline is NOT practical (1.3 TB of VRAM): run 02_baseline.py --skip-fp16.
# HONEST STATUS: this config is UNTESTED at full scale (no one has paid for that run yet). Run
# 01_inspect_model.py first; it tells you whether the checkpoint layout streams or needs a full load.
model: deepseek-ai/DeepSeek-V3
dtype: bfloat16
device: cuda
quant: {w_bit: 4, group_size: 128, zero_point: true, fold_scales: auto, quantize_experts: true}
search: {n_grid: 20, clip: true}
calib: {dataset: pileval, n_samples: 128, seq_len: 512, batch_size: 4}
eval:
  ppl_datasets: [wikitext2]
  ppl_seq_len: 2048
  lm_eval_tasks: [arc_easy]
io: {work_dir: /workspace/runs/deepseek-v3-awq}
```

configs/examples/llama-3.1-8b.yaml (12 lines)

```
# 1x RTX 4090 (quantize) + 1x L40S or 4090 (FP16 eval, tight). GATED: accept the license on HF first.
model: meta-llama/Llama-3.1-8B-Instruct
```

```
dtype: bfloat16
device: cuda
quant: {w_bit: 4, group_size: 128, zero_point: true, fold_scales: auto}
search: {n_grid: 20, clip: true}
calib: {dataset: pileval, n_samples: 128, seq_len: 512, batch_size: 8}
eval:
  ppl_datasets: [wikitext2, c4]
  ppl_seq_len: 2048
  lm_eval_tasks: [arc_easy, arc_challenge, hellaswag, piqa, winogrande]
io: {work_dir: /workspace/runs/llama-3.1-8b-awq}
```

configs/examples/llama-3.3-70b.yaml (14 lines)

```
# Quantize: 1x A100/H100 80GB, >= 64 GB CPU RAM, ~250 GB volume.
# FP16 baseline: needs ~170 GB VRAM (2x H200 or 3x A100/H100) — run 02_baseline.py on a bigger pod,
# or use --skip-fp16 and compare against published scores. GATED model.
model: meta-llama/Llama-3.3-70B-Instruct
dtype: bfloat16
device: cuda
quant: {w_bit: 4, group_size: 128, zero_point: true, fold_scales: auto}
search: {n_grid: 20, clip: true}
calib: {dataset: pileval, n_samples: 128, seq_len: 512, batch_size: 4}
eval:
  ppl_datasets: [wikitext2, c4]
  ppl_seq_len: 2048
  lm_eval_tasks: [arc_easy, arc_challenge, hellaswag, piqa, winogrande]
io: {work_dir: /workspace/runs/llama-3.3-70b-awq}
```

configs/examples/qwen2.5-0.5b-cpu.best.yaml (45 lines)

```
model: Qwen/Qwen2.5-0.5B
revision: main
trust_remote_code: false
dtype: float32
device: cpu
plugin: null
seed: 42
quant:
  w_bit: 4
  group_size: 64
  zero_point: true
  skip_modules: []
  fold_scales: auto
  quantize_experts: true
search:
  n_grid: 20
  duo_scaling: true
  max_search_tokens: 8192
  clip: true
  clip_grid: 20
  max_shrink: 0.5
  clip_tokens: 512
  propagate: fp
calib:
  dataset: pileval
  n_samples: 64
  seq_len: 512
  batch_size: 8
eval:
  ppl_datasets:
  - wikitext2
  - c4
  ppl_seq_len: 2048
  max_windows: 20
  lm_eval_tasks:
  - arc_easy
  - piqa
  lm_eval_limit: 300
  lm_eval_batch_size: '8'
  gen_max_new_tokens: 48
  custom_jsonl: null
  tune_dataset: wikitext2_val
  tune_max_windows: 20
io:
  work_dir: runs/qwen2.5-0.5b-cpu
```

configs/examples/qwen2.5-0.5b-cpu.yaml (30 lines)

```
# PROOF RUN — CPU only, no GPU needed. This is the config used for proof/cpu-qwen2.5-0.5b/.
# Small calibration/eval budgets so it finishes on a 4-core laptop; numbers are real but
# a 0.5B model loses more to 4-bit than a 7B+ model does.
model: Qwen/Qwen2.5-0.5B
dtype: float32
device: cpu
quant:
  w_bit: 4
  group_size: 128
  zero_point: true
  fold_scales: auto
search:
  n_grid: 20
  max_search_tokens: 8192     # subsample tokens in the scale search (CPU speed)
  clip: true
calib:
  dataset: pileval
  n_samples: 64
  seq_len: 512
  batch_size: 8
eval:
  ppl_datasets: [wikitext2, c4]
  ppl_seq_len: 2048
  max_windows: 20             # first 20 x 2048 = 41k tokens of each eval set (CPU budget)
  lm_eval_tasks: [arc_easy, piqa]
  lm_eval_limit: 300
  lm_eval_batch_size: "8"
  tune_max_windows: 20
io:
  work_dir: runs/qwen2.5-0.5b-cpu
```

configs/examples/qwen2.5-7b.yaml (12 lines)

```
# 1x RTX 4090 / L40S. Ungated — no license click needed.
model: Qwen/Qwen2.5-7B-Instruct
dtype: bfloat16
device: cuda
quant: {w_bit: 4, group_size: 128, zero_point: true, fold_scales: auto}
search: {n_grid: 20, clip: true}
calib: {dataset: pileval, n_samples: 128, seq_len: 512, batch_size: 8}
eval:
  ppl_datasets: [wikitext2, c4]
  ppl_seq_len: 2048
  lm_eval_tasks: [arc_easy, arc_challenge, hellaswag, piqa, winogrande]
io: {work_dir: /workspace/runs/qwen2.5-7b-awq}
```

configs/examples/qwen3-30b-a3b-moe.yaml (14 lines)

```
# MoE example (128 experts, top-8). Experts are fused 3-D tensors in transformers >= 5 and are
# quantized per expert natively (AwqExpertsWeight). The vLLM (GEMM) export does NOT cover fused
# experts -> ship the native format, or set quant.quantize_experts=false for attention-only AWQ.
model: Qwen/Qwen3-30B-A3B
dtype: bfloat16
device: cuda
quant: {w_bit: 4, group_size: 128, zero_point: true, fold_scales: auto, quantize_experts: true}
search: {n_grid: 20, clip: true}
calib: {dataset: pileval, n_samples: 128, seq_len: 512, batch_size: 8}
eval:
  ppl_datasets: [wikitext2]
  ppl_seq_len: 2048
  lm_eval_tasks: [arc_easy, hellaswag]
io: {work_dir: /workspace/runs/qwen3-30b-a3b-awq}
```

tests/conftest.py (74 lines)

```
"""Tiny random models built on the fly — the test-suite needs no downloads and no GPU."""
import pytest
import torch


def _save(model, path):
```

```
    model.save_pretrained(str(path), safe_serialization=True)
    return path


def add_outliers(model, channels=(3, 17, 40), factor=25.0):
    """Real LLMs have a few input channels with huge activations; recreate that so AWQ has work to do."""
    for name, p in model.named_parameters():
        if name.endswith("layernorm.weight"):
            with torch.no_grad():
                p[list(channels)] *= factor


@pytest.fixture(scope="session")
def tiny_llama(tmp_path_factory):
    from transformers import LlamaConfig, LlamaForCausalLM

    torch.manual_seed(0)
    cfg = LlamaConfig(vocab_size=512, hidden_size=128, intermediate_size=256, num_hidden_layers=3,
                      num_attention_heads=4, num_key_value_heads=4, max_position_embeddings=256,
                      tie_word_embeddings=False)
    m = LlamaForCausalLM(cfg).eval()
    add_outliers(m)
    return _save(m, tmp_path_factory.mktemp("tiny_llama"))


@pytest.fixture(scope="session")
def tiny_qwen_gqa(tmp_path_factory):
    """GQA + tied embeddings + qkv bias: the Qwen2 shape of things."""
    from transformers import Qwen2Config, Qwen2ForCausalLM

    torch.manual_seed(1)
    cfg = Qwen2Config(vocab_size=512, hidden_size=128, intermediate_size=256, num_hidden_layers=2,
                      num_attention_heads=4, num_key_value_heads=2, max_position_embeddings=256,
                      tie_word_embeddings=True)
    m = Qwen2ForCausalLM(cfg).eval()
    add_outliers(m)
    return _save(m, tmp_path_factory.mktemp("tiny_qwen"))


@pytest.fixture(scope="session")
def tiny_mixtral(tmp_path_factory):
    from transformers import MixtralConfig, MixtralForCausalLM

    torch.manual_seed(2)
    cfg = MixtralConfig(vocab_size=512, hidden_size=128, intermediate_size=256, num_hidden_layers=2,
                        num_attention_heads=4, num_key_value_heads=2, num_local_experts=4,
                        num_experts_per_tok=2, max_position_embeddings=256, tie_word_embeddings=False)
    m = MixtralForCausalLM(cfg).eval()
    add_outliers(m)
    return _save(m, tmp_path_factory.mktemp("tiny_mixtral"))


@pytest.fixture
def make_rc(tmp_path):
    from awq.config import load_config

    def _make(model_path, *extra):
        return load_config(None, overrides=[
            f"model={model_path}", "device=cpu", "dtype=float32", f"io.work_dir={tmp_path / 'run'}",
            "calib.n_samples=16", "calib.seq_len=64", "calib.batch_size=4", "quant.group_size=64",
            "search.n_grid=10", *extra])
    return _make


@pytest.fixture(scope="session")
def calib_tokens():
    g = torch.Generator().manual_seed(123)
    return torch.randint(0, 512, (16, 64), generator=g)
```

tests/test_config.py (20 lines)

```
import pytest

from awq.config import load_config


def test_overrides_are_typed():
```

```
    rc = load_config(None, overrides=["model=x", "device=cpu", "dtype=float32", "quant.group_size=64",
                                      "quant.zero_point=false", "eval.ppl_datasets=[wikitext2]"])
    assert rc.quant.group_size == 64 and rc.quant.zero_point is False and rc.eval.ppl_datasets ==
["wikitext2"]


@pytest.mark.parametrize("bad", ["quant.w_bit=3", "quant.group_size=100", "search.propagate=maybe",
"dtype=float16"])
def test_bad_values_fail_fast(bad):
    with pytest.raises(ValueError):
        load_config(None, overrides=["model=x", "device=cpu", "dtype=float32", bad])


def test_unknown_key_rejected():
    with pytest.raises(ValueError, match="Unknown"):
        load_config(None, overrides=["model=x", "device=cpu", "dtype=float32", "quant.groupsize=64"])
```

tests/test_export.py (51 lines)

```
import json

import pytest
import torch

from awq.export import export_autoawq, gemm_dequant, gemm_pack_codes, gemm_unpack_codes, to_gemm
from awq.quantizer import compute_qparams, dequantize, pack_int4, quantize
from awq.pipeline import quantize_model


def test_gemm_pack_roundtrip():
    codes = torch.randint(0, 16, (64, 96))
    assert torch.equal(gemm_unpack_codes(gemm_pack_codes(codes)), codes)


def test_gemm_known_layout():
    # one int32 holds outputs [0..7] in the AWQ order [0,2,4,6,1,3,5,7] at nibbles 0..7
    codes = torch.arange(8).view(1, 8)
    packed = gemm_pack_codes(codes).item() & 0xFFFFFFFF
    nibbles = [(packed >> (4 * i)) & 0xF for i in range(8)]
    assert nibbles == [0, 2, 4, 6, 1, 3, 5, 7]


def test_to_gemm_dequant_matches_native():
    torch.manual_seed(0)
    w = torch.randn(48, 256)
    s, z = compute_qparams(w, 4, 128, True)
    q = quantize(w, s, z, 4, 128)
    native = dequantize(q, s, z, 128, torch.float32)
    qw, qz, sc = to_gemm(pack_int4(q), z.to(torch.uint8), s.half())
    assert qw.dtype == torch.int32 and qw.shape == (256, 6) and qz.shape == (2, 6) and sc.shape == (2, 48)
    assert torch.allclose(gemm_dequant(qw, qz, sc, 128), native, rtol=2e-3, atol=2e-3)


def test_export_folder(tiny_llama, make_rc, calib_tokens, tmp_path):
    qdir = quantize_model(make_rc(tiny_llama), calib_tokens=calib_tokens)
    out = export_autoawq(qdir, tmp_path / "gemm")
    cfg = json.loads((out / "config.json").read_text())
    assert cfg["quantization_config"]["quant_method"] == "awq"
    assert cfg["quantization_config"]["version"] == "gemm"
    from safetensors import safe_open
    with safe_open(str(out / "model.safetensors"), framework="pt") as h:
        keys = set(h.keys())
    assert "model.layers.0.self_attn.q_proj.qweight" in keys
    assert "model.layers.0.self_attn.q_proj.weight" not in keys


def test_export_refuses_unfolded(tiny_llama, make_rc, calib_tokens, tmp_path):
    qdir = quantize_model(make_rc(tiny_llama, "quant.fold_scales=never"), calib_tokens=calib_tokens)
    with pytest.raises(RuntimeError, match="1/s"):
        export_autoawq(qdir, tmp_path / "gemm")
```

tests/test_hardware.py (23 lines)

```
"""The sizing formulas, checked against the hand calculation in the playbook (Llama-3.1-8B)."""
import torch
```

```
from awq.hardware import config_dims, count_params, estimate


def test_llama_8b_numbers_match_playbook():
    from accelerate import init_empty_weights
    from transformers import AutoModelForCausalLM, LlamaConfig

    cfg = LlamaConfig(vocab_size=128256, hidden_size=4096, intermediate_size=14336, num_hidden_layers=32,
                      num_attention_heads=32, num_key_value_heads=8, tie_word_embeddings=False)
    with init_empty_weights():
        m = AutoModelForCausalLM.from_config(cfg, dtype=torch.bfloat16)
    counts = count_params(m)
    assert abs(counts["total_params"] / 1e9 - 8.03) < 0.01
    est = estimate(counts, config_dims(cfg), ckpt_gb=16.06)
    assert abs(est["fp16_gb"] - 16.06) < 0.05
    assert abs(est["awq_gb"] - 5.73) < 0.05
    assert abs(est["phases"]["A_quantize"]["vram_gb"] - 14.1) < 0.2
    assert abs(est["phases"]["B_fp16_eval"]["vram_gb"] - 19.3) < 0.1
    assert abs(est["kv_cache_gb"] - 1.07) < 0.01
    assert est["phases"]["A_quantize"]["best"]["gpu"] == "RTX 4090"
```

tests/test_moe.py (25 lines)

```
"""Fused 3-D MoE experts (transformers >= 5 layout) are quantized per expert."""
import json

import torch

from awq.awq_linear import AwqExpertsWeight
from awq.model_io import load_quantized
from awq.pipeline import quantize_model


def test_mixtral_experts_quantized(tiny_mixtral, make_rc, calib_tokens):
    qdir = quantize_model(make_rc(tiny_mixtral), calib_tokens=calib_tokens)
    qc = json.loads((qdir / "quant_config.json").read_text())
    assert qc["n_expert_tensors_quantized"] == 2 * 2          # gate_up_proj + down_proj per layer
    log = [json.loads(line) for line in (qdir.parent / "search_log.jsonl").read_text().splitlines()]
    stats = list(log[0]["experts"].values())[0]
    assert stats["with_tokens"] >= 1
    qm = load_quantized(qdir, "cpu")
    assert any(isinstance(m, AwqExpertsWeight) for m in qm.modules())
    from transformers import AutoModelForCausalLM
    fp = AutoModelForCausalLM.from_pretrained(str(tiny_mixtral), dtype=torch.float32).eval()
    with torch.no_grad():
        a, b = fp(calib_tokens[:2]).logits, qm(calib_tokens[:2]).logits
    assert torch.isfinite(b).all()
    assert ((a - b).norm() / a.norm()).item() < 0.3
```

tests/test_pipeline.py (90 lines)

```
"""End-to-end on tiny models: stream -> AWQ -> pack -> reload, plus resume and AWQ-vs-RTN."""
import json

import pytest
import torch

from awq.awq_linear import AwqLinear
from awq.evaluate import apply_rtn_
from awq.model_io import load_quantized
from awq.pipeline import quantize_model


def _fp(path):
    from transformers import AutoModelForCausalLM
    return AutoModelForCausalLM.from_pretrained(str(path), dtype=torch.float32).eval()


@torch.no_grad()
def _err(a, b):
    return ((a - b).norm() / a.norm()).item()


def test_end_to_end_llama_folded(tiny_llama, make_rc, calib_tokens):
```

```
    rc = make_rc(tiny_llama)
    qdir = quantize_model(rc, calib_tokens=calib_tokens)
    qc = json.loads((qdir / "quant_config.json").read_text())
    assert qc["n_linears_quantized"] == 3 * 7
    assert qc["n_linears_with_inv_s"] == 0, "Llama (MHA) groups should all fold"
    qm = load_quantized(qdir, "cpu")
    assert all(isinstance(m, AwqLinear) for n, m in qm.named_modules() if n.endswith(("_proj",)))
    fp = _fp(tiny_llama)
    with torch.no_grad():
        a, b = fp(calib_tokens[:4]).logits, qm(calib_tokens[:4]).logits
    assert torch.isfinite(b).all()
    assert _err(a, b) < 0.25


def test_awq_beats_rtn_end_to_end(tiny_llama, make_rc, calib_tokens):
    qdir = quantize_model(make_rc(tiny_llama), calib_tokens=calib_tokens)
    qm, fp, rtn = load_quantized(qdir, "cpu"), _fp(tiny_llama), _fp(tiny_llama)
    apply_rtn_(rtn, 4, 64, True)
    with torch.no_grad():
        ref = fp(calib_tokens).logits
        e_awq, e_rtn = _err(ref, qm(calib_tokens).logits), _err(ref, rtn(calib_tokens).logits)
    assert e_awq < e_rtn, (e_awq, e_rtn)


def test_never_fold_keeps_inv_s(tiny_llama, make_rc, calib_tokens):
    qdir = quantize_model(make_rc(tiny_llama, "quant.fold_scales=never"), calib_tokens=calib_tokens)
    qc = json.loads((qdir / "quant_config.json").read_text())
    assert qc["n_linears_with_inv_s"] > 0
    qm = load_quantized(qdir, "cpu")
    with torch.no_grad():
        assert _err(_fp(tiny_llama)(calib_tokens[:2]).logits, qm(calib_tokens[:2]).logits) < 0.25


def test_gqa_tied_qwen(tiny_qwen_gqa, make_rc, calib_tokens):
    qdir = quantize_model(make_rc(tiny_qwen_gqa), calib_tokens=calib_tokens)
    qc = json.loads((qdir / "quant_config.json").read_text())
    assert qc["tie_word_embeddings"] is True
    groups = [json.loads(line) for line in (qdir.parent / "search_log.jsonl").read_text().splitlines()]
    o_proj = [g for g in groups[0]["groups"] if g["linears"] == ["self_attn.o_proj"]][0]
    assert o_proj["mode"].startswith("no-scale"), "GQA o_proj cannot fold into v_proj"
    qm = load_quantized(qdir, "cpu")
    assert qm.lm_head.weight is qm.model.embed_tokens.weight
    with torch.no_grad():
        assert _err(_fp(tiny_qwen_gqa)(calib_tokens[:2]).logits, qm(calib_tokens[:2]).logits) < 0.25


def test_resume_gives_identical_result(tiny_llama, make_rc, calib_tokens, monkeypatch):
    import awq.pipeline as P

    rc = make_rc(tiny_llama)
    full = quantize_model(rc, calib_tokens=calib_tokens)
    ref = {f.name: f.read_bytes() for f in full.glob("block-*.safetensors")}
    for f in full.glob("block-*.safetensors"):
        f.unlink()

    real = P.quantize_block
    def crash_on_block_2(block, idx, *a, **k):
        if idx == 2:
            raise RuntimeError("simulated pod reclaim")
        return real(block, idx, *a, **k)
    monkeypatch.setattr(P, "quantize_block", crash_on_block_2)
    with pytest.raises(RuntimeError):
        quantize_model(rc, calib_tokens=calib_tokens)
    monkeypatch.setattr(P, "quantize_block", real)
    quantize_model(rc, resume=True, calib_tokens=calib_tokens)
    got = {f.name: f.read_bytes() for f in full.glob("block-*.safetensors")}
    assert got == ref
```

tests/test_plugin.py (25 lines)

```
from pathlib import Path

import torch

from awq.awq_linear import AwqLinear
from awq.model_io import load_quantized
from awq.pipeline import quantize_model
```

```
PLUGIN = str(Path(__file__).with_name("tiny_plugin.py"))


def test_custom_model_via_plugin(make_rc):
    rc = make_rc("custom-tiny-lm", f"plugin={PLUGIN}")
    qdir = quantize_model(rc)
    qm = load_quantized(qdir)
    assert sum(isinstance(m, AwqLinear) for m in qm.modules()) == 4      # up/down x 2 blocks, head stays FP
    import importlib.util
    spec = importlib.util.spec_from_file_location("tp", PLUGIN)
    tp = importlib.util.module_from_spec(spec)
    spec.loader.exec_module(tp)
    fp, _ = tp.load()
    x = torch.randint(0, 300, (2, 32))
    with torch.no_grad():
        a, b = fp(x), qm(x)
    assert ((a - b).norm() / a.norm()).item() < 0.2
```

tests/test_quantizer.py (54 lines)

```
import torch
import torch.nn as nn
import torch.nn.functional as F

from awq.awq_linear import AwqLinear
from awq.quantizer import compute_qparams, dequantize, fake_quant, pack_int4, quantize, unpack_int4


def test_pack_unpack_roundtrip_is_exact():
    q = torch.randint(0, 16, (64, 256), dtype=torch.uint8)
    p = pack_int4(q)
    assert p.shape == (64, 128) and p.dtype == torch.uint8
    assert torch.equal(unpack_int4(p), q)


def test_error_is_bounded_by_half_a_step():
    torch.manual_seed(0)
    w = torch.randn(32, 256)
    for zp in (True, False):
        scales, zeros = compute_qparams(w, 4, 128, zp)
        q = quantize(w, scales, zeros, 4, 128)
        assert q.min() >= 0 and q.max() <= 15
        w_hat = dequantize(q, scales, zeros, 128, torch.float32)
        step = scales.repeat_interleave(128, dim=1)
        assert ((w - w_hat).abs() <= step / 2 + 1e-6).all(), f"zero_point={zp}"


def test_per_channel_group():
    w = torch.randn(8, 96)
    out = fake_quant(w, 4, -1, True)
    assert out.shape == w.shape and torch.isfinite(out).all()


def test_awq_linear_matches_fake_quant():
    torch.manual_seed(0)
    lin = nn.Linear(256, 64, bias=True)
    x = torch.randn(10, 256)
    m = AwqLinear.from_float(lin, 4, 128, True)
    ref = F.linear(x, fake_quant(lin.weight.data, 4, 128, True), lin.bias)
    assert torch.allclose(m(x), ref, atol=1e-5)
    # storage really is 4-bit: 0.5 byte per weight
    assert m.qweight.numel() * m.qweight.element_size() == 64 * 256 // 2


def test_awq_linear_with_inv_s_undoes_the_scale():
    torch.manual_seed(0)
    lin = nn.Linear(128, 32, bias=False)
    s = torch.rand(128) + 0.5
    scaled = nn.Linear(128, 32, bias=False)
    scaled.weight.data = lin.weight.data * s
    m = AwqLinear.from_float(scaled, 4, 64, True, inv_s=1 / s)
    x = torch.randn(4, 128)
    expected = F.linear(x / s, fake_quant(lin.weight.data * s, 4, 64, True))
    assert torch.allclose(m(x), expected, atol=1e-5)
```

tests/test_scaling.py (70 lines)

```
"""The AWQ identity and the scale folding must be EXACT before any rounding happens."""
import torch

from awq.pipeline import _fold


def test_scaling_identity_exact():
    torch.manual_seed(0)
    x = torch.randn(64, 128, dtype=torch.float64)
    w = torch.randn(32, 128, dtype=torch.float64)
    s = torch.rand(128, dtype=torch.float64) * 10 + 0.1
    assert torch.allclose((x / s) @ (w * s).T, x @ w.T, atol=1e-10)


def _llama_block():
    from transformers import LlamaConfig
    from transformers.models.llama.modeling_llama import LlamaDecoderLayer, LlamaRotaryEmbedding

    cfg = LlamaConfig(hidden_size=64, intermediate_size=128, num_attention_heads=4, num_key_value_heads=4,
                      num_hidden_layers=1, vocab_size=100)
    torch.manual_seed(0)
    blk = LlamaDecoderLayer(cfg, 0).double().eval()
    for p in blk.parameters():
        p.data.normal_(0, 0.2)
    rope = LlamaRotaryEmbedding(cfg)
    return blk, rope


def _run(blk, rope, x):
    pos = torch.arange(x.shape[1]).unsqueeze(0)
    cos, sin = rope(x.float(), pos)
    out = blk(x, position_embeddings=(cos.double(), sin.double()), position_ids=pos)
    return out[0] if isinstance(out, tuple) else out


def test_folding_keeps_block_output_identical():
    blk, rope = _llama_block()
    x = torch.randn(2, 8, 64, dtype=torch.float64)
    ref = _run(blk, rope, x)
    groups = {
        ("self_attn.q_proj", "self_attn.k_proj", "self_attn.v_proj"): ("norm", "input_layernorm"),
        ("mlp.gate_proj", "mlp.up_proj"): ("norm", "post_attention_layernorm"),
        ("mlp.down_proj",): ("linear_out", "mlp.up_proj"),
        ("self_attn.o_proj",): ("linear_out", "self_attn.v_proj"),
    }
    for names, target in groups.items():
        in_f = blk.get_submodule(names[0]).in_features
        s = torch.rand(in_f, dtype=torch.float64) * 4 + 0.25
        for n in names:
            blk.get_submodule(n).weight.data *= s
        _fold(blk, target, s)
    assert torch.allclose(_run(blk, rope, x), ref, atol=1e-9)


def test_gemma_style_norm_offset_fold():
    class OffsetNorm(torch.nn.Module):
        def __init__(self):
            super().__init__()
            self.weight = torch.nn.Parameter(torch.randn(16, dtype=torch.float64) * 0.1)

        def forward(self, x):
            return x * (1 + self.weight)

    blk = torch.nn.Module()
    blk.norm = OffsetNorm()
    x = torch.randn(4, 16, dtype=torch.float64)
    s = torch.rand(16, dtype=torch.float64) + 0.5
    ref = blk.norm(x) / s
    _fold(blk, ("norm_offset", "norm"), s)
    assert torch.allclose(blk.norm(x), ref, atol=1e-12)
```

tests/test_search.py (43 lines)

```
import torch

from awq.quantizer import fake_quant
from awq.search import apply_clip, search_clip, search_scale


def _outlier_data(seed=0):
    g = torch.Generator().manual_seed(seed)
    x = torch.randn(2048, 256, generator=g)
    x[:, [5, 77, 130]] *= 40.0                    # ~1% salient channels, like real LLMs
    w = torch.randn(128, 256, generator=g) * 0.05
    return x, w


def test_awq_beats_rtn_on_salient_channels():
    x, w = _outlier_data()
    res = search_scale([w], [None], x, 4, 128, True, n_grid=20)
    assert res["ratio"] is not None, "AWQ should find a scale better than plain RTN here"
    assert res["loss"] < 0.7 * res["loss_rtn"], (res["loss"], res["loss_rtn"])
    # measure independently: Q(W*s)/s on fresh data from the same distribution
    xt, _ = _outlier_data(seed=1)
    s = res["scales"]
    err_awq = (xt @ (w - fake_quant(w * s, 4, 128, True) / s).T).pow(2).mean()
    err_rtn = (xt @ (w - fake_quant(w, 4, 128, True)).T).pow(2).mean()
    assert err_awq < err_rtn


def test_search_never_worse_than_rtn():
    torch.manual_seed(3)
    x, w = torch.randn(512, 128), torch.randn(64, 128)     # no outliers: RTN may already be optimal
    res = search_scale([w], [None], x, 4, 64, True, n_grid=10)
    assert res["loss"] <= res["loss_rtn"] + 1e-12


def test_clip_search_does_not_increase_error():
    x, w = _outlier_data()
    w = w.clone()
    w[0, 3] = 3.0                                           # one outlier weight stretches its group
    mv = search_clip(w, x, 4, 128, True, clip_grid=20, max_shrink=0.5, n_tokens=512)
    wc = apply_clip(w, mv)
    err_clip = (x @ (w - fake_quant(wc, 4, 128, True)).T).pow(2).mean()
    err_none = (x @ (w - fake_quant(w, 4, 128, True)).T).pow(2).mean()
    assert err_clip <= err_none * 1.001
```

tests/tiny_plugin.py (47 lines)

```
"""Example plugin: a tiny custom (non-transformers) language model. Used by tests/test_plugin.py.

Copy this file as a template for your own model: implement load(), blocks(), calib_batches().
"""
import torch
import torch.nn as nn


class Block(nn.Module):
    def __init__(self, d):
        super().__init__()
        self.norm = nn.LayerNorm(d)
        self.up = nn.Linear(d, 4 * d)
        self.down = nn.Linear(4 * d, d)

    def forward(self, h):
        return h + self.down(torch.relu(self.up(self.norm(h))))


class TinyLM(nn.Module):
    def __init__(self, vocab=300, d=64, n=2):
        super().__init__()
        self.emb = nn.Embedding(vocab, d)
        self.layers = nn.ModuleList(Block(d) for _ in range(n))
        self.head = nn.Linear(d, vocab)

    def forward(self, x):
        h = self.emb(x)
```

```
        for layer in self.layers:
            h = layer(h)
        return self.head(h)


def load():
    torch.manual_seed(0)
    return TinyLM().eval(), None


def blocks(model):
    return model.layers


def calib_batches(tokenizer, n_samples, seq_len, batch_size):
    g = torch.Generator().manual_seed(0)
    data = torch.randint(0, 300, (n_samples, seq_len), generator=g)
    for i in range(0, n_samples, batch_size):
        yield {"x": data[i:i + batch_size]}
```